INDEX

S. No	Topic	Page No.
	Week 1	
1	Biological Neuron	1
2	From Spring to Winter of AI	8
3	The Deep Revival	18
4	From Cats to Convolutional Neural Networks	25
5	Faster, higher, stronger	29
6	The Curious Case of Sequences	31
7	Beating humans at their own games (literally)	38
8	The Madness (2013-)	41
9	(Need for) Sanity	50
10	Motivation from Biological Neurons	54
11	McCulloch Pitts Neuron, Thresholding Logic	62
12	Perceptrons	73
13	Error and Error Surfaces	82
14	Perceptron Learning Algorithm	85
15	Proof of Convergence of Perceptron Learning Algorithm	98
	Week 2	
16	Deep Learning(CS7015): Linearly Separable Boolean Functions	108
17	Deep Learning(CS7015): Representation Power of a Network of Perceptrons	113
18	Deep Learning(CS7015): Sigmoid Neuron	124
19	Deep Learning(CS7015): A typical Supervised Machine Learning Setup	136
20	Deep Learning(CS7015): Learning Parameters: (Infeasible) guess work	146
21	Deep Learning(CS7015): Learning Parameters: Gradient Descent	157
22	Deep Learning(CS7015): Representation Power of Multilayer Network of Sigmoid Neurons	179
	Week 3	
23	Feedforward Neural Networks (a.k.a multilayered network of neurons)	209
24	Learning Paramters of Feedforward Neural Networks (Intuition)	222
25	Output functions and Loss functions	229
26	Backpropagation (Intuition)	244
27	Backpropagation: Computing Gradients w.r.t. the Output Units	255
28	Backpropagation: Computing Gradients w.r.t. Hidden Units	267
29	Backpropagation: Computing Gradients w.r.t. Parameters	279
30	Backpropagation: Pseudo code	287

31	Derivative of the activation function	292
32	Information content, Entropy & cross entropy	294

Week 4

33	Recap: Learning Parameters: Guess Work, Gradient Descent	312
34	Contours Maps	324
35	Momentum based Gradient Descent	333
36	Nesterov Accelerated Gradient Descent	346
37	Stochastic And Mini-Batch Gradient Descent	355
38	Tips for Adjusting Learning Rate and Momentum	368
39	Line Search	377
40	Gradient Descent with Adaptive Learning Rate	383
41	Bias Correction in Adam	404

Week 5

42	Eigenvalues and Eigenvectors	412
43	Linear Algebra : Basic Definitions	424
44	Eigenvalue Decompositon	434
45	Principal Component Analysis and its Interpretations	442
46	PCA : Interpretation 2	465
47	PCA : Interpretation 3	477
48	PCA : Interpretation 3 (Contd.)	482
49	PCA : Practical Example	484
50	Singular Value Decomposition	491

Week 6

51	Introduction to Autoncoders	510
52	Link between PCA and Autoencoders	543
53	Regularization in autoencoders (Motivation)	556
54	Denoising Autoencoders	564
55	Sparse Autoencoders	582
56	Contractive Autoencoders	590

Week 7

57	Bias and Variance	599
58	Train error vs Test error	608
59	Train error vs Test error (Recap)	624
60	True error and Model complexity	631
61	L2 regularization	639
62	Dataset augmentation	656

63	Parameter sharing and tying	660
64	Adding Noise to the inputs	662
65	Adding Noise to the outputs	667
66	Early stopping	671
67	Ensemble Methods	679
68	Dropout	686

Week 8

69	A quick recap of training deep neural networks	700
70	Unsupervised pre-training	706
71	Better activation functions	726
72	Better initialization strategies	752
73	Batch Normalization	774

Week 9

74	One-hot representations of words	786
75	Distributed Representations of words	793
76	SVD for learning word representations	802
77	SVD for learning word representations (Contd.)	803
78	Continuous bag of words model	814
79	Skip-gram model	839
80	Skip-gram model (Contd.)	846
81	Contrastive estimation	856
82	Hierarchical softmax	861
83	GloVe representations	871
84	Evaluating word representations	877
85	Relation between SVD and Word2Vec	885

Week 10

86	The convolution operation	888
87	Relation between input size, output size and filter size	907
88	Convolutional Neural Networks	919
89	Convolutional Neural Networks (Contd.)	935
90	CNNs (success stories on ImageNet)	951
91	CNNs (success stories on ImageNet) (Contd.)	972
92	Image Classification continued (GoogLeNet and ResNet)	974
93	Visualizing patches which maximally activate a neuron	989
94	Visualizing filters of a CNN	993
95	Occlusion experiments	999

96	Finding influence of input pixels using backpropagation	1003
97	Guided Backpropagation	1008
98	Optimization over images	1012
99	Create images from embeddings	1022
100	Deep Dream	1032
101	Deep Art	1039
102	Fooling Deep Convolutional Neural Networks	1044

Week 11

103	Sequence Learning Problems	1050
104	Recurrent Neural Networks	1057
105	Backpropagation through time	1067
106	The problem of Exploding and Vanishing Gradients	1080
107	Some Gory Details	1089
108	Selective Read, Selective Write, Selective Forget - The Whiteboard Analogy	1095
109	Long Short Term Memory(LSTM) and Gated Recurrent Units(GRUs)	1103
110	How LSTMs avoid the problem of vanishing gradients	1126
111	How LSTMs avoid the problem of vanishing gradients (Contd.)	1133

Week 12

112	Introduction to Encoder Decoder Models	1152
113	Applications of Encoder Decoder models	1169
114	Attention Mechanism	1185
115	Attention Mechanism (Contd.)	1205
116	Attention over images	1207
117	Hierarchical Attention	1216

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture - 04
Feed forward Neural Networks, Backpropagation

So, welcome to lecture 4 of CS7015, the course on deep learning. Today, we will talk about feed forward neural networks and back propagation. So, quick recap of the story; so far it, so, we started with mp neurons. We saw there were some problems with the mp neurons. They could handle only Boolean inputs and Boolean outputs and threshold needed to be hard coded. So, from there, we moved on to perceptrons which allowed for real inputs real outputs and sorry real inputs and binary outputs. And we also learned an algorithm for learning these weights and parameters right? So, we need there was no need to hand code these parameters anymore.

But then, we found that, for a single perceptron, there is a limitation. It cannot; it can only deal with functions which are linearly separable. So, then we went on to a multi-layer network of perceptrons and we proved by illustration that, it can handle any arbitrary Boolean function; whether linearly separable or not, the catch is that you will need a large number of neurons in the hidden layer, right? Then we also observed that perceptrons have this harsh thresholding logic. So, which makes the decisions very unnatural; it is 0.49, it is negative; 0.51 is positive; so, you wanted something more smooth.

So, the smoothest approximation to this step function which is the perceptron function was a sigmoid function. Sigmoid is a family of functions and we saw one such function which was logistic function. And then, we saw that, it is very smooth; now, it is continuous and differentiable.

Now, for the sigmoid neuron on a single sigmoid neuron we saw a learning algorithm which was gradient descent. And, we proved principally that it will always go in the direction where the loss decreases right; so, that is what is the basis for gradient descent.

And then, we graduated from a single neuron to a network of neurons and made a case that such a network of neurons with enough neurons in the hidden layer can approximate

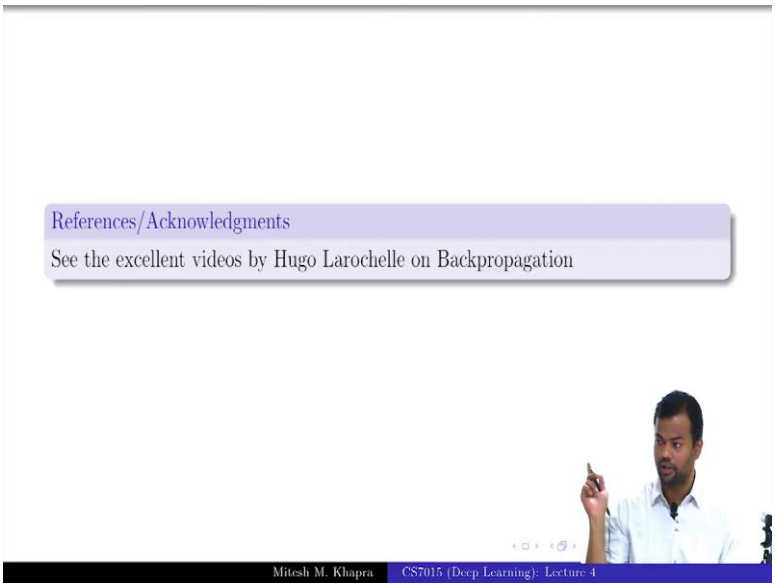
any arbitrary function right, ok. So, I have told you that, it can approximate any arbitrary function. What does that mean? And what is the thing in the network that does all this? All the tower functions and the tower functions depend on weights and biases. So, there, in that illustrative proof, again we were adjusting the weights and biases by hand right? We knew that we wanted these very tiny tower functions and we were doing it.

Now, from there, where should we go?

Student: (Refer Time: 02:39).

We need an algorithm to learn these weights and biases right. So, that is what back propagation is. So, today I am going to formalize these feed forward neural networks. We just did it by illustration the other day. I will introduce you to the terminology and see what the input outputs are and so on. And then, we will look at an algorithm for learning the weights in this feed forward neural network.

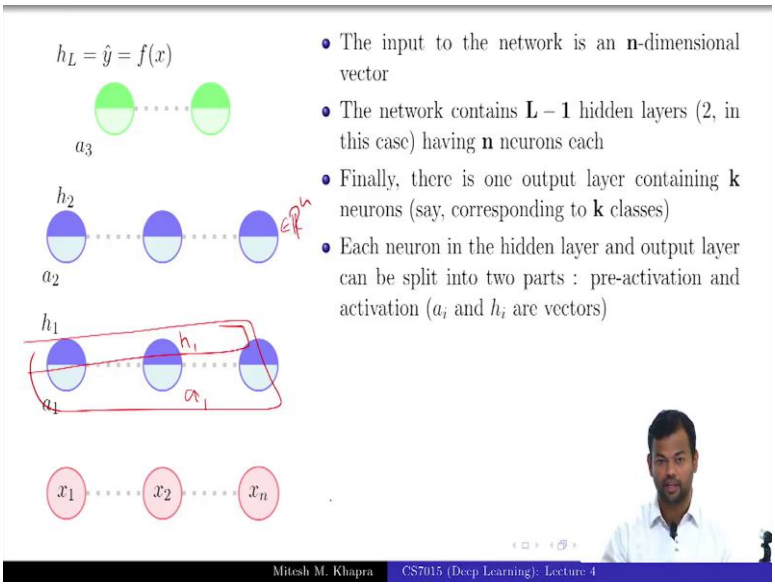
(Refer Slide Time: 03:03)



Let us begin. So this, a lot of this material is inspired by the video lectures by Hugo Larochelle on back propagation. He has a course on neural networks.

It is available on YouTube; you can check it, ok. So, let us first begin by introducing feed forward neural network, right?

(Refer Slide Time: 03:18)



So, what is a feed forward neural network? The input to the network is an n-dimensional vector so, ok that means, my input belongs to Rn, that fine. The network contains L minus 1 hidden layers. Where do you already know what hidden layers are right? We have been defining that terminology since multi layered perceptron. So, you have these hidden layers and there are L minus 1 of these and then it has one output layer containing k neurons, ok. Those are the feed forward neural network looks like. What is missing here?

Student: (Refer Time: 03:57).

The weights, right.

So, each neuron in the hidden layer, ok, before that each neuron in the hidden layer and the output layer can be split into 2 parts right? So, I will call the first part as the pre activation and the second part as the activation. Have you seen this plate before right? What does the pre activation do?

Student: Aggregation.

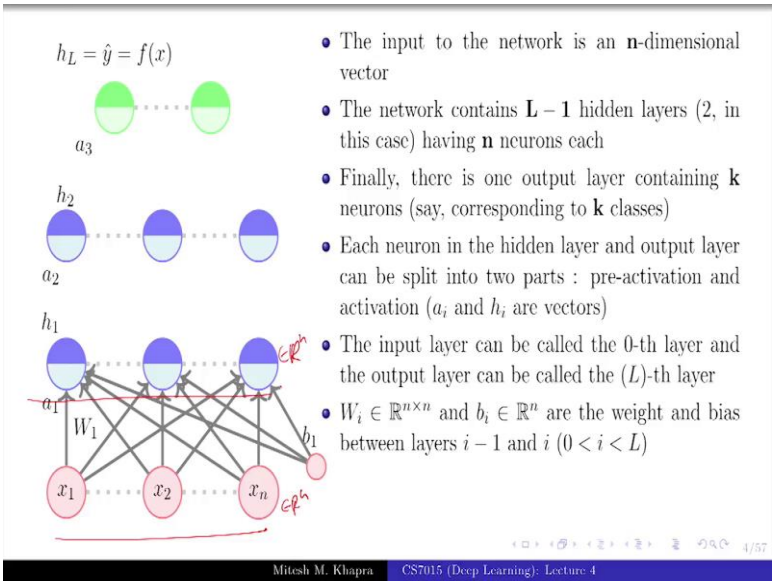
Aggregation and what does the activation do?

Student: Non-linearity.

Non-linearity, right? So, we have this pre activation and activation at every layer and a i and h i are vectors. Is that correct? Because, this entire thing or rather this part is h 1 and this part is a 1. Both of these are vectors, right? And for this discussion, am going to assume that, everything till here belongs to R n.

So, the input was R n and all the hidden layers also have n neutrons. Is that fine? So, please pay a lot of attention to this couple of slides because, this is going to stay with us for the rest of the lecture and perhaps 2 more lectures and even for the course alright. So, this is very important that you understand this; the way we are defining a feed forward neuron network.

(Refer Slide Time: 05:08)

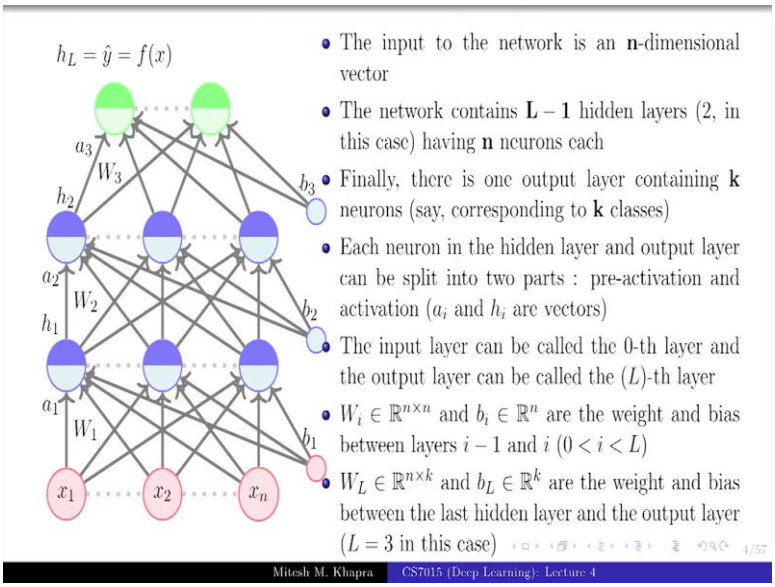


The input layer can be called as 0th layer. What I mean by that is that, I could refer to this as h 0, ok. There is no a 0 h 0 here because, there is no pre activation, activation. You are just given the input. So, I just call it as h 0 ok. And the last layer can be called as h of L, right? Whatever you get from this green part, you will call it as h of L. Ok, what is the dimension of h of L? R raised to k. It belongs to R k, because I have said here that, you have k neurons, each corresponding to k classes, ok.

Now, we have weights between the input layer and the first hidden layer. Now, can you tell me this belongs to R n. This also belongs to R n. So, what is the dimension of W 1? n cross n, right? Because it contains weights for connecting each of these inputs to each of these hidden layers; there are n here n there right? So, it is n cross n.

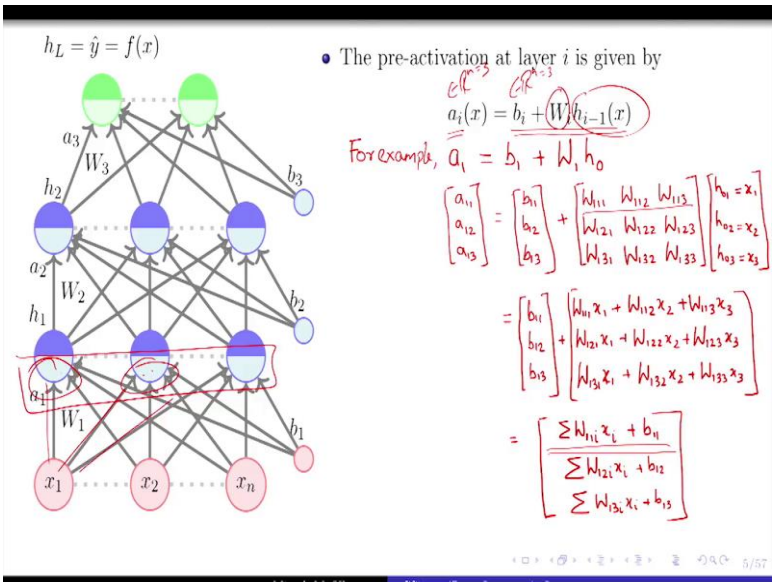
And what are the dimensions of the bias? n. One corresponding to each of the hidden inputs, fine. And this is only for up to this layer because, till here I have assumed everything is n.

(Refer Slide Time: 06:23)



Now, what about the output layer? n cross k and the biases k, k dimensional ok. So, this is what the network looks like. But now, I have to give you some function. So, I have just I have shown you a diagram, but what does it mean mathematically? Because, remember that, we are always interested in writing something of the form y is equal to function of x, right. And that is not well defined yet.

(Refer Slide Time: 06:49)



So, let us start defining that. Ignore the red portion for now, ok. I will go over it. So, each of these activations right or rather the pre activations is given by b i plus W i into h i minus 1. So, what it means is that, these activations take inputs from the previous layer, multiply by them by weights and also add the bias. Is that clear? So, let us see it, right. For example, if I look at a 1 which is this vector. So, that is 3 dimensional and assuming it is 3 dimensional for simplicity.

So, it is a 1 1, a 1 1, a 1 2, a 1 3 right? And that is equal to how do you get rid off this? b 1 1 b 1 2 b 1 3 plus this matrix multiplication is this clear to everyone. I know it is trivial, but am still going over it right. So, let us not ok. And then, how do you do this matrix multiplication? Row was multiplied by the column. So, this is what you get right? And in the end, I can write it as this, right? And this looks very similar to what we have been seeing throughout it from a mp neuron to perceptron to sigmoid neuron and now this case, right?

So, it is just an aggregation of all your inputs or weighted aggregation of all your inputs. That is the case which I want it to know; and that is obvious now. So, you understand what these are right?

So, this is R n. In our case, we have assumed n equal to 3. What is this? I will keep asking till this is completely fine with everyone. R n and this is?

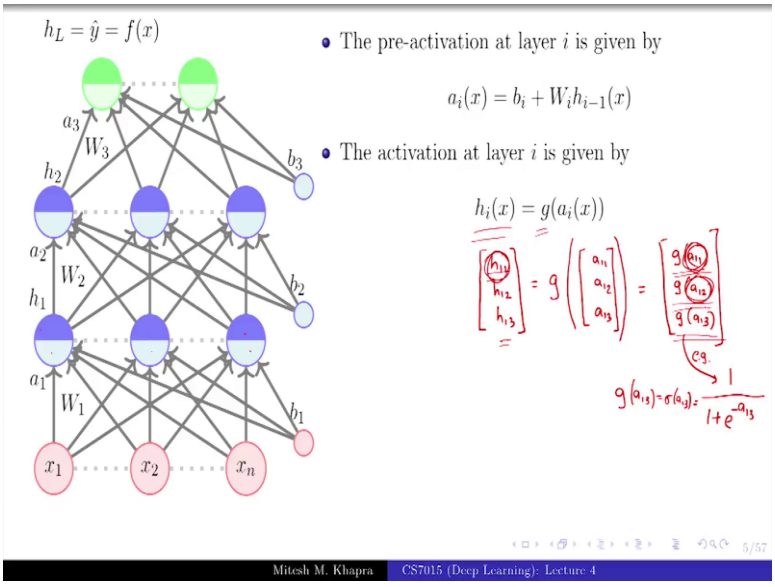
Student: (Refer Time: 08:30).

n cross n and this is?

Student: (Refer Time: 08:34).

n cross 1 n cross n I mean R n sorry is it fine? So, everyone understands the operation happening here. It is a weighted aggregation of your inputs. So, every guy here is a weighted aggregation of all the inputs, ok.

(Refer Slide Time: 08:47)

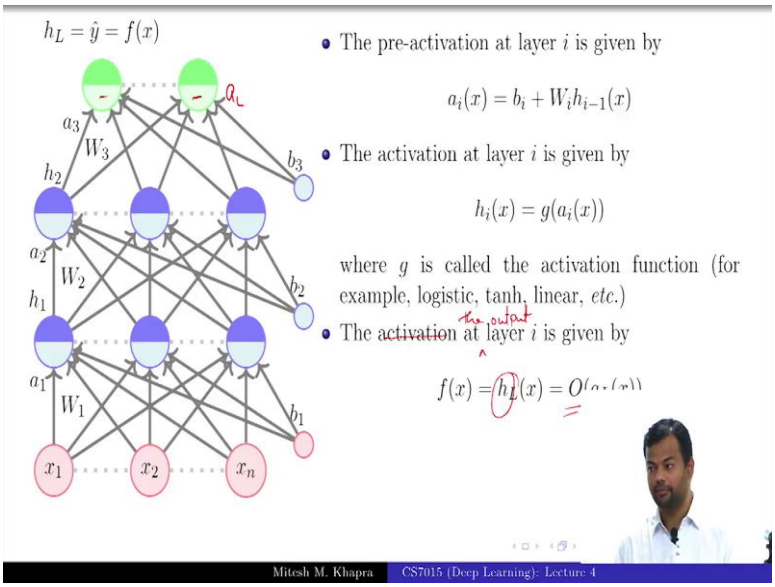


Now, after that I do h i of x is some function of a i of x, ok. What does this mean? So, this is again a vector, right? I have assumed that, it is 3 dimensional. So, these are the 3 elements of h i; so, these are the 3 guys. Now, these are some function of these light blue guys, ok. Now, how does that function operate on the vector? It operates element wise; not all functions on vectors are element wise. But this particular function, we are going to do element wise.

That means, that h 1 1 is equal to g of a 1 1, h 1 2 is equal to g of a 2 and h 1 3 is equal to g of a 1 3 right, where if I take g of a 1 3, one of the functions that I could choose is the sigmoid function. So, it would just be 1 over 1 plus e raised to minus here. So, what is happening is I am taking this value and passing it to the sigmoid function to get oh sorry am taking this value and passing it to the sigmoid function to get h 1 1 taking this value passing it to the sigmoid function to get h 1 2 right.

So, the key thing to understand here that, this is a element wise operation, right? It is not operating on the vector. That does not make sense. It is operating on every element of the vector, right ok. And g is called the activation function.

(Refer Slide Time: 10:02)

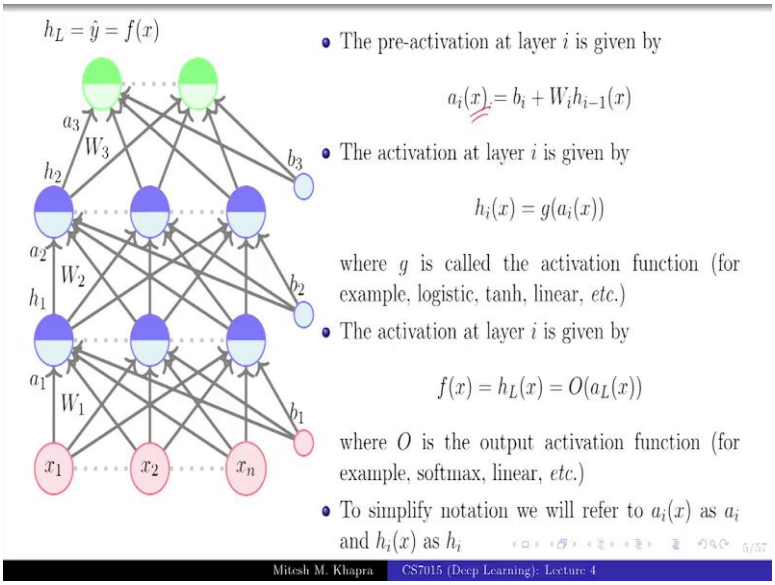


It could be logistic, tanh, linear anything right. So, we will see some of these functions later on ok.

Now, the activation at layer i sorry they are supposed to be activation at the output layer. The activation at the output layer is given by the final function which is f of x is equal to O of a of. So, let us see; so, this is a 3. In our case, L was equal to 3 because, we had L minus 1 hidden layers and the Lth layer was the output layer right; so, this is a L. So, this is what I have computed here; that light green part of the figure that you see right now, based on that, I want to produce an output.

So, that is, someone had asked me a question that why do we always choose sigmoid? Because, sigmoid will clamp the output to 0 to 1. What if I want to predict the amount of oil which will not be between 0 to 1, right? That is why, for the output, we will use a special function that will call the output function and later on, I will show you that it depends on the task at hand. So, it is going to change with the task that we are going to do, right? So, we are just going to say that, the final output which is h of L is equal to some function of the pre activation at that layer. Is this terminology clear to everyone? How is each function operating, is that clear to everyone?

(Refer Slide Time: 11:22)



And we will see some examples of the output activation function right? Now just for simplicity am going to remove the x's from the brackets right? So, instead of calling everything ai of x hi minus of x and so on. I will just call them ai, hi and so and so, that just simplifies things, but we know that everything is a function of x. Because, x is the input and that passes through some functions and we get the final output, right. So, this is the notations that we are going to use. Is the dimension of everything that you see every variable that you see here completely, clear to everyone?

Dimension of ai, bi, W, hi, x, everything is clear ok. And the output layer has a slightly different dimension than the other layers because, there we have k classes as opposed to n neurons everywhere else, ok, fine. Now, I need to put this in the paradigm that, we saw for supervised machine learning. What were the five components there? Data.

Student: Model.

Model.

Student: Parameters.

Parameters.

Student: Learning.

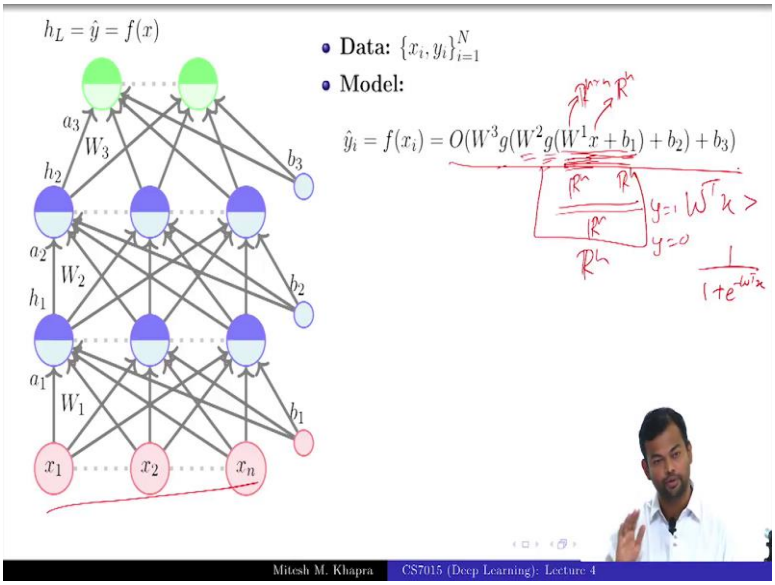
Learning algorithm.

Student: (Refer Time: 12:29).

Objective function right? Ok. Everyone remembers that ok?

So, I said that, we will do deep neural networks and we are trying to write this y hat as a function of x, but then, what I gave you is just a diagram from which this is not clear whether y hat is actually a function of x. How many of you think y hat is actually a function of x? Very few, ok.

(Refer Slide Time: 12:52)



So, let us see what exactly is our model assumption here, right? So, the question let me repeat the question just to be clear. So, I said that they are given some data we do not know the true relation between y and x we make an assumption that y is related to x using some function f right and it is has some parameters and then we like to try to learn the parameters of that function. So, what is the function here?

So, what is your model? What have you assumed as the model? Can you write y as a function of x? If yes, what is that function? How many of you have the answer? I think you have your answer, ok. I think I cannot wait more. So, I will give you the answer. Then it will become very obvious ok. So, this is how y is a function of x, right? So, let us see what is happening. I took the original x which was this, I transformed it, added b 1 that was the dash at layer 1.

Student: (Refer Time: 13:56).

No, this thing.

Student: (Refer Time: 14:00).

Pre-activation at layer 1; I passed it through the activation function right, ok.

Now, again, let us be clear about the dimensions? What is the dimension of this?

Student: n.

n. What is the dimension of this? n cross n. So, what is the dimension of this product?

Student: (Refer Time: 14:21).

n. What about this? So, what is the product the final dimension of this?

R n. Now, you are passing it through a function g that function is operating element wise. So, what is the output dimension?

Student: R n.

R n. So, this is again R n, ok. Now this.

Student: (Refer Time: 14:42).

So, now you see the whole story, right. So, now, this n cross n guy multiplies with this n guy again. You get a vector again pass it through a non-linearity was it so hard? it is obvious now, right. You just take an x; just note down all the transformations that you have done. That is what a function does right. It passes it through the through first a linear transformation; this is a linear transformation, then a non-linear transformation, then again linear non-linear and so on.

So, just see how far we have come from where we started off, right? We started off with simple things like W transpose x, right? That was the perceptron model where we were taking decisions based on W transpose x and we were saying y is equal to 1. If this quantity is greater than something, y is equal to 0; if this quantity is greater than something right, that is why, we started off with we made it slightly more complicated by doing this. This was sigmoid neuron.

Now, from there, where have we gone to this right? So, we have increased the complexity of the network with great complicity. Complexity comes great?

Student: (Refer Time: 15:46).

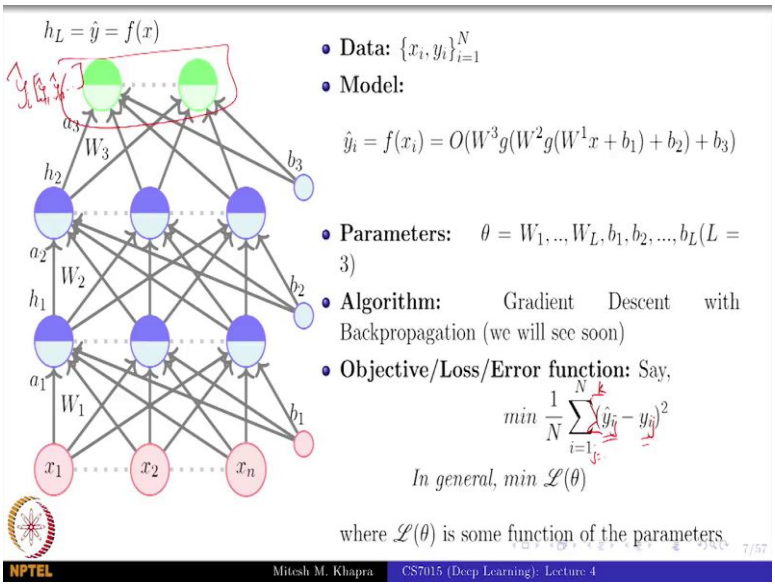
No power, right? We have already seen the representation power of deep neural networks, right. So, it comes from this complexity that you have you have a lot of linear and non-linear transformations, right? That adds to the complexity of the network. It has more parameters at each linear transformation you have some parameters and you are also using a lot of non-linearity. So, that is the reason why deep neural networks are so, powerful right do you get that? Ok, So just to impress again, right?

So, any machine learning algorithm that you have you should be able to write it in this form right, that y is a function of x with some parameters and then your job boils down to learning these parameters, right. It just happens that here, y is a very complex function of the inputs. Is that clear? Ok. So, I am not deviated from the original story. I am still being able to write y as a function of x with some parameters, ok. What are the parameters?

Student: (Refer Time: 16:42).

All the W's, all the b's, right. So, W 1 to W L and b 1 to b L.

(Refer Slide Time: 16:44)



And the algorithm that we are going to see today for learning these parameters is called Gradient descent, but we will use it with back propagation, where back propagation will help us to compute gradients. It is ok; it does not, it does not make sense at this point. That is what the lecture is supposed to be about, right. So, and what is an objective function?

Student: (Refer Time: 17:07).

Loss function. So, I could just go with this loss function, right ok. There is an error here. I thought we corrected this; there is a summation. So, actually these are vectors, right. So, this does not make sense. So, you should have summation j equal to 1 to k yij minus yij. Does that make sense? So, this is the vector y hat ok. For the i th example, it will be called as y hat high i which will have k elements, right. So, y hat i 1 y hat i 2 up to y hat i k, right.

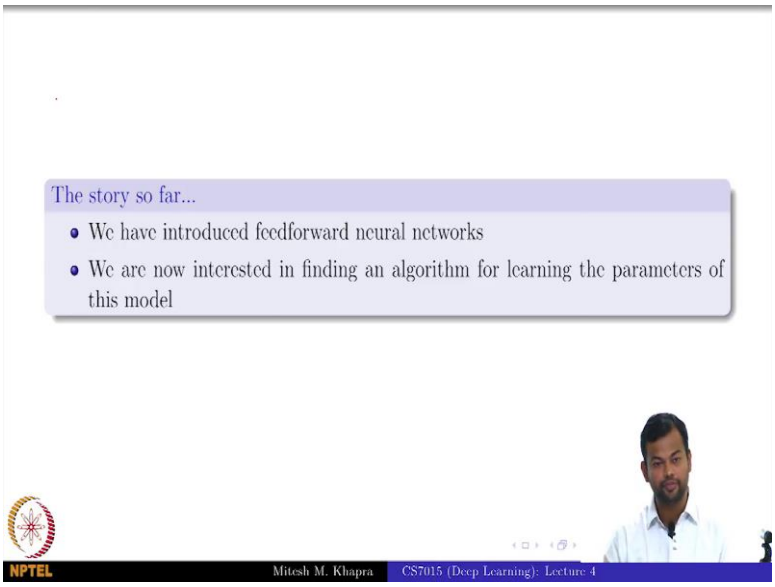
So, that is what my predictions are and I will have the corresponding true vector also. I am trying to take the difference between them which is going to be an element wise difference. Everyone understands the error in the slide? How many of you do not get it? How many of you get it? If you do not get it, please raise your hands. It is a minor thing. I can correct it. And how does deep neural networks fit into these this paradigm?

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module - 4.2
Lecture - 04
Learning Parameters of Feedforward Neural Networks (Intuition)

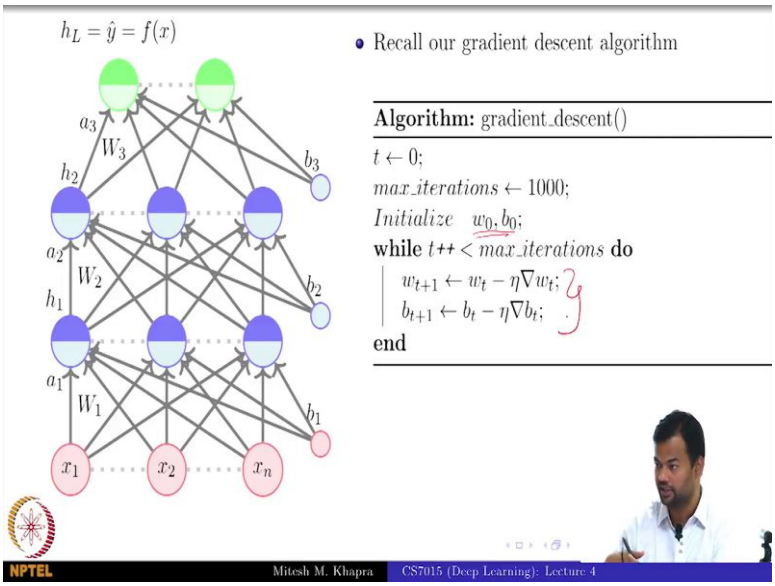
Now, we will move on to the next module, where we want to learn the parameters of feed forward neural networks, and we first start with some intuition, and then mathematical details.

(Refer Slide Time: 00:26)



So, we have introduced feed forward neural networks, and we are now interested in finding an algorithm which can allow us to learn the weights of this network.

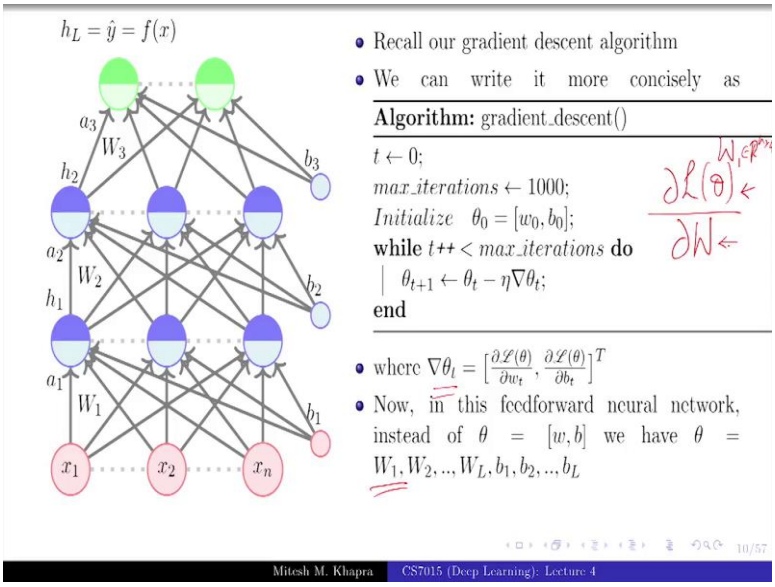
(Refer Slide Time: 00:33)



So, recall our gradient descent algorithm, this is how it looked ok, I had initialized those 2 parameters w naught b naught. And then I was iteratively doing this in a loop, at every step I was moving in a direction opposite to the gradient at that step.

Now, can I write this a bit more compactly? We can write using vectors.

(Refer Slide Time: 00:59)



So, are you ok if I write it this way? So, these 2 was actually nothing but vector at every point? So, I can just write it this way. So, theta is the vector containing w and b, ok? or theta is the vector of all the parameters my network had it just. So, happened that

network had only 2 parameters. So, see where am going with this? How many of you see where am going with this?

Good, so where delta theta t right just to remind you it was this the partial collection of all the partial derivatives with respect to all the parameters. In this toy example all was equal to 2, right we just had 2 parameters. Now you see where am going with this, ok. So now, in this feed forward neural network, instead of theta equal to w comma b what do we have? Theta is equal to so many parameters, ok. So, what would grad of theta t now be partial derivatives with respect to?

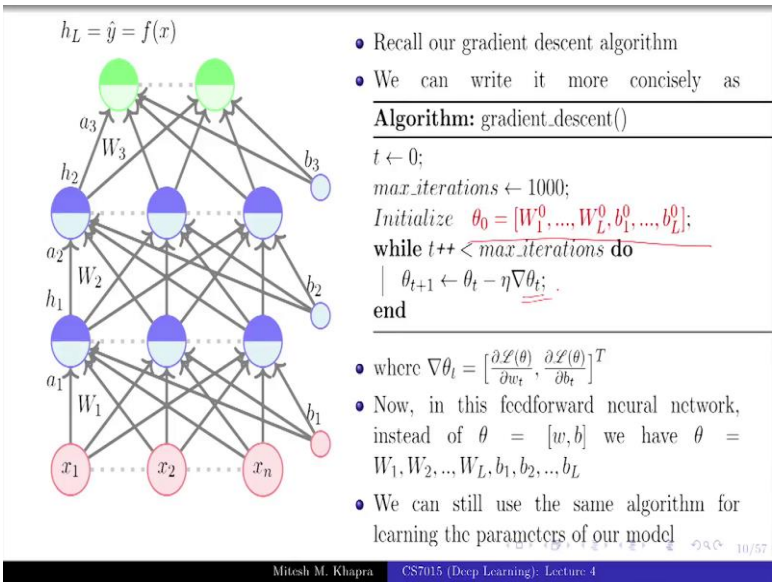
Student: (Refer Time: 01:58).

All the weights, but there is a problem here right, this is the matrix, how do you take the partial derivative with respect to the matrix? Who asked you to use the matrix? How you take the partial derivatives with respect to matrix. So, what I am interested in this right the question. I know there is some loss function which is a function of theta. One of the elements of theta has this matrix W 1 which belongs to r n cross n right? And now I want the derivative with respect to w. So, see what I am trying to do, this is scalar and we take the derivative of that with respect to a matrix. What is all that; the derivative with respect to.

Student: (Refer Time: 02:37).

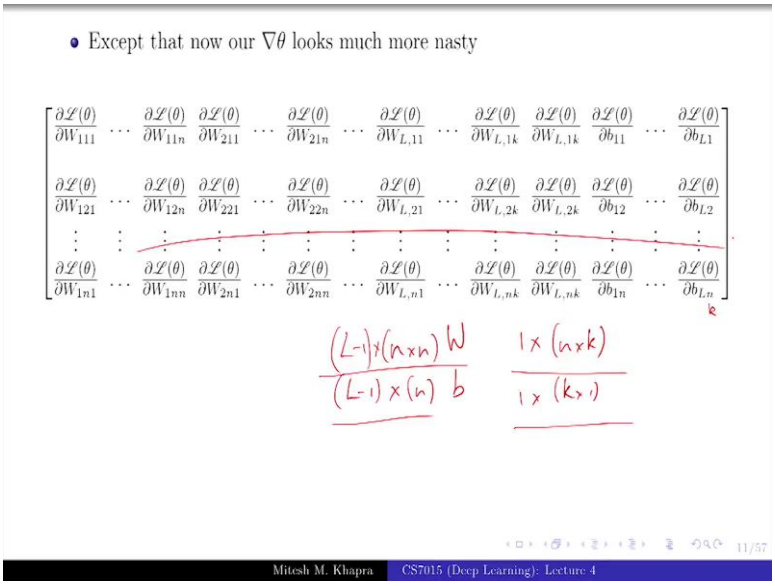
Every element of the matrix. ok

(Refer Slide Time: 02:41)



So, we can still use the same algorithm except that del this grad of hat of so now, I could just say that theta 2 hat I mean initialized all parameters and theta naught, right? Compute the gradient with respect to all of them and then do this update, right. I could just instead of putting them in matrices, I could just think of them as a large vector just had initially I had just had w comma b, now this vector is even more large. In fact, I will show you actually how it is.

(Refer Slide Time: 03:08)



So, this is the grad with respect to theta looks very nasty now. This is how nasty looks, right? So, you have this weight matrix W 1, you have the derivatives with respect to first element of W 1, all the way up to the last element last element; so with respect to all the n cross n elements of W 1. What is the next entry going to be W 211 to

Student: (Refer Time: 03:32).

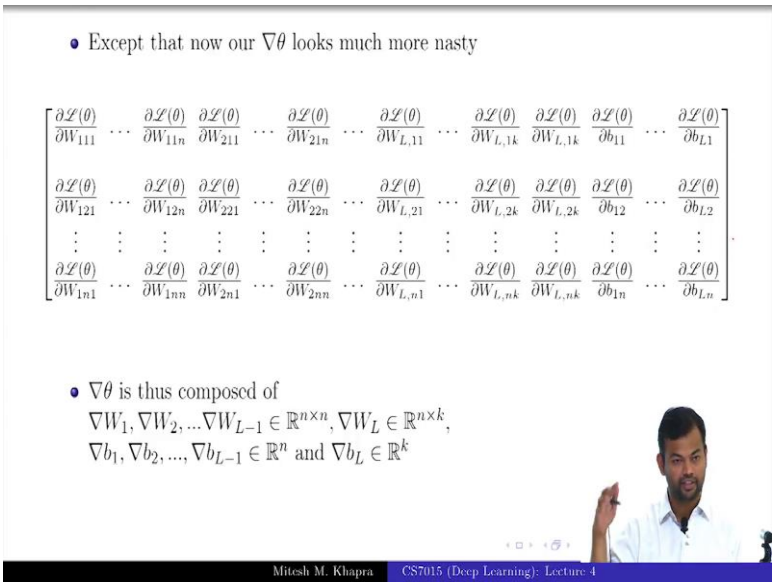
W2nn next after WL11 ok and then after this. Ok?

Student: (Refer Time: 03:41).

What is remaining biases? Right? So, you have b11 to b1n this slight error here, but intentionally this actual is k, because k is not equal to n, right the last layer has only k parameters, whereas so that it looks ok? Is this clear? So, is this are all the partial derivative that we need, right? You do not need to worry about taking a partial derivative with respect to our matrix. It just boils down to taking the partial derivative with respect to all elements of the matrix.

So, earlier you just had 2 parameters. Now you have these n cross n plus n cross n upto L right. So, L into n cross n plus L into n that many number of parameters is what you have. You get the calculation? Right or rather you have L minus 1 layers each of which has n cross n parameters, right and L minus 1 layers which also have the biases. So, these are the W's these are the b's, then the output layer one layer which has n cross k parameters and k cross one bias. So, these are all the number of parameters that you have. And this is exactly what this size of this matrix is, right it has all these parameters. And you need to compute the partial derivative with respect to each of these parameters.

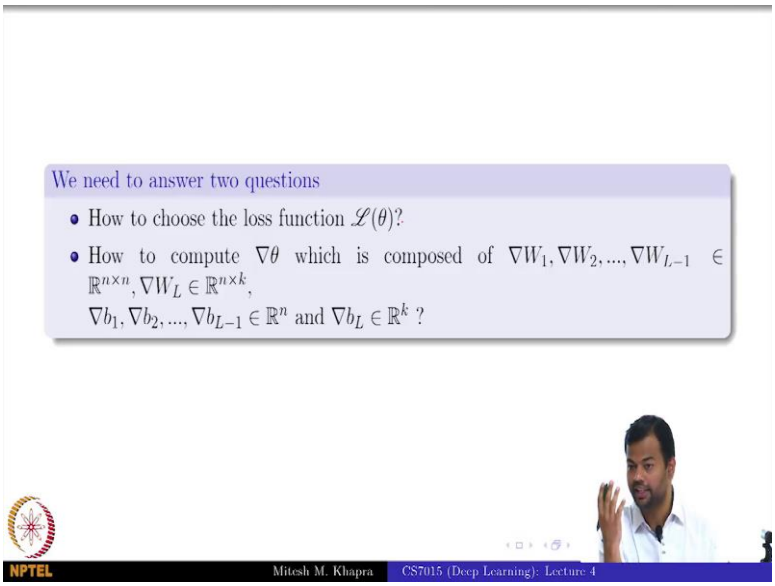
(Refer Slide Time: 04:59)



So, this is what grad theta is composed of, it is composed of the partial derivatives with respect to all the parameters of your network. ok So now, if someone gives you each of these quantities, same oracle give you each of these quantities, then can you apply gradient descent right? You can use the exactly the same algorithm that you are using earlier, just the sizes of earlier vectors changes.

How many of you are convinced that now you can use that gradient descent? There is not a trick question how many of you convinced, how many of you not convinced? Assuming that someone has given you these quantities, right? I know that it is hard to compute we will see how to compute that, but let us assume someone has given you this. Then you can use gradient descent that is what the case I made in the previous slide right that you could initialize with all the parameters compute the gradients with respect to all the parameters and just do this update, fine. So now, we need to answer 2 questions, first is this is the key question.

(Refer Slide Time: 05:55)



Because we are taking derivative of what, loss functions. So, we need to know what the loss functions that is the crucial question, right? And then we are taking derivatives with respect to all these elements. So, whatever I was told you that assume that oracle gives you, now you have to do the hard work and actually find it out right? So, if you can answer these two questions then we are done. We have an algorithm for learning the parameters of feed forward neural networks. We all agree that if you have these two elements then we have done.

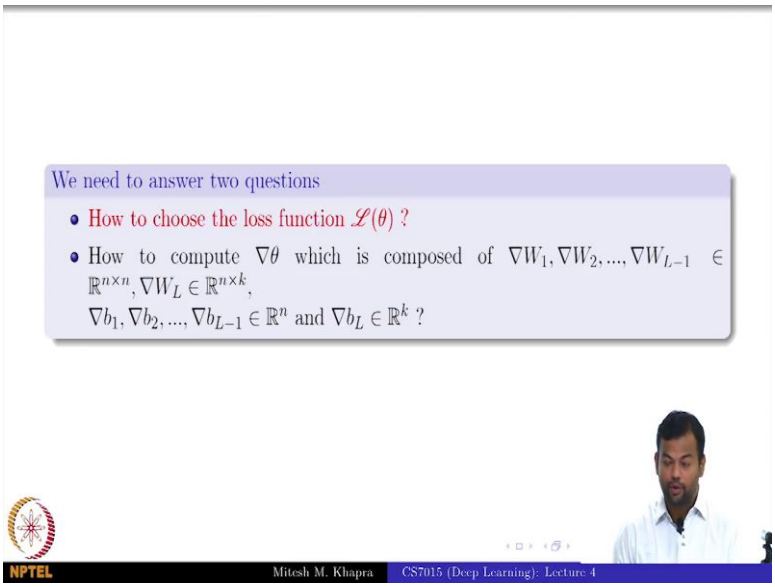
So, here I will end this module.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module - 4.3
Lecture - 04
Output Functions and Loss Functions

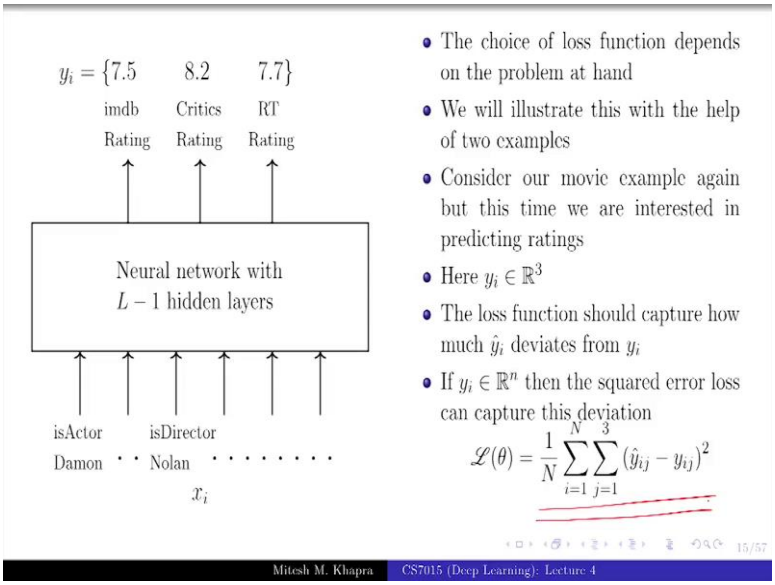
We go on to the next module where we will be talking about Output Functions and Loss Functions.

(Refer Slide Time: 00:19)



The question that we are going to focus on is how to choose the loss function, but I will show you that it is tightly coupled with the choice of the output function also. Remember that we had said that we have a special O function as the output function, I have not told you what that O is, and now that is what we are going to define.

(Refer Slide Time: 00:33)



Now, the choice will be loss function actually depends on the problem at hand. And that is exactly the question which had come up, right that in some cases it is to have sigmoid as the output function because your values are between 0 to 1, but whatever there are cases where your output is not between 0 to 1 right? So, it definitely depends on the choice of the on the problem that you are trying to solve. So, we will illustrate this with the help of 2 examples, and these 2 examples will cover a broad range of problems that you will encounter or if you are working in machine learning right?

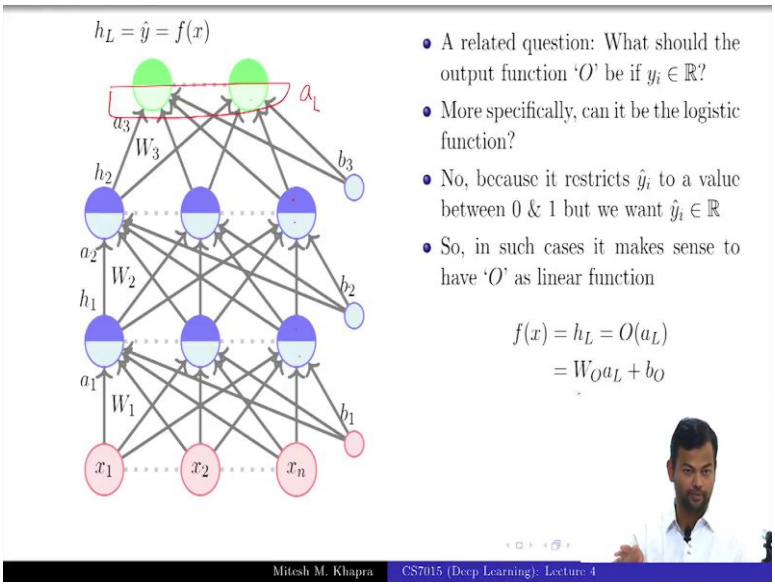
So, the first problem is again you are given the input as movie. You are using a neural network with L minus 1 hidden layers and an output layer y hat right? So, this is, sorry, this is a true one. So, you have an output layer and the output layer is going to predict the IMDB rating the critics rating and the rotten tomatoes rating.

Is that fine, ok? So, what kind of problem is this? People have done machine learning, this is a regression problem. And notice that the output values that you want to predict are not bounded it by 0 and 1, they are still bounded by 1 to 10, but in general you could imagine that there could be problems. So, there are no bounds at all right it could be a very large number, is that clear? Now here yi belongs to R 3.

So, remember in all these cases we were assuming that we just want to predict one value, but nothing stops you from predicting multiple values at the same time. So, your output is now 3 dimensional, you are taking an n dimensional input and trying to predict 3

values from it, ok fine the loss function should capture how much yi had deviates from yi. Ok so, this is a valid or maybe we corrected on this way, ok. So, this is the formula which was supposed to be in there right? So, you take, you have predicted 3 values, and you know the true 3 values, you just take the difference between these right? Is that clear? The first element of the predicted value, minus first value of the actual value and so on for all the 3 values that you want to predict.

(Refer Slide Time: 02:42)



Now, you have a loss function, but what should be the output function in this case? Can it be the logistic function? Yes, no, it will be bounded between 0 to 1, and you know that your output cannot be bounded between 0 to 1, ok. So, in such cases then what is a good output function to use? One option is to scale it. So, I will keep that aside, why do that? It is unnatural and you are actually clamping it and then trying to scale it, right? So, can you do something more natural in that, just use a sum which is linear function, right? So, what we could do is you could have O as a linear function.

So, what that means is, again remember that this is a of L, ok and I know all the computations that have happened so far; a linear transformation, non-linear, linear, non-linear and then again linear. So, I have computed a of L, from that I want to compute the final output, right? So, I could just have it as a linear function of the input which is a of L in this case.

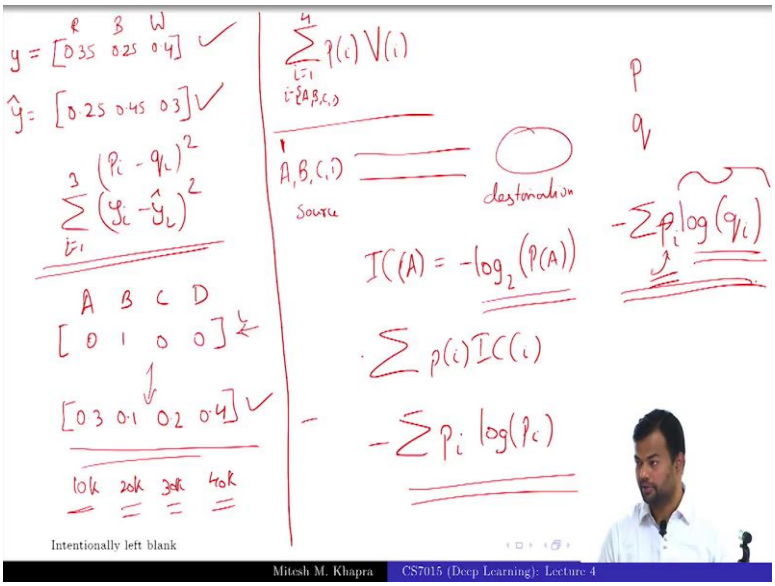
Does it make sense? How many of you feel it makes sense, ok why? Because now it is no longer bounded, right you could this linear transformation your weights could be adjusted in any way to get a value whatever you wanted, whether you wanted between 1 to 10 or 1 to 100 or 1 to 1000, these weights could be adjusted to do that right? So, at least you are not bounding it and it is free to learn, what is the range from the data it should be able to run, but how should you adjust these W's.

So, that you get the desired range, now tell me why would it not happen that you learn W's you start predicting values like 1000, 10000 and so on in this particular case where your input is bounded by 1 to 10 sorry; your output is bounded between 1 to 10. Why would it happen? I this is my argument and you prove me wrong right? I would say that if you have chosen a linear transformation which is not bounded, I then network could learn weights which start producing a rating of 10000, 20000 and so on because it is not bounded.

But you know that that is wrong because the ratings can only be between 1 to 10; so why would that not happen, because you are minimizing this loss function right? So, if you start predicting values like 10000, when your actual rating was 9 then you have a 10000 minus 90 whole squared loss, that is a very high loss. So, it will start moving you away from that configuration right? So, the training is always guided by the objective function. So, if your training happens well it will try to prevent this.

Now, suppose let us take a simple thing rate that; you are given a our same ball example for probability. So, you are given an urn which has balls of 3 colors; say black, white and yellow.

(Refer Slide Time: 05:38)



And you have to put the balls in that. So, you know that the true probability distribution is actually 0.35, 0.25 and 0.4, for red, black and white, ok. This is the true probability distribution, you have put say thousands of balls in urn. Now what you do is, you just allow me to peep into the urn or you allow me to take some samples from there. You tell me take these 100 samples, and you ask me, tell me what this probability is right? So, this is the true probability that you know is true right because you know it because you have estimated.

Now, you just give me a small sample from there and ask me to estimate it, and based on that I actually estimate this, ok. So, there was a true probably distribution and an estimated probably distribution. Now I want to find out how wrong I went, right afterwards you tell me the answer you tell me that this is what the true was and this is what you predicted.

Now, I want a way of computing how wrong I was, right? So, how do I do that? You already know this and these are 2 vectors, what can I do? You could just do the; this is valid anything wrong with this? In principle no, you could just treat these as any 2 vectors you have a true value you have a predicted value you just take the squared error difference between them right? But you know this is a probability distribution, right? You should be able to do something better than this, you know this is a special quantity this is not just any number that you are predicting, you are trying to predict a

distribution. So, you should be able to do something better than that right? So, that is what we want to see, how to do something better than this, that is what our quest is?

Now, again why we are at this, right, I also want to make a because this is something people do not immediately understand. So, I just want to make a case for something else. So, I will just do that ok. Now suppose there is this IPL, ok and there are 4 teams in the semifinal, let us call them A B C and D. Ok now, I was not in town after the semifinal. So, I just know the results up to semifinal, and then the finals also happen, and one of these teams wins, let us call it the B team, right the B team wins. Can you express this in terms of probability? Can you express this in terms of distribution? What do you mean my 0 and 1 B has won.

So, it is a certain event because it has 1 now. So, what is going to be the distribution? 0 1 0 0 right? So, this event happens with 100 percent probability ok. Now, the same case can you ok. So now, let us do the same thing that is as I said I was not in town, right and you asked me tell me which team would win that is; I know these 4 teams have qualified in the semifinals, and I know who the players are and so on.

And with my limited knowledge of cricket I will predict something right so, say I predict this. Ok Can you again tell me how wrong I was? You know what the true label is and you know what I predicted. You can tell me how wrong I was, ok. So, the case which I am trying to make is that even if the event is certain, you can still write it as a probability distribution where all the mass is allocated to the correct output. Can you relate this to a classification problem? When you see training data, you have already observed it, suppose there were 4 classes possible.

Apple, orange, mango and banana; if you have seen it is apple, and if you ask you what is the distribution, what will you tell me 0, 1, 0, 0; you will express it as this one hot vector; where all the probability mass is concentrated on the guy which is correct, right. So, even certain events which happen with certainty you can write them as a distribution rate, where all the masses are located on the true label. So, that is how all classification problems when you are dealing with multiple class classification problems it is often the case that you will write it as this.

That your true label is given to you in this format, there were 4 possible events, 4 possible classes or k cost possible classes, out of which only one is correct and then you

make a prediction, and you want to now find out how different was your prediction from the true label. You are trying to get the set of how this relates to a classification problem, and this is that is why this is of interest to us ok.

So, this so, we will see this soon. Now the next thing that we need is how many of you know what is entropy? Forget about cross just entropy, ok that is why I have left 2 slides intentionally blank ok. So, so now, let us see where I go with entropy, ok. How many of you know what is expectation? Please, fine. So, again the same thing now I knew that this was the distribution which I think I am into gambling, am not I am into gambling, and I try to bet on these teams.

And I bet some amount on each of these, can you tell me what is the expected? Reward that I will get. So, what am I saying wait. Suppose this is the case that if team A wins I get 10K rupees or my net profit is 10K rupees, if team B wins my net profit is 20K rupees and C and D so on, right. You get the setup for every even there is an associated value with it. This is the value of event A winning, B winning, C winning, D winning. So, the net profit in each of these case. So, what is my expected net profit? No, give me a formula sigma overall events right? How many events do I have here, four right. So, rather I should say i equal to ABCD, right probability of i multiplied by the value associated with that event. So, this is how you compute expectation, ok everyone gets this.

So now suppose say am doing this right? There are suppose 4 symbols. I do not know what I am teaching ok. So, and I am trying to communicate this from a source to a destination, ok. And now, suppose these are the 4 symbols that I give, and if these one of these symbols is say with probability 1, and if I transmit it, what is the information that this guy gets? So, this is assumed that a is that sun is going to rise today. If I tell you this when you are sleeping in the night, what will you tell me? So basically are not gaining any information? Well it is a certain event you know this is going to happen right?

Now, one of these events, suppose I am going to say that this there is going to be a cyclone tomorrow morning. What is the probability of a cyclone happening? In Chennai almost one, but still it is a very rare event. So, if I tell you something which is very rare that message has a very high information content right? So, if event which has a very high probability has a very low information content, and an event which has a very low

probability has a very high information content right? So, you can measure the information content of an event.

So, so the point is that what you can have is that the information content of an event, you can write it as, how many of you get this? how many of you have seen this before? All of you have seen this right. So, this is the value associated with an event, ok, now can you tell me what is the expected information content? For every event now I have given you the value associated with that even. So, what is the expected information content? Summation p of I into information content of I, and this like and this is of course, log right. So, it would be so, what is this called? This is called the entropy.

Now, what is cross entropy? How many distributions are you dealing with here? 1 which is the p distribution, which tells you how likely these messages were, and based on that you are trying to calculate the entropy of this situation right? So now, what is cross entropy? You have a true distribution say you have a predicted distribution. Ok this is what you predicted. So, that means, according to your predictions, the information content of every event is going to be log of qi, because that is what you predicted right? But what are the actual properties which with these which these events are going to occur pi's right? So, then the expectation has to be computed over pi's right?

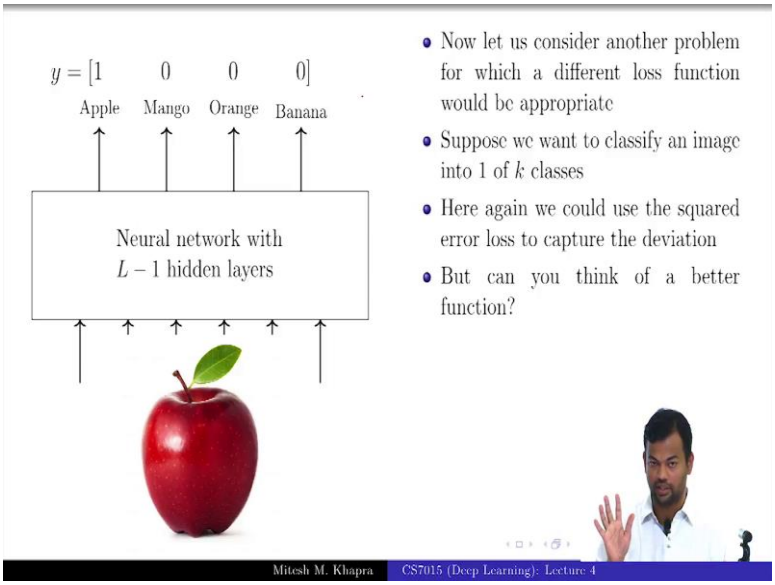
So, then what you will have is summation pi log qi. So, this is what you estimated the information content to be, but the actual events are going to happen with this probability, right? So, this is your value associated with the event, and this is the actual probability of the event right? So, this quantity is known as the cross entropy, is it clear? And this is a way of measuring when would this be in when would this be minimized? When both are same that means, if your prediction is very close to your true distribution this quantity will be low, minimized actually.

So, that is what we wanted actually, you wanted to predict some distributions in all of these cases, and you wanted a measure which tells you that this prediction was good, and what is the definition of good? It is as close to the correct value. So, cross entropy gives you a measure of telling how close a predicted distribution is to a true distribution.

So now instead of using the squared error which was actually pi minus qi right? So, pi was my true distribution, and qi was my predicted distribution I can use cross entropy

which is given by this model. And it does the same thing it gives me a principled way of measuring how close my predicted distribution is to my true distribution, do you get this?

(Refer Slide Time: 16:46)

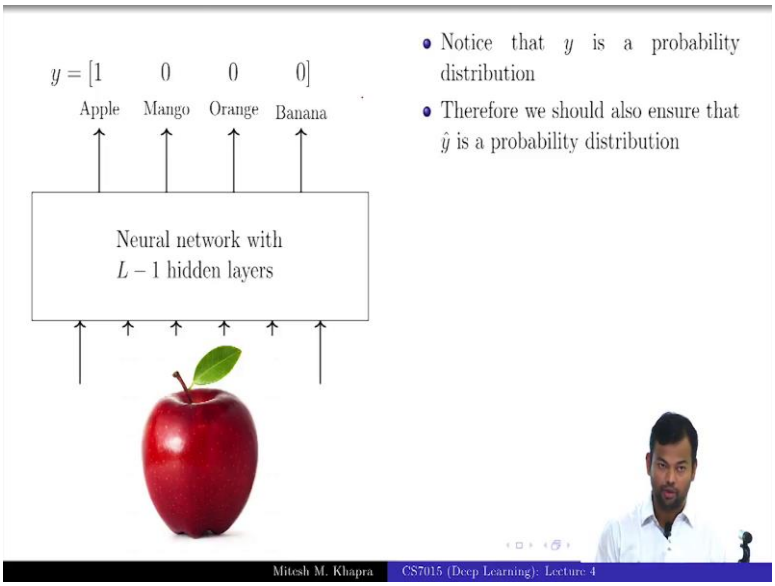


So now, so, this was for whatever we have done so far, right till this point this was for regression, right? Now I wanted to enter into classification, for which I have built this set up of how to take the difference between 2 distributions. So now, let us consider this problem where we have this situation and which is a classification situation, that you are given 4 possible classes out of which one is the correct class. And this is the true data given to you this is the true distribution, all the probability mass is focused on one of these classes.

Now, we want to given an image classify this into one of k classes. If you could again use a squared error loss, but since we are dealing with probability distributions here, we want to use something special. So, before we get to what the special is going to be, what do I first need to tell you? In the earlier case my output was not bounded was it also dependent, was there any condition on if the IMDB rating is something the critics rating should be something else or the rotten tomatoes rating should be something else, no.

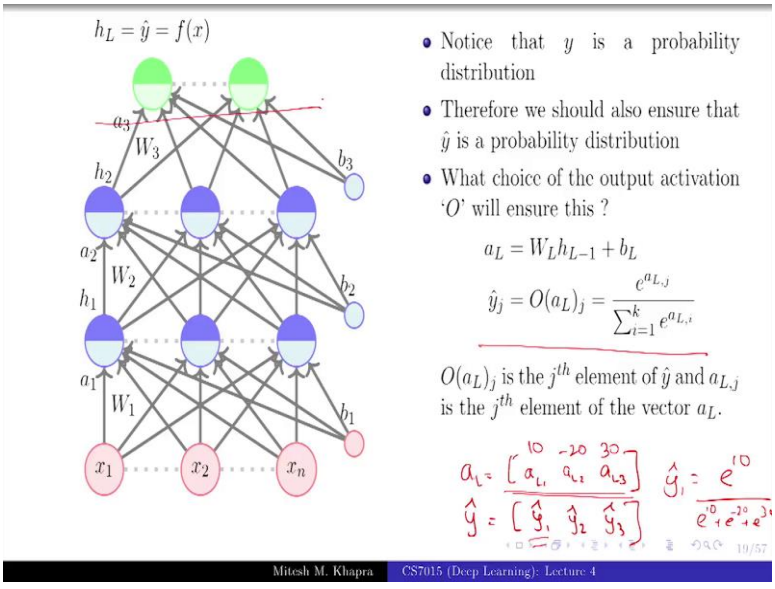
Now, in this case is there a tightly coupled behavior between the outputs, why? Because they should sum to one, we are trying to predict a probability distribution. So, the sum should one right. So, I need an output function which ensures this, you get this setup?

(Refer Slide Time: 18:08)



Now, we should ensure that y hat is also a probability distribution, whatever we are predicting is also a distribution. So now, can I use a sigmoid function, yes, it will give me values between 0 to 1 and probabilities are between 0 to 1. But the sum would not be y so, sigmoid is ruled out.

(Refer Slide Time: 18:32)



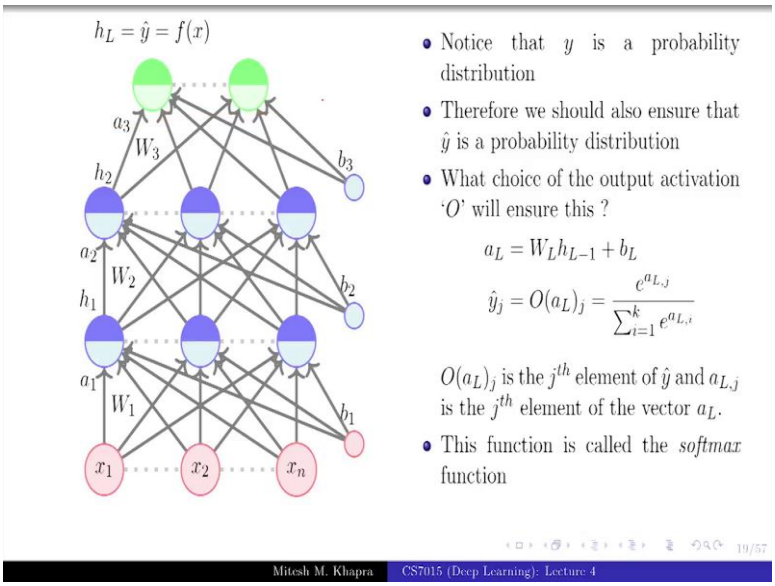
So, what we use is something known as the Softmax function. How many if you have seen this before? Please everyone raise your hands, otherwise you will get 0 on the assignment, fine. So, what does this? What does this function actually do let us look at

this function right? So, here you had a L which was say a L 1, a L 2, a L 3, right suppose we had 3 classes, ok. So, from here I actually want to go to hL or rather I going to want to go to y hat, right; which is going to consider y hat 1, y hat 2, y hat 3, right it is going to give me probability of each of the 3 classes.

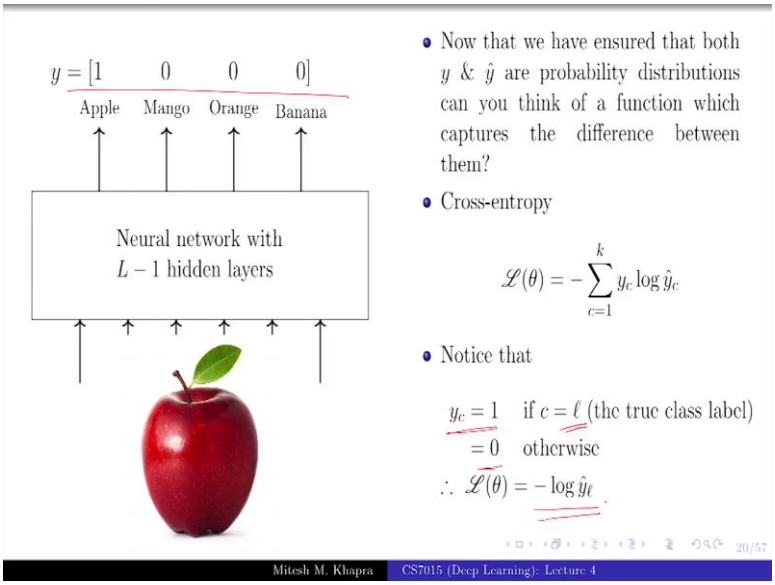
Let us assume there are only 3 classes right. So now, what this function does is, how is it going to predict y 1 hat, suppose these values were 10 minus 20 and 30. So, what is going to be y 1 hat is going to be e raised to 10 divided by e raised to 10 plus e raised to minus 20 plus e raised to 30. So now, you see how the output is comp computed from each of these values. Right So, why did we do this e raised to stuff why could not I have just taken 10 plus minus 20 plus 30 divided by the sum, because we have negative values.

So, once we take the exponent even the negative values become positive, right. So, that is why we need the Softmax function, I hope all of you wrote this in your assignment, they did ok. So, you get this we have a different output function now, and this output function does it make sense? It gives us a probability distribution now the summation would be 1, and each of these values would be; between 0 to 1, that is exactly what we wanted.

(Refer Slide Time: 20:24)



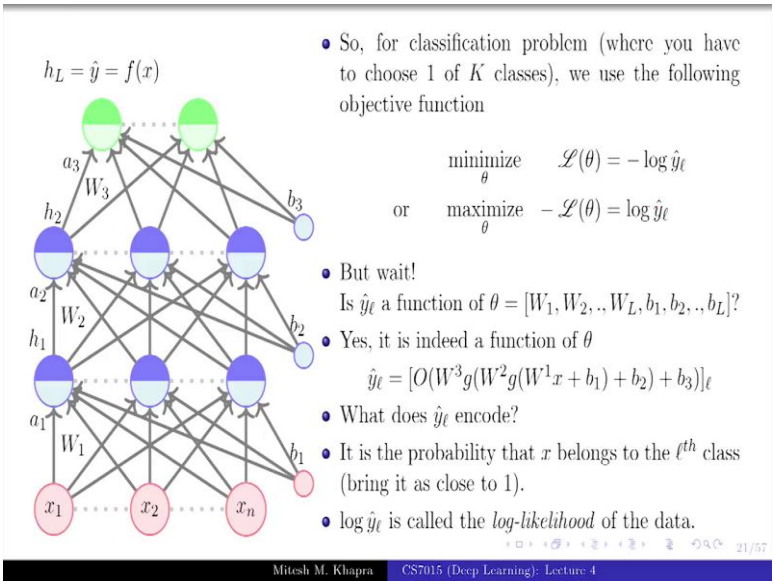
(Refer Slide Time: 20:25)



And now that we have ensured that y and y hat both our distributions what is the objective function that we are going to use? Cross entropy, how many of you convinced it is cross entropy? We have 2 distributions now, we saw that a principled way of computing the difference between 2 distributions is the cross entropy so, we will use the cross entropy.

Now can you tell me something about this sum, there is something special about this sum. What are these? 3 true values and these are the predicted values. What is so special about this sum? How many terms are there in this summation; k as many as the number of classes? In this case 4, how many of those terms will go to 0? All but one, right except for the correct class everything else will go to 0. So, this just boils down to the following loss function; that if L is the true class, right for that class yc is going to be one it is going to be 0 for everything else, that is exactly what this vector tells you only that term will remain. So, were actually trying to minimize this quantity.

(Refer Slide Time: 21:35)



Let us see so, for classification problems this is your objective function. You either minimize the negative log of y hat l or you can say you are maximizing this thing, ok. Now what is this quantity y hat l? No, it is a predicted probability of the correct event. Right So, this is a probably no wait this is an important question. So, you have y hat l here, and this is a function of I mean this optimization problem is with respect to theta. Is this a well formed objective function? Does y hat l actually depend on theta, yes, it does.

So, theta because why I tell is a function of all these things, everything here, and then a log on top of that, right. So, it is actually a function of all your parameters. So, this is a properly set objective function, we are trying to minimize or maximize with respect to theta, ok and you told me that y hat l is actually the probability of the predicted probability of the correct class ok. Hence, this quantity is also known as the ml class? pattern recognition class? log dash of the data.

Student: All (Refer Time: 22:53).

All good and fill in the blanks.

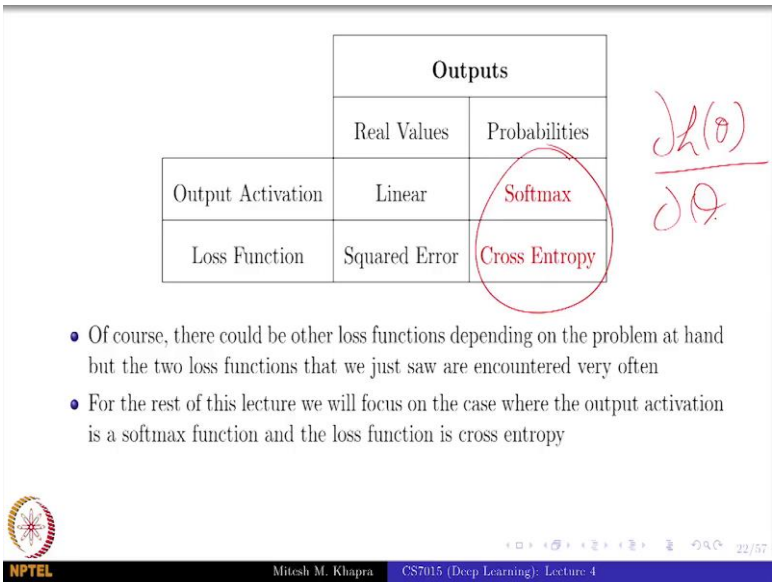
So, it is a priority of the x belonging to the l th class and then hence y hat l because it is the probability, it is the likelihood of it is called as the log likelihood of the data log likelihood.

So, what have we done so far? We started with a feed forward neural network, we defined the hidden layers and the input layers and the weights and the biases. We kept a provision for the output layer to be something special, right? Then we went to 2 classic problems, one is regression and the other is classification. In regression we wanted to predict values of all sorts of ranges.

So, we decided to use a linear layer there. So, that there is no bound on the values that you can predict, and your objective function should take care of where the bound lies. It should not allow values which are way off from the true values, right? And that is why we use the squared error function. There the other problem that we looked at was classification; where we saw that the label actually can be treated as a distribution, where all the mass is focused on the true label and 0 everywhere.

And our job is then again to predict our distribution. So, we are given the true distribution, and we predict another distribution. So, the output again we want something special in this case which is a distribution. So, to ensure that use a spatial function which is called the who said sigmoid softmax function, fine? And then we got a prediction which is a probability distribution, and then how did we find? What was the objective function what is the difference between the true and the predicted the cross entropy, right? So, we use cross entropy as the objective function, and then with some simplification we realize that it boils down to maximize the log of the probability of the true class, or other log of the predicted probability of the true class.

(Refer Slide Time: 00:53)



So now let us look at the summary. So, if your outputs are real values, what is your output activation going to be? Linear, what is the loss function going to be? Squared error if your output is a distribution what is the output function going to be? Softmax, what is this loss function? Squared error cross entropy. Right now this grid light actually takes care of a wide range of problems that you will see right think of any examples that have been giving you so far; movie prediction or sentiment prediction or image classification or anything, all of that you can fit into this frame of it.

And so, if you know these 2 loss functions how to deal with them, then you can deal with a large class of problems that you are going to deal. And for the rest of this lecture which will happen tomorrow we are going to focus on this. At this particular output function and this particular loss function, how do we compute? I have a loss function, what I am going to compute now the gradient with respect to all the parameters.

So, this is what we are going to focus on right? So, we have seen the loss function in detail, we have seen that the loss function is tightly coupled with the output function. Now we are all set, but given this loss function how do we start computing gradients of this loss function

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

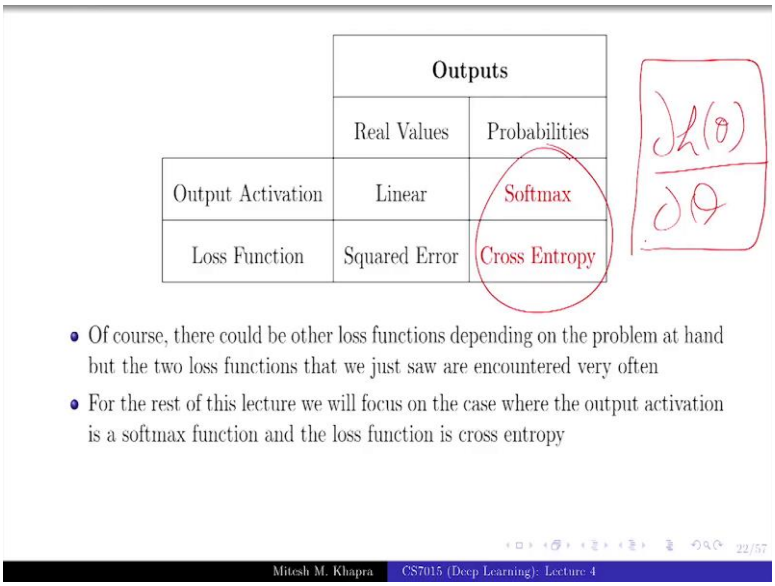
Module - 4.4
Lecture - 04
Backpropagation (Intuition)

This lecture is on Backpropagation and feed forward neural networks. So, we introduced a feed forward neural networks, we saw the input layer hidden layer and the output layers. And we saw that the output layer actually the output function depends on the task at hand. And we considered two main tasks, one was classification the other was a regression.

For regression it made sense to use a linear layer at the output. Because we did not want the outputs to be bounded, they could be any range. And for the classification problem we realized that we want a special kind of output, because we are looking for a probability distribution over the output. And for that we use the softmax function. And in both cases we used a different kind of a loss. For the regression problem the squared error loss made sense, because we predict some values and we want to see how far we are from those values.

But for the other case the classification we realize that it is a distribution, so maybe we could use something; which allows us to capture the difference between the true distribution and the predicted distribution.

(Refer Slide Time: 01:20)

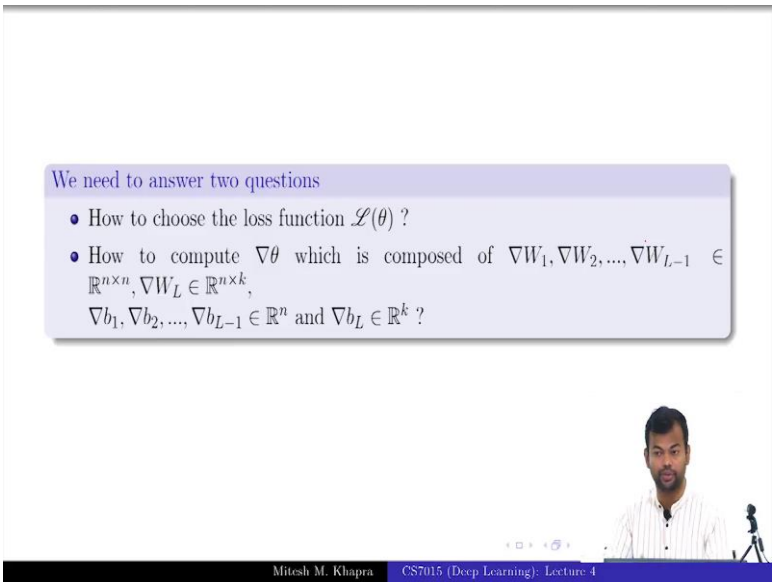


And therefore, we had this figure emerging which was depending on the output, whether it is real values or probabilities. You will have different types of output activation functions and different types of losses.

And of these combinations today we are going to focus on softmax and cross entropy, and our aim is to actually find these gradients. Remember there are many of those we have seen this large matrix which had many such partial derivatives, and we want to find that entire matrix. I hopefully do it in a way that it is not a repetitive we could compute a large number of partial derivatives at one go.

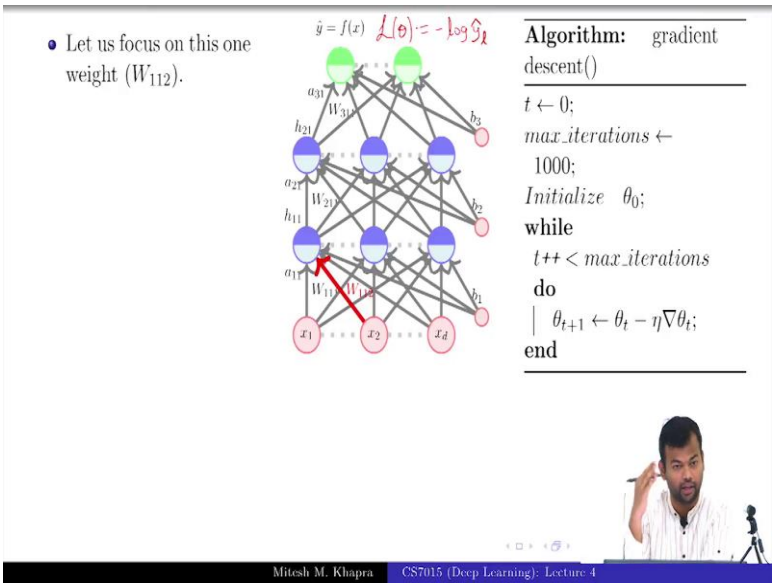
So, before we look at the mathematical details we just get an intuition for backpropagation.

(Refer Slide Time: 02:11)



And then we will get into the gory details of how to actually compute these gradients and partial derivatives. So, this is the portion that we are in we are intended to ask these two questions. And this is where we are.

(Refer Slide Time: 02:21)

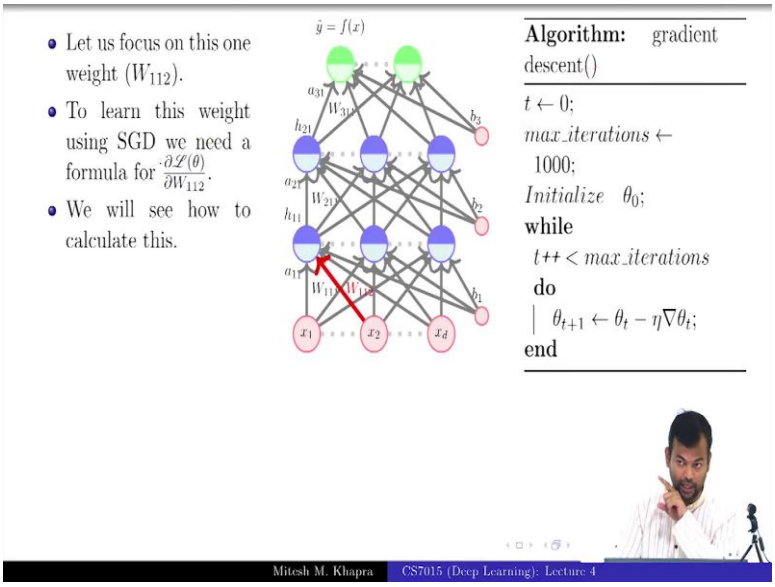


So now this is what our network looks like, this is clearly much more complex than that single neuron that we had, and which had only 2 weights W and b that was very easy to compute the gradients there. Now imagine that I want to compute the gradient of the loss

function. And let us assume it is a classification problem then what is the loss function minus log of y hat.

So, this is the loss function, and we want to compute the derivative of this with respect to one of these weights in the network. And am deliberately taking something which is much farther away from the loss, but why do you say why do I say it is much farther away, it is right at the input layer, right? And the loss is somewhere at the output layer. So, we want to compute this gradient.

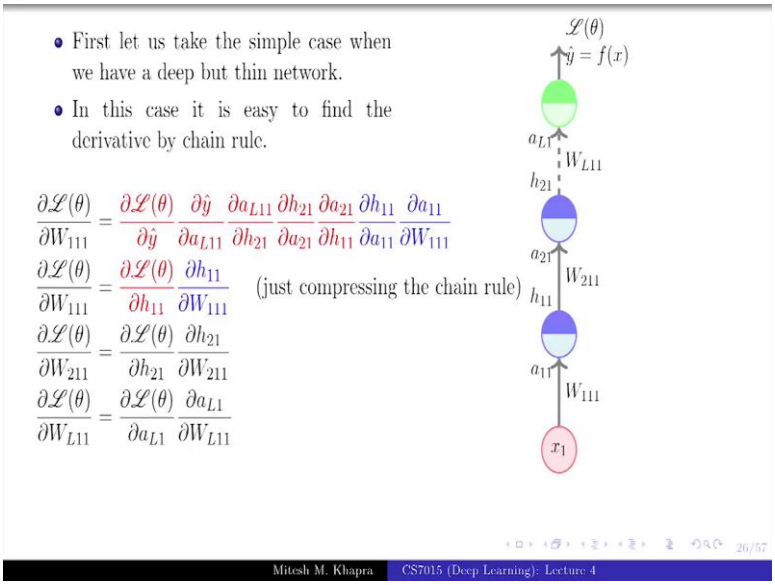
(Refer Slide Time: 03:10)



Now, to learn sorry you want to learn this way, to learn this weight we know that we can use gradient descent. We are all convinced that this gradient descent algorithm which I have shown here, as long as we put all these variables or all these parameters that we have into theta.

We can just run the gradient descent algorithm and compute, them the only thing that we will need is this partial derivative with respect to all the weights in the network. And in particular with respect to this weight that I am interested at.

(Refer Slide Time: 03:40)



Now so, we will now see how to calculate this, we will first this is only to get the intuition. So, we will first think of a very simple network, which is a very deep, but the thin network, it has many layers, but it is a very thin network, here you see what I mean by a thin network, ok. Now this is what I am interested in. Can you tell me how to compute this? This looks like a chain. So, it is justified the user chain rule of derivatives. So, what would the chain rule look like?

You want to compute the derivative of this with respect to this. And you have done this in high school, right? So, you have functions of the form of sine of cos of tan of e raised to sine of x, and this is exactly how this chain is right, you have some function of x followed by another function of x, another function of x function of x function of x and so on. You just keep making a composite function of the input. We actually wrote down that function if you remember. It was just one function applied after the other function, or a very composite function. So, you just need to apply the same idea here. So, we take we go step by step. So, I am almost accounting for every shade of color here.

So, dl theta by d y hat, then dy hat by d a L 11, there is only one neuron here, then this with respect to the sorry h 21, then h 21 with respect to a 21, a 21 with respect to h 11, h 11 with respect to a 11, and then a 11 with respect to W 11.

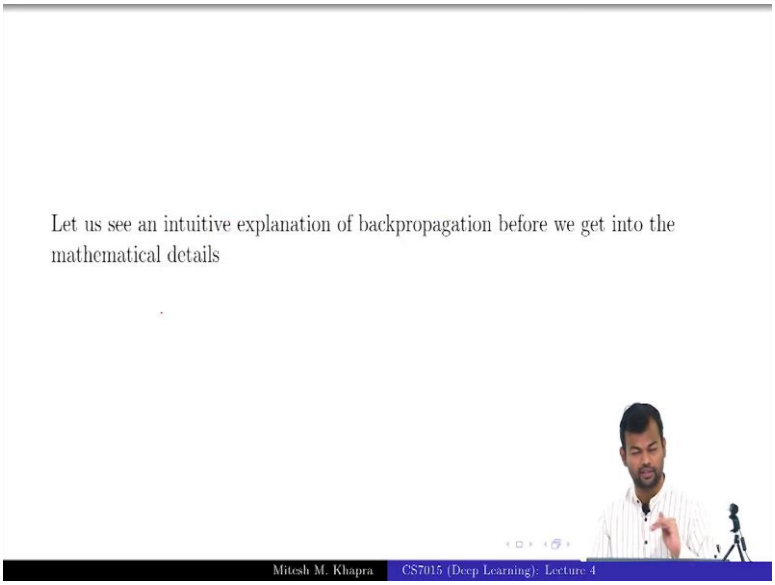
So, I just traversed down the chain in the reverse order, this is how the chain rule works, right? Anyone has a problem with this? It is straight forward? Right? And now what I

have done is for convenience I have just compressed the chain. You see the red part and the green part, I have just compressed this weight. So, that and this is again something that you have done in high school if you have this you could just write the chain as the first and the last it. So, this is what you can do, and I also compress this other chain, ok and am going to use these kinds of compressions later on.

So, what am trying to impress on you is that, if I want to go from here to here, right that is what my intention is. If somehow I have already travels from here to here, then I can just reuse that computation. That is the idea which I am trying to impress on it. I do not need to follow the entire chain every time, I can do these partial computations up to a point, have you seen this something similar idea somewhere else? Dynamic program is something like that. So, you have just computed up to a certain point, and then it is reuse the value for further down the chain.

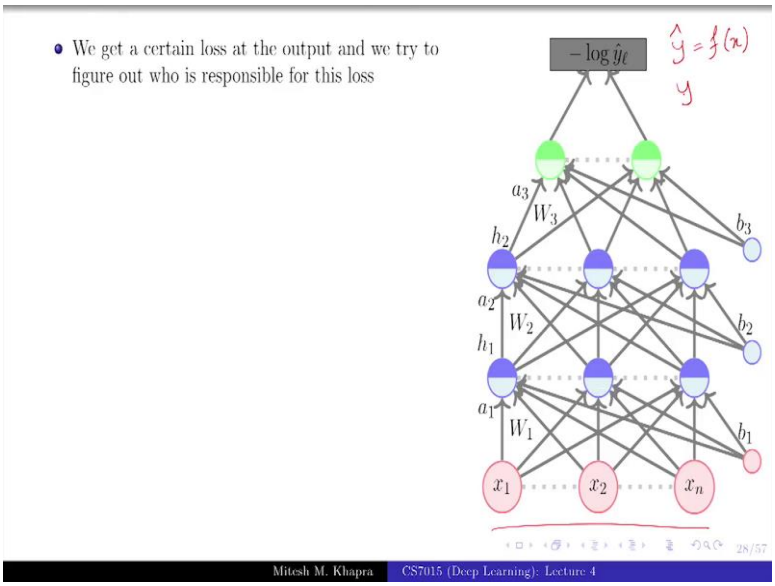
So, that is what we are going to do. And same for all the weights, right. For each weight the chain size would be different depending on where it lies in the network, right for the weights which are very close to the output layer the chain would be very small, makes sense ok? So, this is the intuition and we will see the intuition a bit more.

(Refer Slide Time: 06:40)



So, let us now understand this in the terms of the wide complex network that we are using.

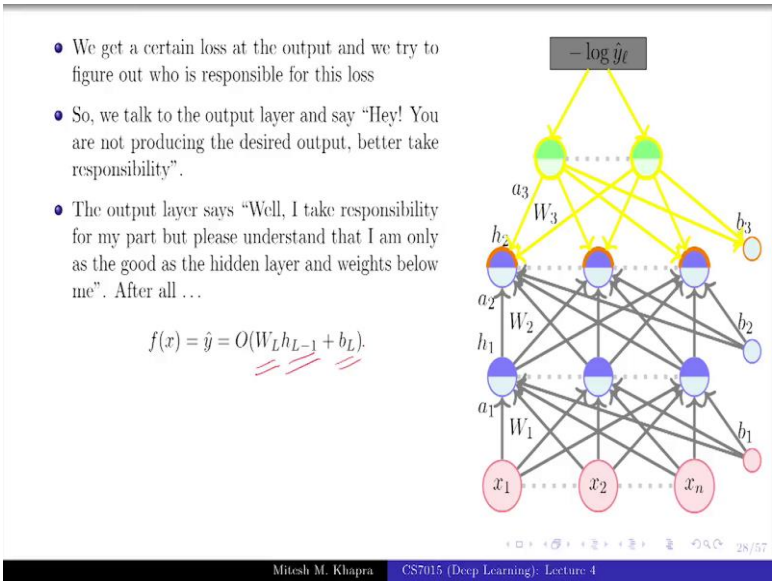
(Refer Slide Time: 06:47)



So, what actually is happening is that, we are at a certain stage; that means, we have some values of W's and b's ok, at the initial stage we just have these W knots and b knots. But let us assume that we have done some training, and we are at a certain level we are at Wt at time step t and bt at time step t, right for all the weights inverse. Now we feed it a training example, we do this entire compute computation, what do we get at the end? We get y hat which is a function of this x that we have fed it. But we also know this true y. We know the true value we know y hat.

So, we can compute the loss function. So, we compute the loss and to our surprise we see that the loss is not 0, we are getting a non 0 loss; that means, the network has not yet learnt properly, right the weights and biases are still not in the right configuration that we want them to be in right. So now, what do we do? We go on this path of investigation, we want to find at who in this network is messing up things? There is someone who is causing this problem, because of which I am not getting the desired output. And we are on our quest is to now find out who this guy is who is responsible for this. So, what would you do? Where would you start? The output layer.

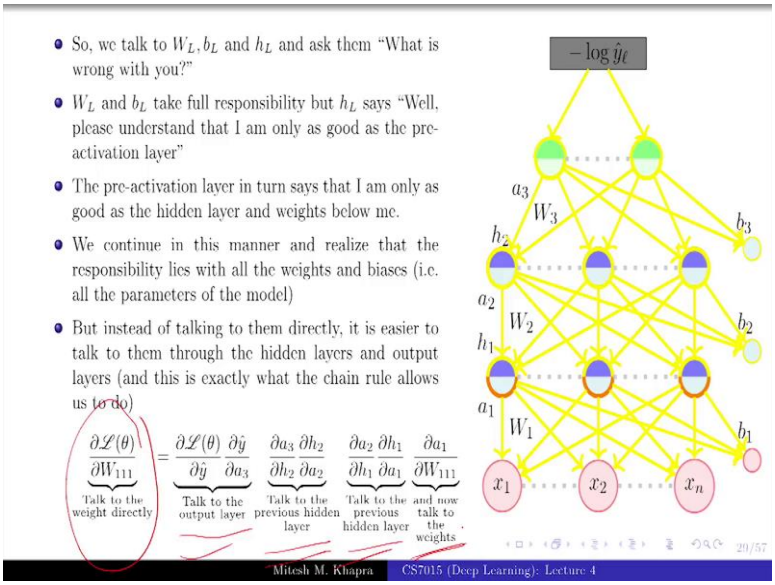
(Refer Slide Time: 08:11)



Because the output layer is the guy who give you the output, right. So, go and talk to him, and we say that hey what is wrong with you why are you not producing the desired output, right? Now what is the output layer going to tell you? In very civil language, I will say I cannot do anything boss, I mean, I was just given some weights and inputs from the previous layer, and those weights and inputs were messed up.

So, there is nothing which I can do go and talk to them. So, who will it directors do? It will say that I am just as good as WL hL minus 1, and bl because these are the guys that I completely depend on. If these guys were ok, then I would have been fine. So, we then go and talk to these guys, that what is wrong with you.

(Refer Slide Time: 08:58)



So now they say ok fine Wn and bL take the responsibility, they are the nice guys, they say we are the weights, we are supposed to make a, we are the ones who are responsible for the adjustments in the network. So, we have failed to do our job properly, and I think we should get adjusted right. But then hL will resist, it will say it is not my fault, why will it resist? Because it against again depends on the previous activation layer.

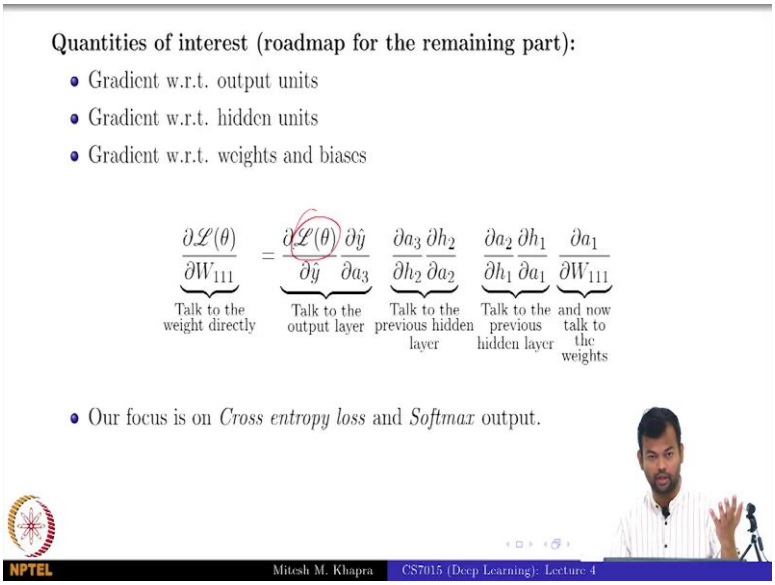
So, till then point as to what? The W's and b's in the previous layer, right. And you see how the investigation is now proceeding where will we reach? Well keep going down the network, we are talking to everyone in the network, we are talking to every dark green guy every light green guy, every dark blue guy every light blue guy, we are also talking to all these weights and biases. And in the end what do we figure out? The responsibility lies with all the weights and all the biases. They are the ones who are responsible for this.

Now, but now we find out that this is also one of those weights which is responsible, and this is also one of these weights that is responsible. But it was have been very difficult for us to talk to them directly. So, then what are we going to do? Instead of talking to them directly which is this, we will talk to them through the chain rule. So, we will talk to the output layer that is exactly how what we did maybe went to the first guy that we knew, that guy pointed out to the previous hidden layer, that guy pointed us to the previous hidden layer. And then finally, we get to the weights right?

So, this talking to is fine, but where do derivatives figure out in this, why are why is the language derivatives, why are we not talking in English or Hindi or something else? What does the derivative tell us? So, talking about gradient descent like what we saw in gradient descent, but in general what does the derivative tell us? If I change this a bit, how much does my loss change right?

So, that is how much this guy is responsible for the loss, because if this is very sensitive, even adjusting a bit of this I could drastically reduce the loss right? So, that is what the derivative tells us, that tells us how sensitive is the loss function to the weight or any quantity with this with respect to which am taking the derivative right? That is why the language is of derivatives right? Is that clear? Is the intuition fine to everyone?

(Refer Slide Time: 11:29)



So now will convert this intuition into actual math and try to figure out. How to compute every guy along the way, right? And we will use this idea that we have made some partial computations and then well, use it for the rest of the chain. So, we have made this much, at some point, we will reach where we have made this much, and then you could use it for the rest of the chain. In fact, we will start right from here well start with this guy and then keep expanding the chain.

So, the rest of the story is going to be about computing 3 quantities. Can you tell me which are these 3 quantities? Gradients with respect to the output units, gradients with respect to the hidden units, and then gradients with respect to the parameters. So, these

are the 3 things that we need to do, if we do this we have everything in the chain, and we are done. And the other thing that we need to do is, we cannot sit down and compute this for every element, right, we want to have it in a generic fashion, where instead of talking about W 1 1 1, W 1 1 2 and so on.

We should at least be able to talk about W 1, W 2 and so on. So, that means, we have only 3 matrices and 3 biases, right at least at that level. So, we have to do a collective computation instead of just computing for every guy. So, instead of looking at scalars, which is what we are doing when we are doing gradient descent for W naught and v naught? We were just computing the update rule for W and b, we want to now do it for vectors and matrices.

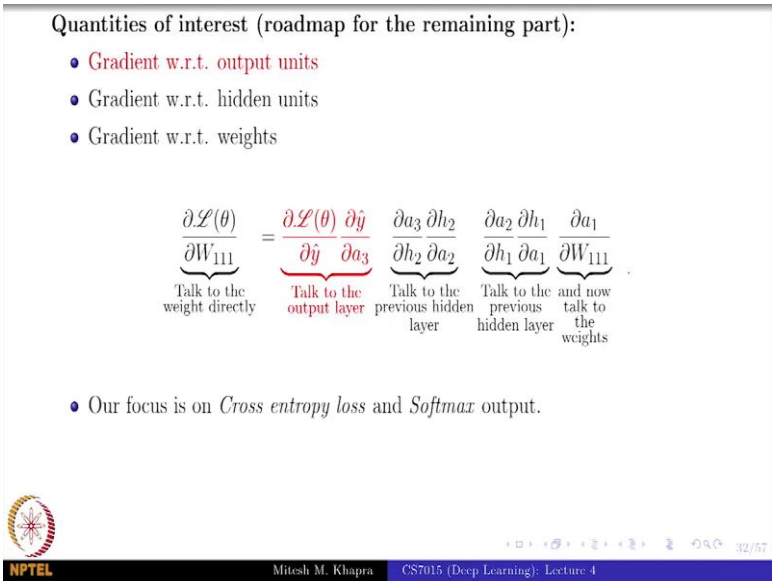
So, that is that is the transition that is going to happen. And our focus is going to be on, what cross entropy and softmax, why is that important, because that is the loss function. So, that is the quantity that am going to take the derivative, if I change the loss function all the gradients are going to change. Are all the gradients going to change? Only the first guy will change in the change, all this should remains still same, right? Modulus some conditions, but largely it should remain the same right, unless you change something in between.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 4.5
Lecture - 04
Backpropagation: Computing Gradients w. r. t. the Output Units

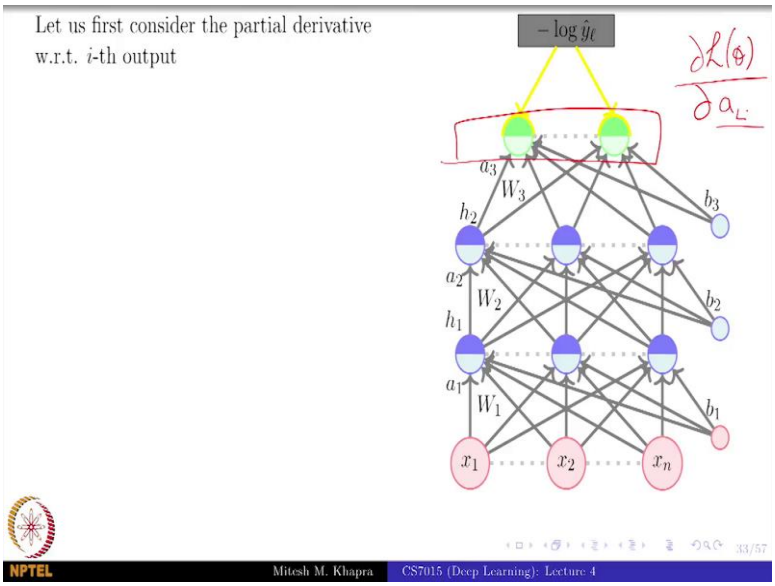
Now, we go to the next module where we will first see how to compute the Gradient with respect to the output units, well that was the first guy in our chain right that is the first person that we need to talk to.

(Refer Slide Time: 00:23)



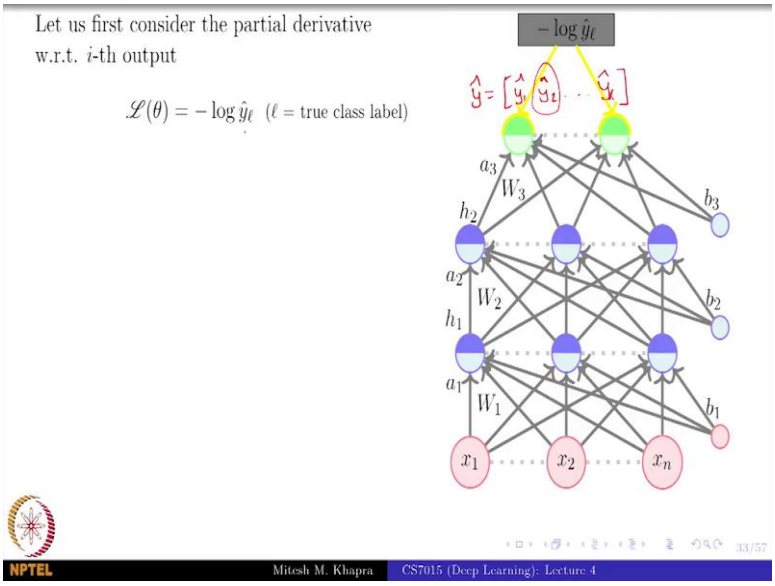
So, that is the part that we are going to focus on.

(Refer Slide Time: 00:26)



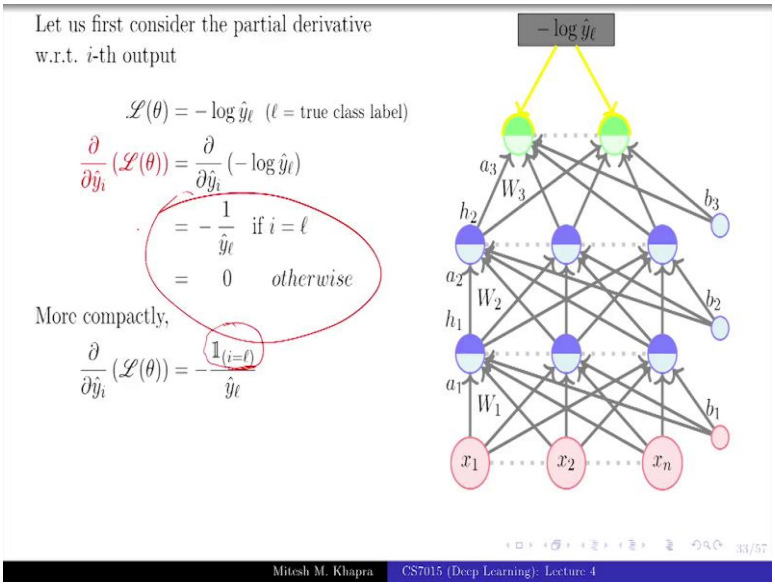
So, this is the output and when I say I want to compute the gradient with respect to output unit, what do you actually mean; what is the quantity that I am looking for? I will help you out, actually what I meant by output unit is this entire thing right. So, I actually meant aL's ok, but it is it is a fair answer and even y hat is a fair answer ok. In fact, am going to start with y hat and then go to aL. So, I will have to start with this guy and then come to this guy.

(Refer Slide Time: 00:54)



So, this is the loss, this is y hat which is equal to y 1 hat, y 2 hat up to yk hat. So, these are the k values that we have here and we are looking at cross entropy; that means, we are looking at the classification problem, right? So, we have got a distribution over the k classes; that is what y hat looks like. And we know that one of these guys is the right class maybe say y 2. So, the loss function is minus log of y hat 2, because 2 is the correct class in this toy example that I am considering ok. So, the loss function I am just repeating the definition right that is how the loss function is.

(Refer Slide Time: 01:30)



Now, oh god so again this is what our y hat looks like ok. Now I want to compute the gradient with respect to any of the output units right? So, it could be y 1, y 2, y 3, y 4 up to yk right? So, this i actually can take values from 1 to k, in this case 1 to 2 right ok? Now can you tell me what is this loss? Ok this much is fine; can you tell me what is this derivative? Minus 1 by minus 1 by y hat L if y is equal to L.

Student: (Refer Time: 02:08).

And 0 otherwise, how many of you get that? Cool ok, so it is a very simple thing that you can think of this as z and this is y, only if z is equal to y then the derivative would exist otherwise it is going to be 0 right ok? So, how do I write this fe part using?

Student: (Refer Time: 02:27).

How many of you have seen indicator variables before good? So, this is what you are telling me right? it is going to be minus 1 by y hat l, if i is equal to l ok. And if i is not equal to l, then these 2 things are not related, it this is a function of something else and you are taking a derivative with respect to a different quantity.

So, it is a constant with respect to that quantity and the answer would be 0 ok. Now I am going to write this as this right? So, this is the same as saying so this variable actually this is known as the indicator variable, it takes on the value 1. If the condition in the bracket holds, otherwise it takes on the value 0. So, this is exactly I am writing exactly this, but in a more compact manner ok, is that clear to everyone?

(Refer Slide Time: 03:15)

So, this is what the quantity this is the quantity, that we have computed with respect to one of the output units ok. So, this is what; derivative, partial derivative gradient, how many of you say derivative? No one likes derivative; partial derivative? That is always the safest choice partially [FL] right and gradient oh; there is one brave soul who say is gradient do not worry well fix that ok.

So, this is the partial derivative y because my y hat is actually a vector, and I am taking the derivative with respect to one of those guys ok. Now if I want the gradient with respect to y hat, what would that look like? A vector which is a collection of?

Student: (Refer Time: 04:01).

Partial derivatives. So, let us see this is the quantity that I am interested in am interested in the gradient of the loss function with respect to the vector y hat. So, remember the vector y hat is y 1 hat y 2 hat up to yk hat, right? So, this gradient is going to be a collection of the partial derivatives with respect to y 1 hat y 2 hat and so on.

Now, what is each of these quantities. So, it is simple right, so this quantity the derivative is either going to be 0 or is it going to it is going to be 1 by y 1 hat, right? If l is equal to 1 right? And that is exactly what I have done. So now, how many elements here are actually going to be nonzero? At a time how many of these going to be nonzero? 1, which one?

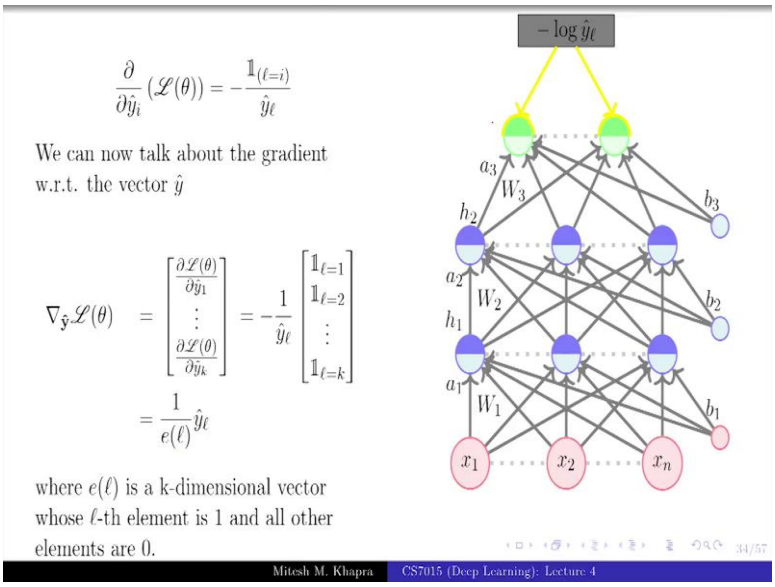
Student: (Refer Time: 05:04).

The 1 corresponding to l, right? Everything else is going to be 0. So, this is a dash vector y not vector ok. So now, am going to write 1 hot vector like this, what have we done? Ok where el is what 1 hot vector, such that; it is l th entry is 1 ok, that is what am going that is how am going to define e l, is that fine with everyone ok?

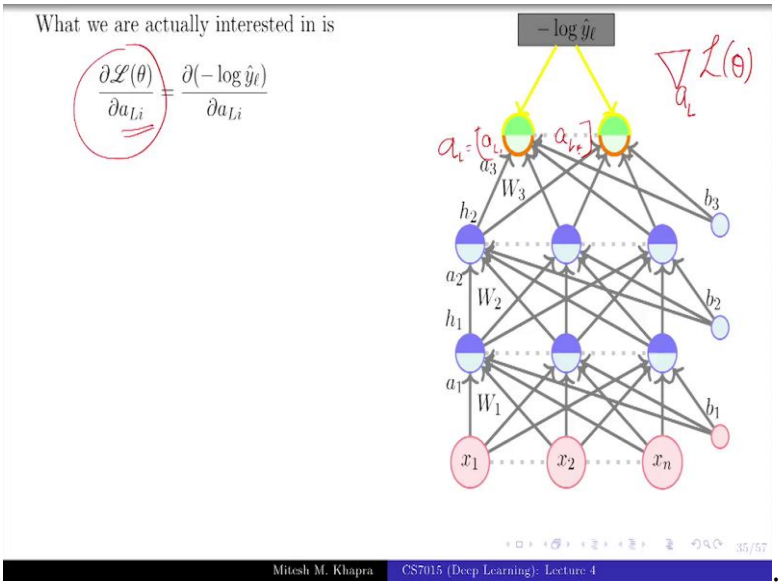
And so, you see the story how did how we went about computing this. We started with a partial derivative with respect to 1 of t guys right we found a formula for y i; we saw that this formula is generic enough. And So now, we can compute the gradient which is a collective of all these yis where i ranges from 1 to k, right? And then we just put that in a gradient vector.

So, this story is going to repeat throughout the lecture where we try to compute the gradient with respect to 1 guy and then generalize oh sorry, we compute the partial derivative with respect to 1 guy and then generalize and try to find the gradient fine ok.

(Refer Slide Time: 06:18)

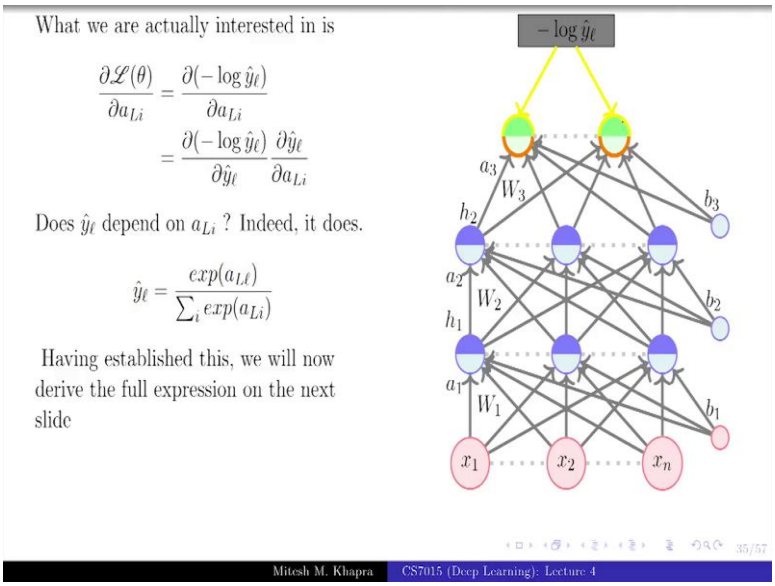


(Refer Slide Time: 06:19)



So, what if I what do I have so far? I have this quantity. What does till which part of the diagram am I currently? The dash green part dark green part I am till, here I need to go till the light green party that is collectively the output unit ok. Although I have divided into 2 halves, but when I say output unit I mean that output neuron right complete neuron. So, what I am actually interested in is these quantities or more specifically ok. This is what I am interested in, what is this? One of those guys right this aL is actually aL1 up to aLk right? So, this is one of those guys; so, this is going to be the gradient or this is going to be the derivative, a partial derivative sorry ok, now what do how do we proceed from here.

(Refer Slide Time: 07:26)



Now, I will again have to compute this, you already know that good, but before that I want you to answer one question right. So, y hat l, what is y hat l? It is the output corresponding to the correct class, does it depend on an arbitrary aL i? So, in the previous thing we saw that only when i is equal to l there is a connection. In this case is there a connection always or only when i is equal to l.

Student: (Refer Time: 08:02).

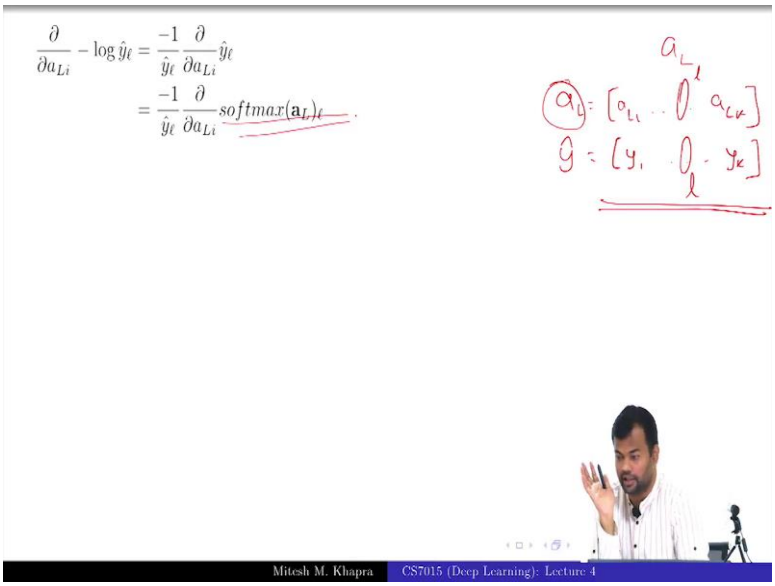
Always why? Softmax so?

Student: (Refer Time: 08:04).

Denominator has all the aLi's right. So, this is there it is y hat l in the numerator of course, it only has this unit which corresponds to the l th probably did not choose my variables very well. So, l th component of a capital L right? And but in the denominator you have the entire sum which means; that every output guy here; each of these dark green guys depends on each of the dash green guys light green guys good.

So, that is at least settled that we always the we can always compute this partial derivative, we do not need an if else here there is nothing like l is equal to i then what will happen it will always have this partial derivative.

(Refer Slide Time: 08:53)

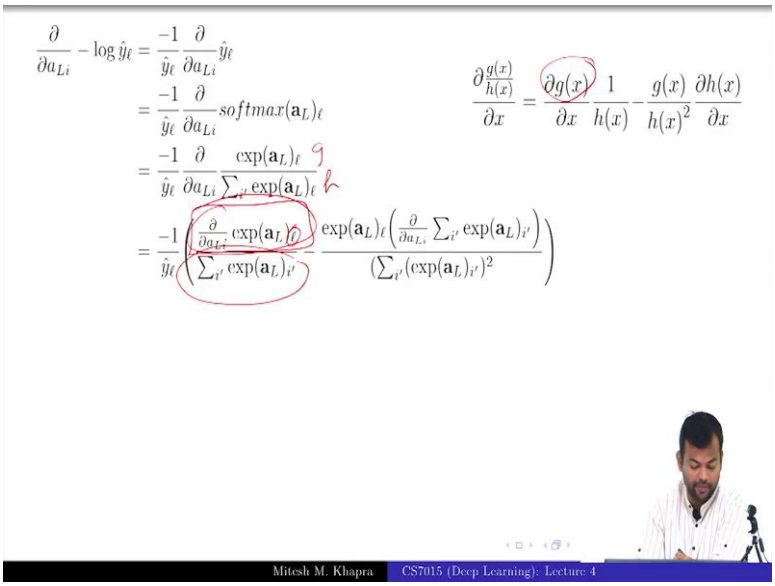


So, we will now derive the full expression for this. So, this is what we are interested in is this fine; so this is a function of the form; so you are taking how do I say this. So, this is log of a function. So, first you will take the derivative with respect to log and then push the partial derivative inside right. So, that would be minus 1 by y hat l and then the derivative with respect to y hat l. Now what is y hat l? The softmax function right.

So, it is the l'th entry of the softmax function applied to that output vector what is the output vector? aL right. So, it is the l'th entry of the softmax or l'th entry of the function applied to the output vector

So, this was our aL what is our output; right? So now, one of these guys here is the l th guy and one of these guys here is the l'th guy right. So, what you do is; you take this you apply the softmax function to it which again gives you a vector and now you are interested in the l'th component of that vector that is what this quantity means, it should be clear now.

(Refer Slide Time: 10:25)



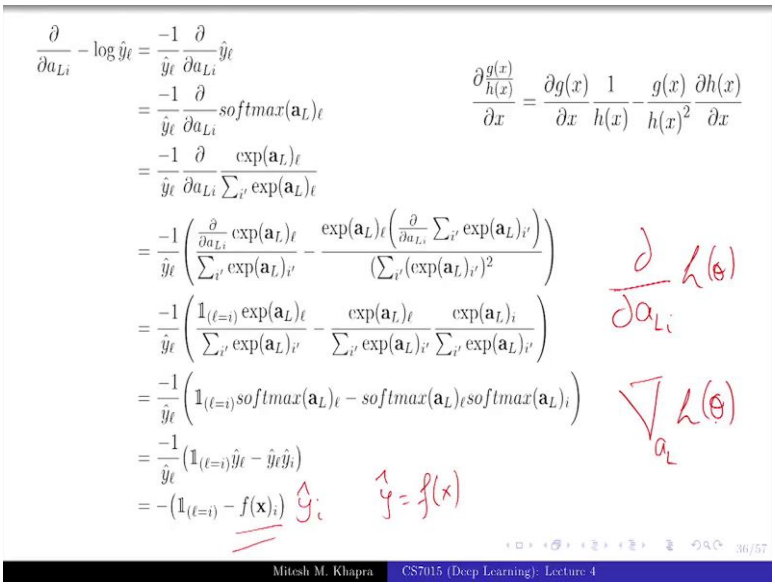
Now I will just do some simple math stuff here and we should be able to derive this, is it fine am just replaced by the actual softmax formula? This is a derivative of the form u by v right, what is the formula for that? Yeah, it perfectly right yeah. So, this is what it would be right, I mean it is you all know this I am not going to spend time on this.

So now am just going to substitute the values here, yeah it is getting a bit nasty, but it is not very difficult right? So, so this so this is our g of x so, am taking the derivative of that then this is this 1 over h of x, you can just figure it out right anyway it everyone just read this for a few seconds and let me know if this is not clear. This is g this is h in this formula right have just substituted the gs and hs in this. Now what is this quantity going to be? It is derivative of the form e raise to x right so it is e raise to x always.

Student: (Refer Time: 11:33).

If i is equal to l right; so now we have this dependence because we are looking at a numerator, but the numerator only depends on the l th entry right. So now, you are trying to take the derivative of the l th entry with respect to some arbitrary i th entry. So, only if l is equal to i you will get the derivative right?

(Refer Slide Time: 11:52)



Now, what about this; how many terms in the summation would remain?

Student: (Refer Time: 12:02).

1, which one?

Student: (Refer Time: 12:04).

Where i dash is equal to i right. So, the i'th guy would remain the rest of it is straightforward right. This square I have just divided into 2 parts ok ah, now let us see; can you simplify this? Because I cannot ok, can you simplify this? What is this?

Student: Softmax.

Softmax and which entry of the softmax?

Student: (Refer time: 12:36).

L'th entry, i'th entry, l'th entry with the saw with the indicator variable, but what is this? This is our input hidden layer output. So, ok now let us see, what is the next step? This is should have been y hat i, but y hat is equal to f of x right. So, we can fix this unit. So, ok fine so we have actually what do we have now? We have the derivative of the loss function with respect to the i'th unit of the output layer, right? And which part of the output layer? The pre activation pattern ok, now what am I going to do? I have a formula

which tells me how to compute this, what was I actually interested in? So now, how am I going to go from here to there? I just put all the partial derivatives into a.

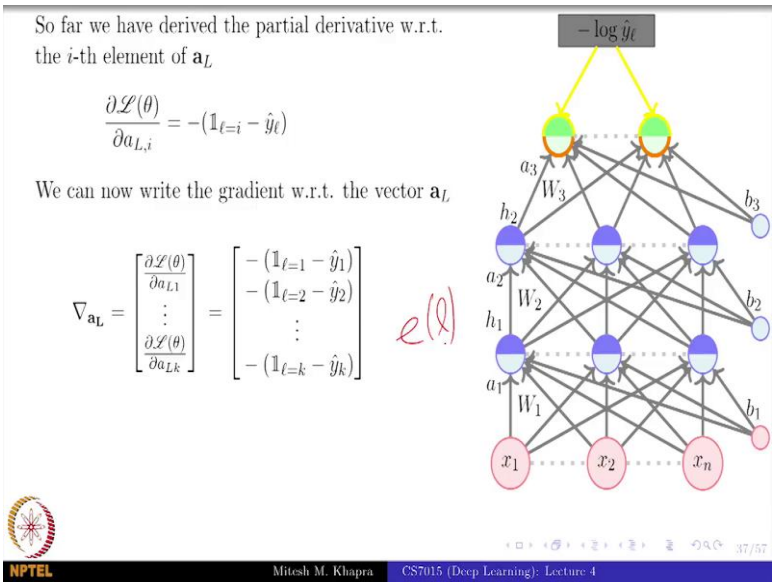
Student: Vector.

Vector, and that vector is the?

Student: (Refer Time: 13:58).

Gradient good.

(Refer Slide Time: 14:00)



So, we have this one formula it is ok, if some of you did not get this derivation right, it is very, very straightforward. If you go back and look at it I am pretty sure you will get it is nothing in this is, very simple elementary stuff right, except for some degree here and there ok. So, now what would this look like?

We should add actually l theta here, this would look like a collection of all the partial derivatives. We have a generic formula, what will we do now? What is the first entry? Minus in indicator l equal to 1 minus y hat 1. Which is the variable that we are indexing over i right not l, oh god oh we are indexing or ok, have I goofed up oh that is wrong, is it oh yeah that is wrong fine. Then this is fine we are indexing over I and then we can do this.

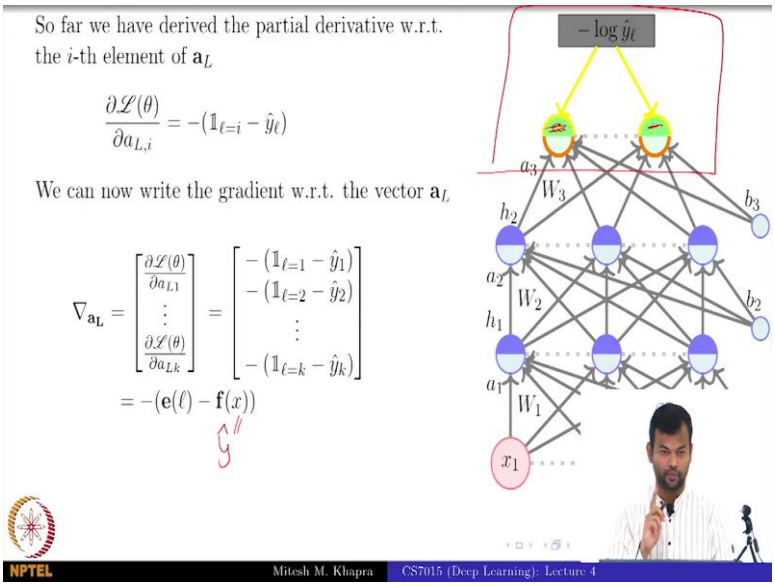
Now, can you simplify this? I am looking for ok this is the element wise difference of 2.

Student: (Refer Time: 15:38).

Of the indicator vector and.

Student: y hat.

(Refer Slide Time: 15:44)



Y hat, oh hey we should change all this, y hat is equal to f of x right, but I just want it to be consistent as y hat. So, is this fine this is a simplification fine, right? So, we have come a long way right you have finish this part ok, we have got the gradients with respect to the output units ok. This much part is a clear to everyone moduler bit of the math which you can go back and look at it this entire derivation is fine, but you get the concept right that we start with one unit from there grow the gradient then keep going applying the chain rule.

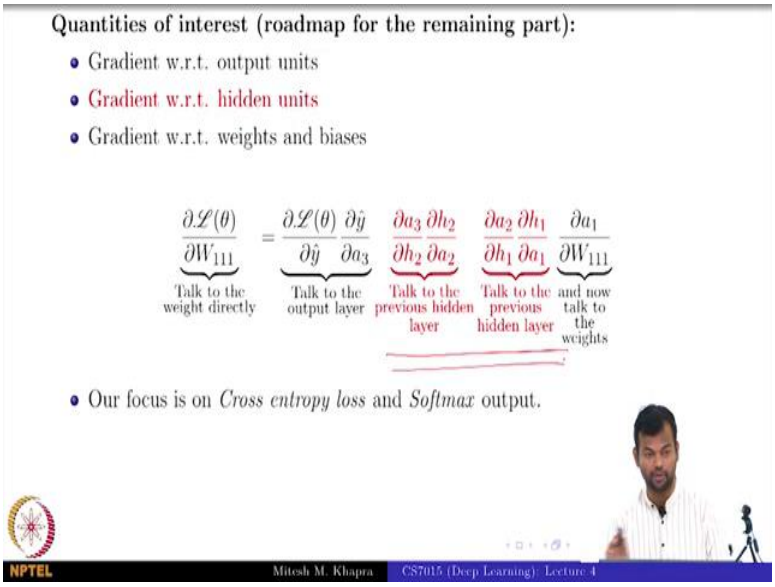
So, we started with the dark green guys and then went to the light green guys. Now we have the derivative with respect to the entire light green vector. And that is what we had started off with that we wanted the gradient with respect to the output units.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 4.6
Lecture - 04
Back Propagation: Computing Gradients W. R. T. Hidden Units

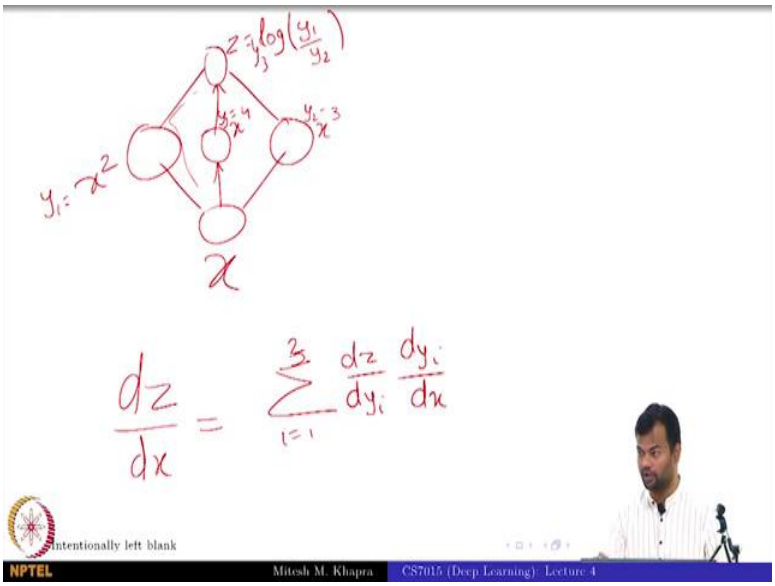
Now, we will go to the Gradient with respect to the Hidden Units.

(Refer Slide Time: 00:16)



So, this portion. So, you already see there is a repetition, here and I do not need to treat each hidden unit separately I can just have a formula for the hidden unit and then I could compute it for all the hidden units. So, that is what our aim is. So, let us do some simple stuff first and then you will come back to it.

(Refer Slide Time: 00:33)



So, suppose you have a variable x you compute 2 functions from that one is x square, the other is x cube. I will call this as y 1 and I will call this as y 2 and I take y 1 and y 2. And compute a z, which is say a log of y 1 by y 2. Now what I am interested in is this, what is the answer for this? How do you get this? This is a fair question to ask y 1 y 2 are functions of x, z is a function of y 1 y 2 hence z is a function of x. So, I can compute this derivative and I can ask for this derivative, how would you compute it? If I cannot really do this right?

So, if this path did not exist, then it is trivial it is just the chain rule along one path, but now you have 2 paths. So, what will happen add them right? So, can you tell me a formula for that? So, let me know if this makes sense to you ok does this make sense now let me complicate this a bit just let me just do it as y 3 now.

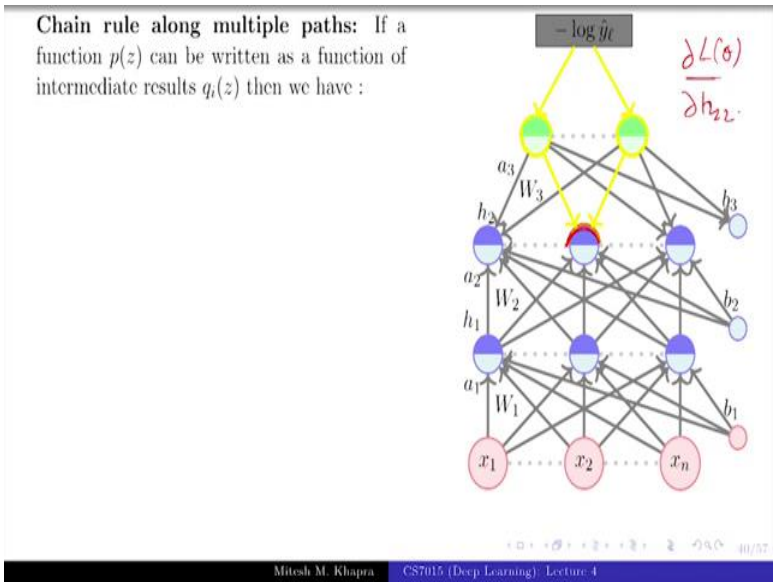
Student: (Refer Time: 02:15).

What will happen?

Student: (Refer Time: 02:16).

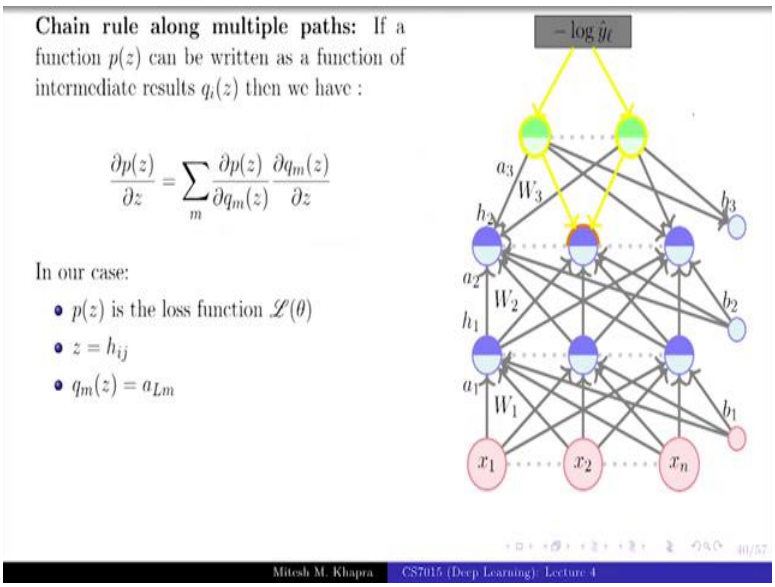
That is all right? So, you see that if there are multiple paths you can just add up the chain rule across all these paths right? That is what chain will across multiple paths does.

(Refer Slide Time: 02:28)



So, with this we will go back to this figure. So now, I am interested in I am interested in going to the hidden layers, again I will do this to bit calculation where I first asked for this guy and then I will ask for the light blue guy right and am going to look at 1 unit at a time. Now what is the, what am I interested in the derivative of the loss function with respect to say d h 2 2, right? The second unit of the second hidden layer.

(Refer Slide Time: 02:58)

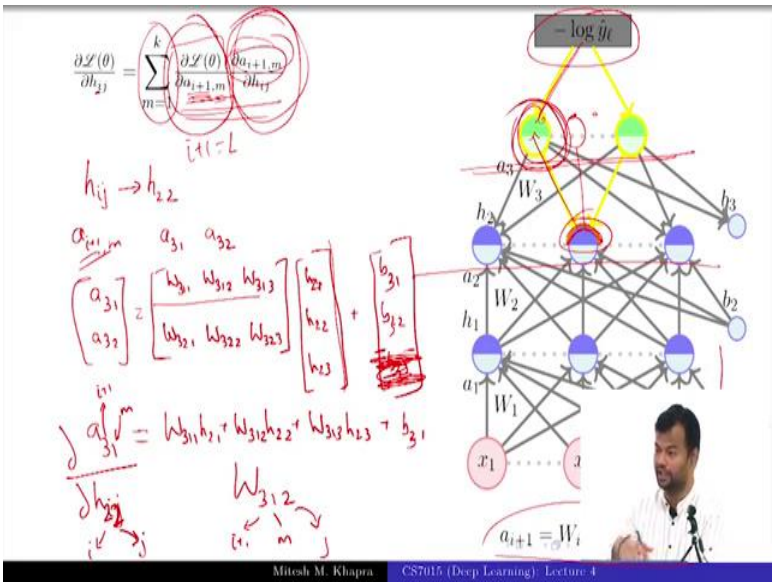


Now, what I am going to say here is exactly what I had written on the previous slide this was our final function, right which was z. So, z was sorry again I have not chosen my

variables well ok, but if. So, we had exactly the same situation, right? Which is which you see here ok. So, we will just have to sum up the derivatives partial derivatives across all the paths, which lead from this guy to this guy and there could be as many paths as there can be, but I do not care I will just sum across all those paths. In fact, actually here there are not just 2 paths because we have always assumed there are k classes. So, there are actually k of these paths right?

So, this form this is exactly the formula which I wrote on the next slide right this one, but just written in terms of the network that we are dealing with. So, you can just go back and look at this, but as long as you understand this figure you from my point of view we can go ahead. So, everyone understands this figure that we just need to compute the derivatives across all the paths and add them up.

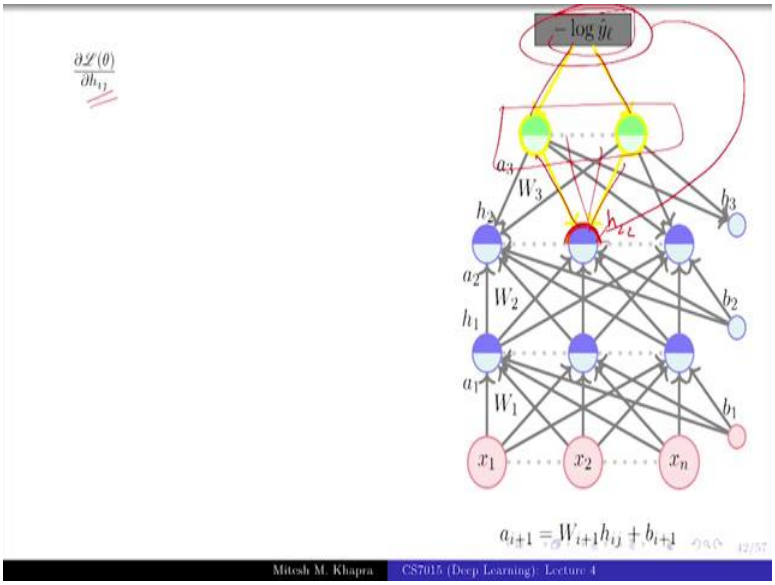
(Refer Slide Time: 04:03)



So now let us start we again the same recipe we will compute it with respect to one guy and then go towards the gradient. So, what is this now? Let me explain right. So, dl theta there are k of these guys between right? So, there are k paths. So, this summation has to happen over k paths just as you told me when there were 2 paths the summation was 2 3 paths to 3 that is k paths of the summation over k guys. The derivative with respect to each of these guys and the k'th the m'th unit rate that is the index that I am iterating over and then the derivative of this guy with respect to whatever you are interested.

That is just that there are only 2 nodes in the path in the chain, but there are k such chains, how many of you exactly get this? Ok how many of you have a problem want me to repeat this? You have problem oh many of you ok, good please do this. So, I am interested in this quantity; that means, I am interested in the partial derivative of this loss function with respect to this guy.

(Refer Slide Time: 05:06)



And this guy is nothing but h ij that much is clear is the j'th unit of the i'th hidden layer. In fact, this is actually h 2 2. So, my I is equal to 2 and j is equal to 2. Now I just made a case on the previous slide that, if you have such a function which first computes some intermediate values, and then your final function is computed based on all these intermediate values right. And now you are trying to find the gradient the partial derivative of this with respect to the original input that you had.

So, then what you will do is you will sum across all the paths that lead from this guy to the output, how many such paths are there? You already see 2 such paths here right, but I am saying there are k such paths, because there are some other nodes here which I have not drawn we have already said that in the output layer we have k nodes right? So, there are k paths. So, that takes care of the first bit that the summation is going to be over the k paths.

Now what is each of these paths composed of? This intermediate value and this quantity that we are interested in, first we will take the derivative of the out of the loss with

respect to this intermediate value, what is that? That is the unit in the, that is the unit in the previous layer or the next layer rather. So, I am interested in i. So, I am looking at the unit in the next layer hence i plus 1 right, because that is what comes in my path the next layer is what comes in my path. We have always the special case, That this guy feeds into k guys, but all the other hidden units before that feed into n guys right?

So, that is let us just keep that complication aside for the minute and we just look at this case ok, is that fine. So, we have agreed there are k paths and each path is composed of these 2 nodes, from the last loss function to this intermediate value and then from this intermediate value to the quantity of interest. And why is this i plus 1 because the next node in the path when I am at the i'th layer.

So, I will be feeding to the I plus 1'th layer right? And in fact, I will be feeding to all the nodes in the i plus 1 th layer, that is why I am taking or all the k paths right; and then that node which is this node with respect to the quantity, that I am interested in, is this clear? Now right this is very similar to the toy example which I did I just have k paths now instead of 2 paths there.

So, let us move ahead, now what is ok which of these quantities do we already know, is there any quantity that we know? This one, why? Because in this special case i plus 1 is actually equal to L right. Because we are feeding into the last layer and they have already seen how to compute the partial derivatives with respect to the last layer.

So, this quantity is known we do not know this for the generic case yet, but we will get that, but for this special case when we are feeding into the last layer we know this does everyone get this? Ok now do we know this quantity. So, what you have told me is that we know this quantity because that is what we have computed in the previous module. Do we know this quantity? We have to compute it; can you compute it? Ok let us just do it right. So, let us assume that this hij that am dealing with is actually h 2 2 ok fine. Now what is a i plus 1 m? Actually which are the elements there a 3 1 and a 3 2, I am assuming that I only have 2 units in the output layer ok.

So, my m is equal to 2. Now is this fine? this is how the next layer is related to the current hidden layer plus biases ok. Now what am I interested in one of these guys ok. Let me take one of these guys. So, can you tell me what is a 3 1, first row multiplied by the first column there is only one right plus b 2 1.

Student: (Refer Time: 10:11).

Sorry.

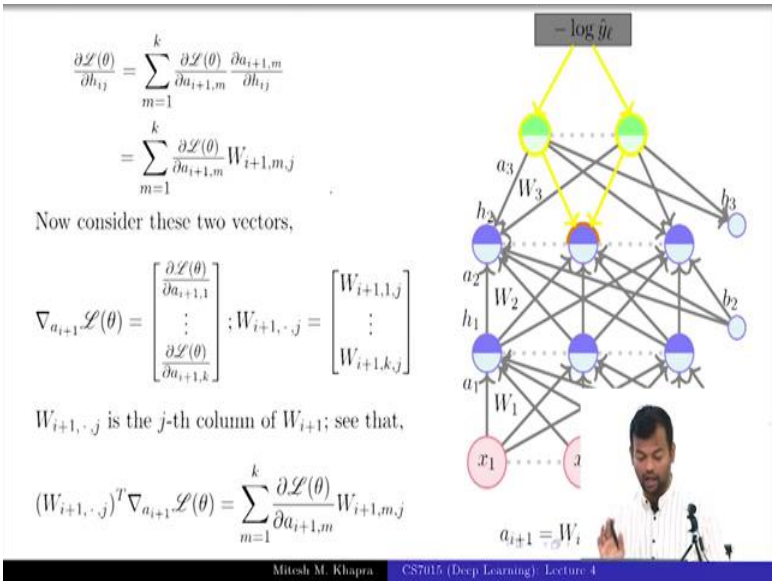
Student: (Refer Time: 10:12).

B 3 1 . Now let me just clarify something what is this in terms of variables i, j, k, m what is this? This is i this is j this is k this is m. This is i plus 1 right ok. This is one of the ms that I am dealing with. Now I want the derivative of this with respect to hij. In fact, I want it with respect to h 2 2 where this is i and this is j is this clear? What is this derivative, w 3 1 2 everything everyone fine with this. Now help me find this, what is this ijkm and i plus 1, what is this? This is coming from the m, how many of you see this? Because that is the unit that you are connecting to and this is j. So, what is the formula; how many? As many as the number of neurons in the next layer a bias will be connected to all the neurons in that layer right? Everyone gets that right there are only 2 units.

So, there will be only 2 guys ok. So, what is the formula for this W i plus 1 mj, everyone comfortable with that ok fine? You can just go back and look at this and it should be cleared right. So, whenever you are dealing with vectors and matrices right if you are really good at it you can imagine the entries and figure out what is happening. If you are not good at it do not be lazy just work it out, right? You just need to write down this product and at the end remember everything is always element wise and you are never dealing with a vector or matrix now just dealing with the individual components of them.

So, you should always be able to compute these derivatives or partial derivatives with respect to the individual components, and that is exactly what I did here, right? If you just work it out if you just write it out then you will always get it if you cannot, but eventually try to get to a point where you can just visualize it, but if you cannot at least try to work it out.

(Refer Slide Time: 12:25)



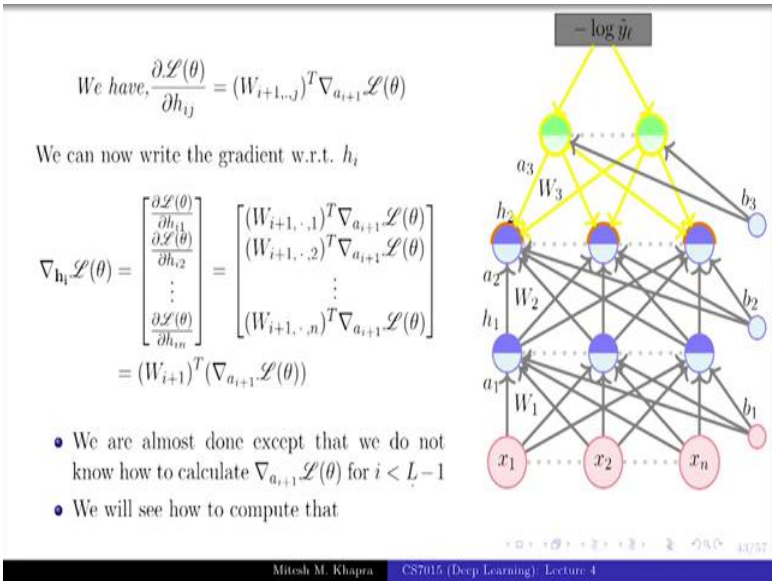
So, this is what it will look like ok. Now consider these 2 vectors one is this vector what does this vector look like, this is a collection of all the partial derivatives. So, this is just a collection of all the partial derivatives nothing new we have already seen this. Now what is this vector actually? In fact, I have started with the matrix and I am saying look at this vector, what does this mean? This i plus 1 is just the layer in which the matrix is right? So, that index we do not really care about, for a matrix what we care about is the i comma j index ok. Now what does this dot comma j mean? All the i's belonging to j; that means, the dash column j'th column everyone gets this, this is all the i's or all the entries belonging to the j'th column.

So, it is effectively just the j'th column. So, it is 1 comma j 2 comma j up to k comma j right? So, these are 2 valid vectors, now tell me what is this quantity going to be? This is the dash between 2 vectors dot product dot product between 2 vectors is a.

Student: (Refer Time: 13:43).

Is a summation over element wise thing ok. I have said enough now try to connect this is a very simple Maths the column that you will ever get in your life, try to connect this to something which is already there in the slide. How many of you think the answer is this? This into this plus this into this plus this into this and just write it as a formula you will get this everyone sees that ok. So now, I have a compact way of writing one of these entries.

(Refer Slide Time: 14:10)



One of these guys I have a compact way of writing this, it happens to be the dot product between 2 vectors one of them is the gradient, but do I know this already; do I know this quantity already? In this special case yes, because I plus 1 is equal to L and that I have already computed this of course, I know right because these are the weights that I am dealing with, where do I go from here? This dot yeah it means anything from that column so; that means, the entire column.

Student: (Refer Time: 14:48).

Ah no these are weights right? So, this is a weight matrix it has columns and rows. I am talking about the j'th column. So, I fixed the value of j. I am talking about the j'th column, but I am not telling your given i'th entry there am just telling you all the entries there that just means the j'th column you can take this offline ok. This is very simple I will take it offline ah. Now where do I go from here.

Student: (Refer Time: 15:16).

I plus 1.

Student: (Refer Time: 15:20)

Ok, no, in this specific case are we done.

Student: (Refer Time: 15:27).

Where are we right now with respect to one unit, where do we want to go? The entire thing. So, what is the quantity that I am interested in gradient with respect to always say with respect to h i right.

Student: (Refer Time: 15:44)

Where i is 2 in this case this special case ok. What is that going to be collection of all these guys that you have already computed ok. Now simplify this. What is this first column of the matrix? Multiplied by the same vector the second column of the matrix, multiplied by this vector, the nth column of the matrix multiplied by this vector this reminds you of something very, very difficult. This is a very, very complicated matrix multiplication right?

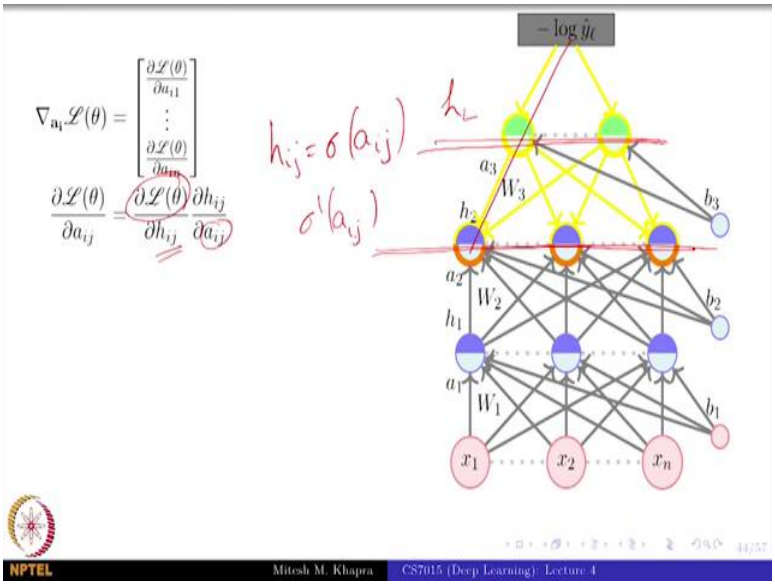
First row of the matrix multiplied by a column the second row of the matrix multiplied by column, how many if you get this; right? So, this is can you tell me what this is Wi plus 1 transpose.

Student: (Refer Time: 16:52).

Perfect right? So now, you see that this entire quantity we can compute in one go by using a matrix vector multiplication right. So, that is what I meant; when I was saying that we should not be doing these unusual computations, but we able to compute that at one row right. So now, we can just do this matrix vector multiplication and get this entire quantity ok. Now, what is still missing in this module.

So, what is the special case that I have assumed, I told you that I already know these quantities, but only if i plus 1 is equal to L. I need to tell you this in the generic case ok. So, we are almost there except that I do not know this when i is not equal to L or i is less than equal to L minus 1 ok; that is the case that I am looking for.

(Refer Slide Time: 17:38)

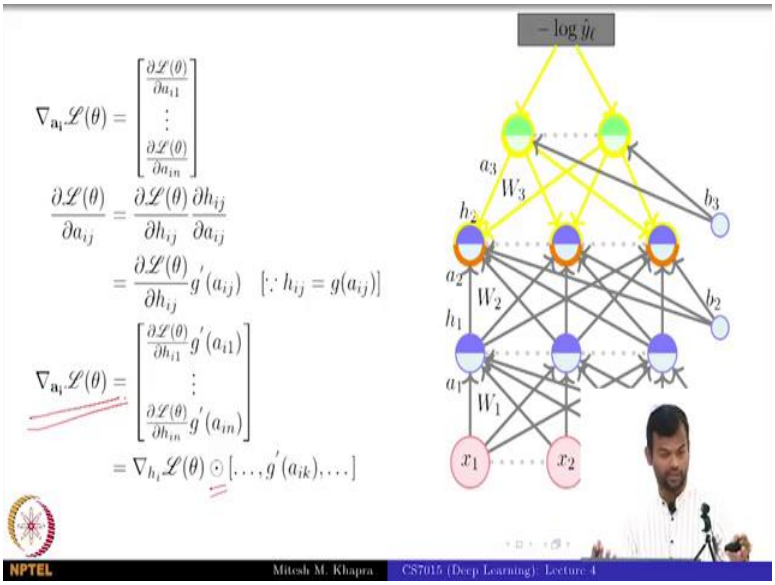


So, that is again very simple, again what will I do? I will compute it with respect to ok. What is this? This is the guy that I am interested in the generic i not the L'th one right the generic i. This is what the vector looks like the gradient vector looks like. I want each of these guys ok. Now I will take one of those and I will write it as this ok. What am I doing? Am saying that, I already have the entries up to here ok at a very general level even here I could have said the same thing, remember that I had said that the output layer you can always write as hL, right?

So, even at the output layer I could say this chain rule always holds, how many of you agree with that? I want to go from the loss function to one of the lighter blue guys. So, am saying that I can go through the intermediary dark blue guys, that is all I am saying. I have just compressed this entire path into up to the dark blue guy. Remember I had said earlier that I will be compressing this chains. Now how many of these quantities do you know? The first one is what we computed on the previous (Refer Time: 18:52). The second one looks very difficult sorry.

So, h ij is nothing but sigmoid of a ij or any non-linearity of the a ij. So, I can just write this derivative as I will just write it as sigma prime ok.

(Refer Slide Time: 19:10)



Or g prime is this fine. Now I have it with respect to 1 unit, what will I do? Go to the gradient fit it all these values. Now simplify this, what is this? A vector right, what is this? Another vector, there is a one to one correspondence between them. So, you have 2 vectors and you are doing a 1 to 1 multiplication, what is this?

Student: (Refer Time: 19:43).

How many of you say dot product? Dot product is always a, what is the output here?

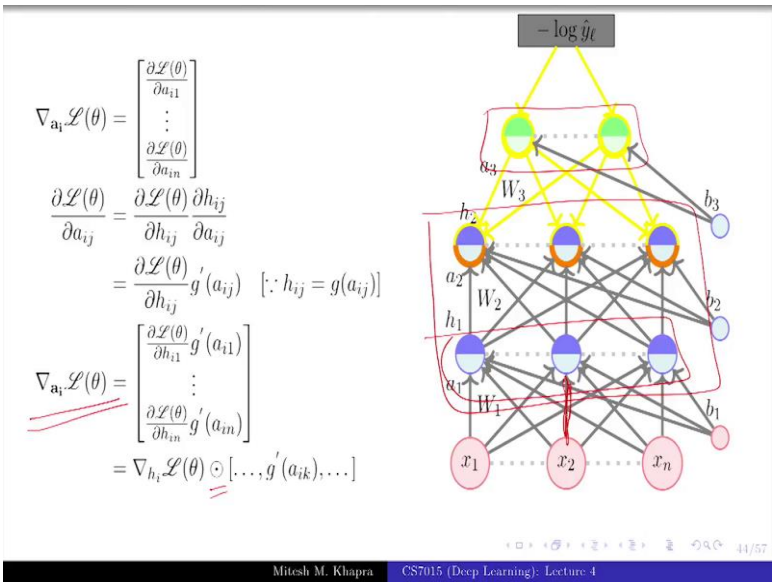
Student: Vector.

Can it be a dot product; can it be a dot product? No please empathic no ok. So, what is it going to be? An element wise multiplication and this is how you denote that ok. So, what is this called? You had a multi-product right. So, this is every element of one vector multiplied by the corresponding element of the other vector ok. So now, again the entire vector we can compute at 1 row right; I am not I am when I am teaching this I am telling you how to compute one element and then go to the gradient, but when you are going to implement this we are just going to compute the gradient at one go.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 4.7
Lecture - 04
Back Propagation: Computing Gradients w.r.t. Parameters

(Refer Slide Time: 00:16)



Before we move on to the next module, a quick summary of what we have done so far. So, we introduced feed forward neural networks, and we wanted to learn the parameters right from the last layer to the first layer. And we figured out that what we can do is that we can just use the gradient descent algorithm as it is; except that, we have this small problem that we have so many parameters now, and located at differ different points in the network, right some at the initial layer some at the final year, and you want to compute the derivatives or the partial derivatives with respect to all of these.

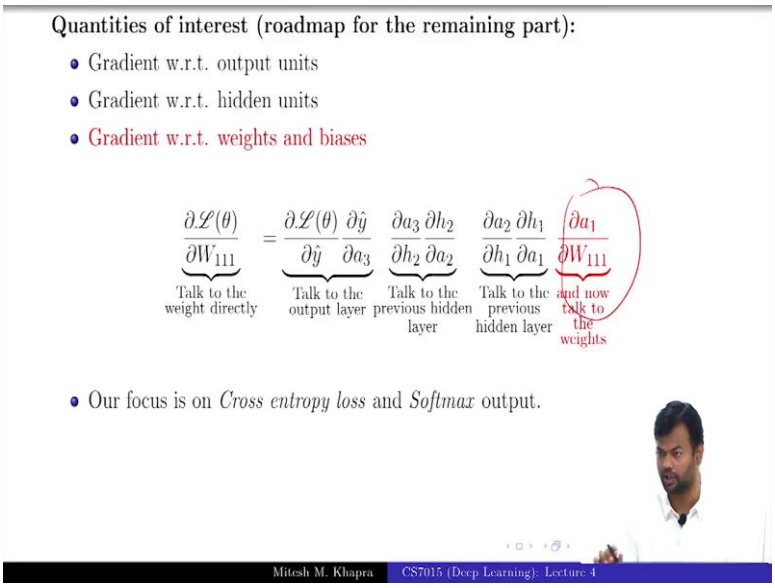
If you can do that, put them all in this large matrix, then we can just use gradient descent as it is. So, that is what we figured out. And then we wanted to find out the gradients with respect to or the partial derivatives with respect to all these parameters. So, then we realize that this can be done using chain rule, because there is a path from your output which is the loss function to any of these weights. So we just need to follow that path and apply this smart this chain rule smartly and just sum up the derivatives across all the

paths that lead to that weight. So, in that process we started from the output layer, we just treated it a bit special, because the output function is special and this is the last layer.

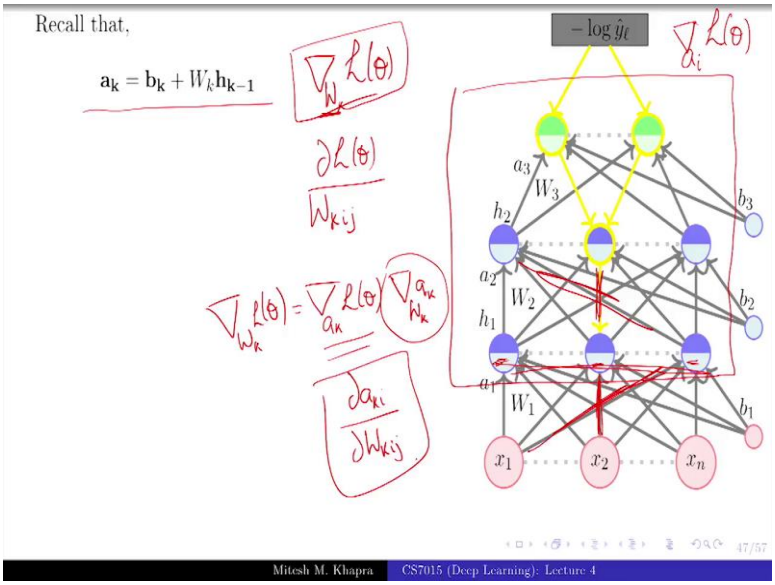
So, we just first computed the gradient with respect to the output layers, then we figured out how to compute the gradients with respect to any of the hidden layers. And now if you are at a particular hidden layer, now the weights that feed into this layer we could or we have not reached there.

So now, the next thing that we need to do is that we have computed the gradients with respect to any of these hidden layers, and now we want to find the gradients with respect to the parameters which is the weights and the biases. So, it is the do you all remember this, or it is all long history or the story is back right, fine. So now, we are at the last point which is computing gradients with respect to parameters.

(Refer Slide Time: 02:04)



(Refer Slide Time: 02:14)



So, again this is the overall picture, we were in this chain rule, and we have come all the way to the last point where we are ready to now compute these quantities. So now start by recalling that a k is equal to b k plus w k h k minus 1, right, this is our activation formula, pre activation formula right. So, I am talking about these light blue guys, ok, which is clear in image.

And now I what have I done so far? I have been able to come up with a formula to write the gradient of the loss function with respect to any of these light green guys, right, that is what where we ended last time right? Where we are able to compute the gradients with respect to the, sorry light blue guys ok. And now I want to compute the gradient with respect to any of these parameters or any of these parameters.

So, any parameter, it does not matter am at some i'th activation layer, pre activation layer, and I just want to compute the gradients with respect to the weights which feed into this layer, and that is what we are interested in. So we are just taking any layer k, and you want to find the gradient with respect to the weights there, now can you tell me? So, can you tell me what is what is the thing that am going to do here? Or what is the recipe that we have been following?

I need to move, what is the recipe that we have been following? Apart from yelling at people who come late, we find the element wise partial derivatives first and then put them all together to get the gradient ok, what is the element here? What is what am I looking for right now? I want to compute this fill this blank, what goes here?

Student: W.

W any of these W is right? And in particular say W k that is what I am looking for. So what is the first thing that I am going to attack?

Student: Wkij.

Good. W k i j and once I have this for one of these guys i just know a generic formula with respect to i j and k and I can just put it into a gradient vector ok, is that fine? Ok so now can you, ok, now from here to here if I want to reach from here to here. So, this is what i am interested in, right? Now how is the chain rule going to look on, look like based on whatever you have already seen? Till where have you already reached? You already know this quantity, right? Now if I want this how am I going to write it?

Student: (Refer Time: 04:59).

I will find up to the light blue guys; which is this I already know how to compute it, and then from the light blue guys I will go to the this; is fine right? So, this is the quantity that i am looking for, ok. Now what is one element of this guy? Dou a k by; is it fine? Ok what is the dimension of this actually? Is it a scalar, a vector, a matrix, matrix or a tensor, what is the tensor? What is it? Is it a matrix? What are the dimensions? What does this derivative mean? Or this gradient mean? I change one element of W k how much does one element of a k change? How many elements are there in ak? n, how many elements are there in W k? n cross n. So, how many partial derivatives which I have? n cross n cross n, what is this?

Student: Tensor.

A tensor, right? So, this is going to be a tensor, ok. So, when I say one element of this, I mean this ok. So, this is one element of this gradient, ok. Now can you tell me the formula for this? What is this quantity? Hk minus.

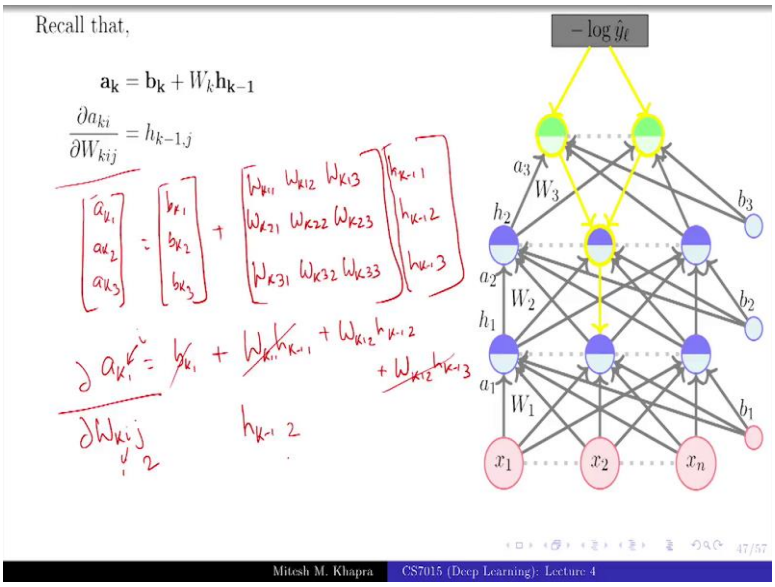
Student: 1 (Refer Time: 06:27).

hk minus 1 or hk minus 1 j or?

Student: (Refer Time: 06:31).

Everyone gets this hk minus 1i. How many of you get this?

(Refer Slide Time: 06:38)

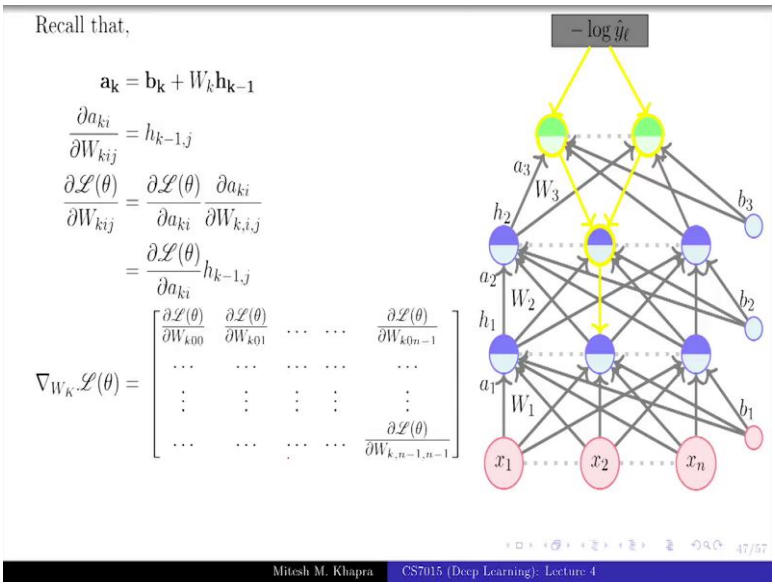


So, let us do it, right? So you have ak1, ak2, ak3 that is your ak vector, ok. You have bk1, bk2, bk3 plus wk1 1, yeah, I know again this is one of those silly things, but if everyone does not raise their hands and compelled to do this; so, h k minus 1 1 hk minus 1 2 hk minus 1 3 ok. So, let us take one of these guys right. So, a k 1 can you tell me the formula for that?

Student: (Refer Time: 07:30).

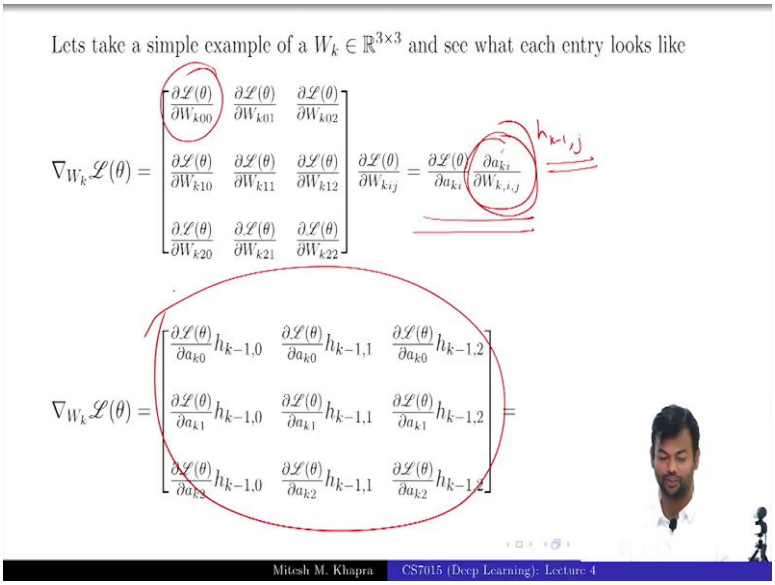
Plus, first row ok, 1 2 this 1 3, now can you tell me this quantity? So, what is i here? 1 ok so, i want this by w k i j right so, i is 1 so, i can take any of the j. So, let me take j equal to 2. So, what is it going to be? This will go off this is constant, this is constant only this term remains, and the derivative is hk minus 1 2 which is j right. So, that is what the formula says. So, I have a formula for one of these guys, ok and that is a generic formula. So, always remember if you cannot figure out what it is just write it down in scalar terms, just add up all the terms and you will get the formula, right. So now this is what the chain rule is going to be?

(Refer Slide Time: 08:37)



So, this is what it is going to be. This is one element of that tensor. This is how that entire thing is going to look, I have just flattened it out and put it here.

(Refer Slide Time: 09:05)

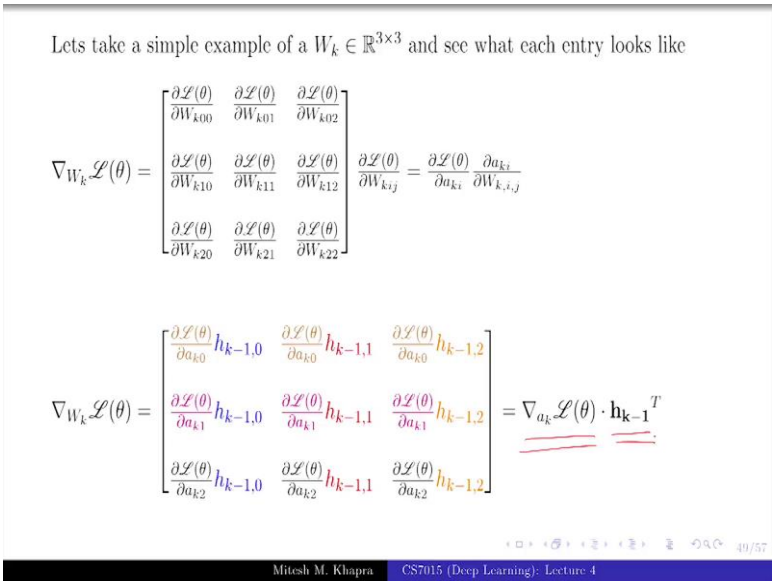


Now let us take a Simple example of wk belonging to r cross; 3 cross 3 everyone is fine so far right or anyone who everyone is fine please raise your hands. I mean fine I mean not in life, but with the lecture fine. So, this is what it looks like right for a 3 cross 3 matrix.

Now, let us see we already found out that this guy is equal to hk minus 1 comma j right. So, this is what this matrix looks like, nothing rocket science here right. So, each of these quantities is actually can be written in this form; where i appropriately substitute i k and j. And I know that this quantity can be further written as this quantity, right? That this is our clear right so, I have written it as this.

Now, can you simplify this, I do use a lot of this ok, can you simplify it? Is it looks similar to something that you did on the assignment. Does this look like matrix which has some very regular patterns? Yeah I can see someone doing this, and this everyone gets it.

(Refer Slide Time: 10:28)

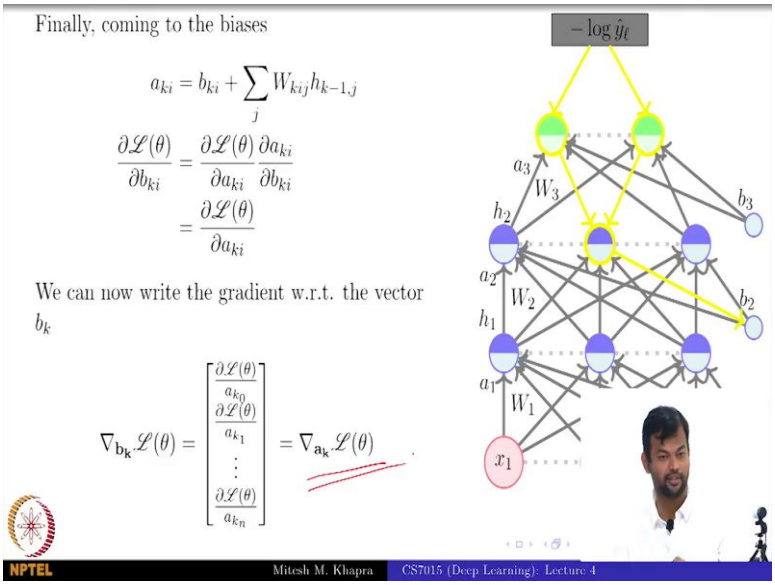


So let us see so this the first column, the second term in the product is all same throughout all the rows, right? What I mean is all these guys are similar, same thing happens in the second row the third row, right ah? That is sorry the second column and the third column. What about the rows? These are all equal right, so what does this look like actually? The outer product of 2 vectors, everyone gets this? Raise your hands? Ok good.

So, I do not need to do an example. So, it is fine right this is an outer product of these 2 vectors, one happens to the quantity to be the quantity that we already knew, right? And the other happens to be a quantity that we can figure out. I mean we already know this,

what is we know how to compute the hidden representations, right? The hk's we can compute.

(Refer Slide Time: 11:20)



So, fine so, finally, we come to the biases. This is what one entry looks like, this is exactly the sum which I had written out. Now I take the derivative with respect to b k i of the loss function. So, I could write it into as this chain rule; where the first quantity is something I already know I have computed the gradient with respect to the pre activation layers what about the second quantity? Anonymous roar is what I was expecting.

Student: 1.

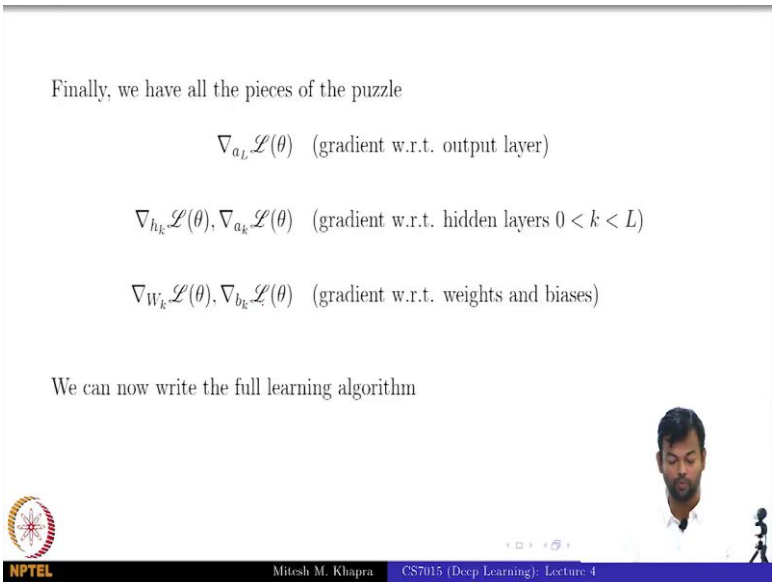
1 ok, fine, we can now write the gradient with respect to the bias, what would it be? What is this? What is this? It is just the gradient with respect to the pre activation layer, right simple. So now, we are done with all the gradients that we were interested in.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 4.8
Lecture - 04
Back Propagation: Pseudo Code

So, we move on to the next module and now we will write pseudo code to for back propagation.

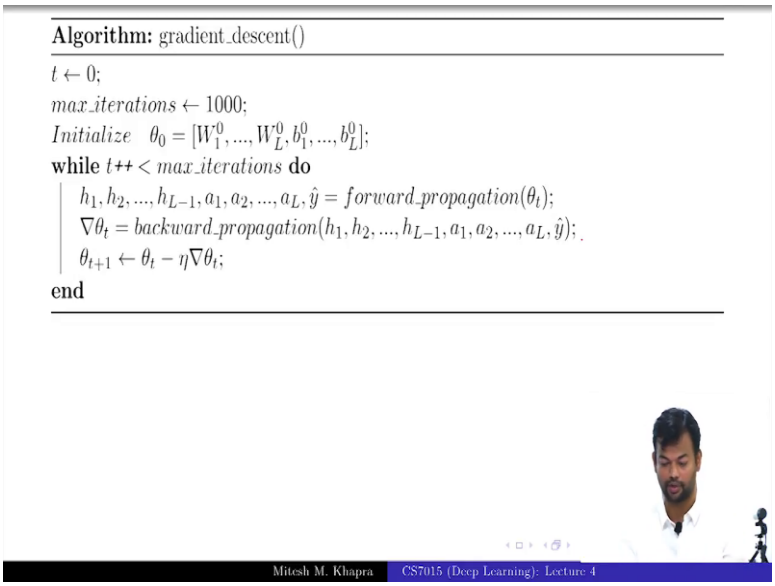
(Refer Slide Time: 00:23)



So, we have all the pieces of the puzzle, we have the gradients with respect to the output layer, that was the special layer because, the output activation function is different. They are the gradients with respect to all the hidden layers; that means, I have the gradients with respect to the activations as well as the pre activations.

So, in the h's as well as the a's and I also have the gradients with respect to the weights and the biases and this is all index agnostic right; that means, I am just using k as the index everywhere, I have a generic formula, which applies at any layer for the weights as well as the activations and the pre activations right ok. Now, we can put all this together into a full learning algorithm. So, let us see what the pseudo code looks like.

(Refer Slide Time: 01:03)



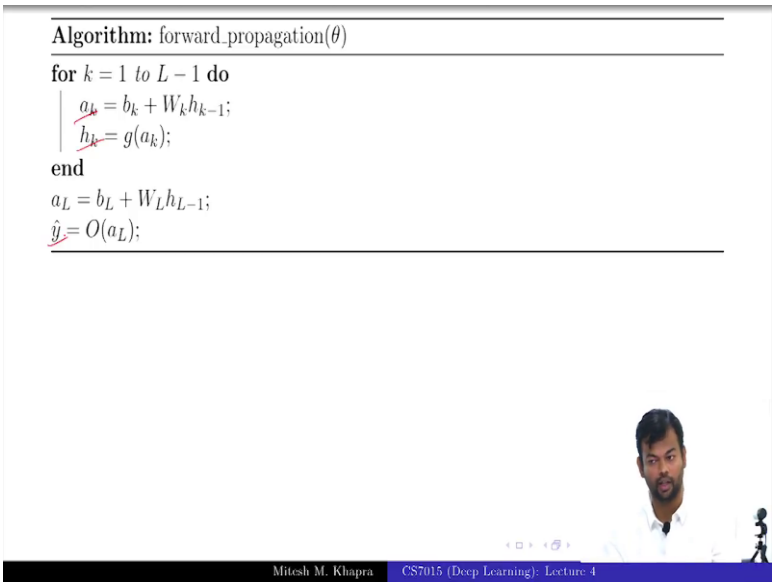
So, we have this t equal to 0 well run this for some max iterations we initialize all the parameters to some quantity will randomly initialize them ok. Now, for these max iterations, can you tell me what is the first thing that I will do? So, there will be 2 functions here ok. Tell me what those 2 functions would be.

Student: Forward.

Forward propagation and then backward propagation right? So, you do a forward propagation and you compute all these activations pre activations output layer loss everything and then you do this backward propagation where you feed all these things which you have computed. These are the quantities which you have computed; you will pass this to your backward propagation algorithm it would not look. So, nasty as this it will not take. So, many parameters you could write it smartly and then you will just do the parameter update.

So, what will the back propagation give you actually all the gradients all the partial derivatives right and then once you have the partial derivatives, you know how to compute the update law. So, now, let us look at these 2 functions more carefully the forward propagation and the backward propagation.

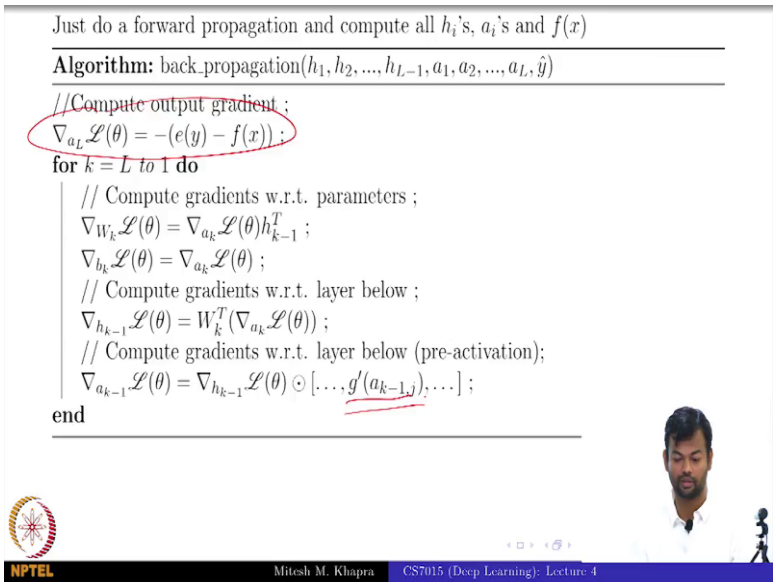
(Refer Slide Time: 02:09)



So, forward propagation is simple for all the hidden layers; that means, from layer 1 to layer l minus 1 what will I do give me the code a k is equal to good then ok. And what it what is h of 0 you are starting the loop from 1 right? So, you will need h of 0 that is x and then you will have a special treatment for the output layer and your final output will be whatever output function you use ok. This makes sense you can write this in python.

You will have to write this in python.

(Refer Slide Time: 02:45)



Now we have computed all the h's and the a's what have we computed all the a's, all the h's and all and the Y, right now you want to do back propagation. So, back propagation the loop will be from i equal to 1 to n minus 1 good. So, the first thing I will compute is the gradient with respect to the output layer. See, even here the output layer was outside the loop. The same thing would happen here also. In the back propagation also first you will compute the gradient with respect to the output layer and this is the formula.

If you remember from last class right, that is the formula which I have substitute here and note that f of x is known to you because you computed that in the forward pass and e of y 1 hot vector which with a correct label said to 1 and you know what the correct label is because, we have given you the (Refer Time: 03:14) data right ok then what would the loop be l to 1 or l minus 1. Let us see first you compute the gradients with respect to parameters. It is l.

So, because, we are using k minus 1, then you compute the gradients with respect to the layer below computes gradients with respect to the pre activation right. This is exactly how you will proceed this. Is clear to everyone? The same 3 components that we have used you might be a bit confused about the ordering in which we have put them because we computed the gradients with respect to pre activation first and then the weights, but once you go back, you will realize because it is the way we have indexed it because this is already outside.

So, this has already been computed. So, you can already compute the gradients with respect to the weights of the outermost layer, is that fine? So, this is straightforward you can go back and check this ok now anything remaining, or you have everything can you just take a minute and see if you can visualize the python code and we will just assume that you are done the assignment you can read you will have multiple these vectors and matrices and so on. And you are just doing a lot of matrix operations using (Refer Time: 04:06) or (Refer Time: 04:08) or whatever you prefer right.

Now, what is missing here? Input is missing, ok. Input we have given right. The ominous data set has been given is there something that yours I have still not shown you how to compute; oh I did not update the parameters here is it? No the parameter update will happen in the outer loop, right. So, those forward prop back prop and then update the parameters right. So, the main algorithm was forward prop back prop update the

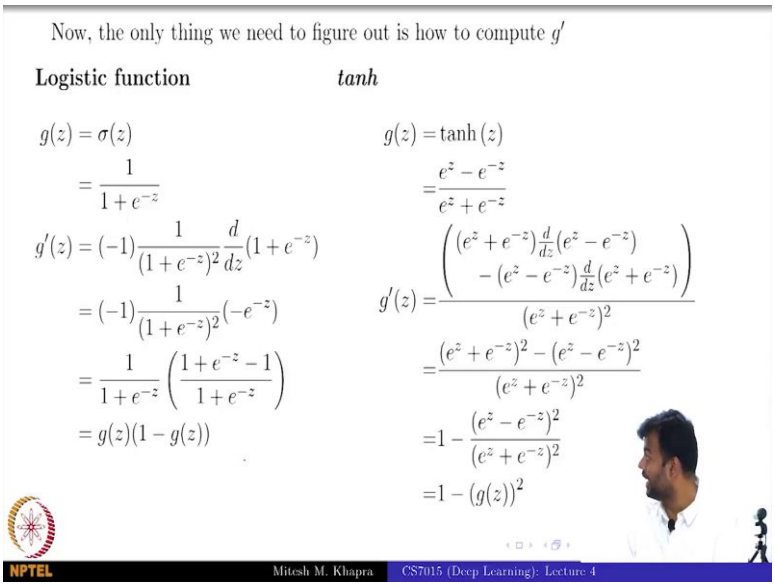
parameters, when we saw forward prob an obvious seeing backward prop. So, what is missing 1000 iterations? Something in the last line before end of course, do you know how to compute this?

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 4.9
Lecture – 04
Derivative of the Activation Function

We have that activation function and we were taking the derivative of the activation with respect to pre activation. And I just pushed it under the rug by saying. We will write it as g dash. So, I need to show you what g dash is.

(Refer Slide Time: 00:29)



What how to compute g dash? So, this is suppose g is the logistic function ok, so that means, what is z actually? It is one of those a's right. So, this is the activation that you are going to feed it right and then you are taking the element wise sorry z is actually the pre activation that you feed it and then g is the activation function. So, I will do element wise activation function. Now what is the derivative of this? So, I will just I will not do this derivation.

It is there and you end up with a very neat formula which is g of z into 1 minus g of z. So, now, that bit is also taken care of is there any more spoon feeding that I can do? You are ready for the assignment now. I will do one more bit. You will also have used a tanh

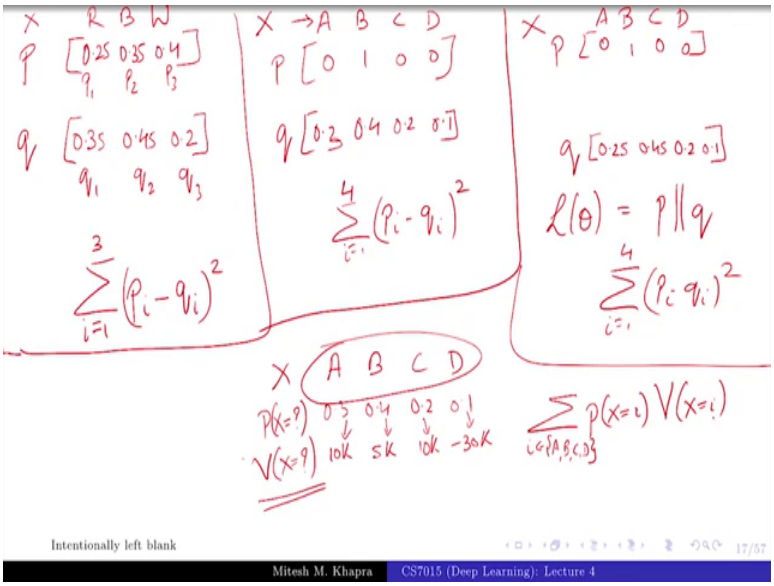
function. So, this is the derivative of the tanh function. It again boils down to a very neat formula which is 1 minus g of zd whole square. So, we will end this lecture.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 4.10
Lecture – 04
Information Content, Entropy & Cross Entropy

So, for the next module we need something known as Cross entropy. So, we will just try to make some develop some intuitions for cross entropy and get to the formula for that. And then I will tell you how it relates to the problems that we deal with, ok. So, first let us start with something simple that what is it that we are trying to do, ok. So, with that I will give you an example and I will ask you a few questions and then from there we will slightly try to go towards cross entropy. So now, suppose you have an urn which has thousands of balls, and these balls are of three different colors which are red black and white.

(Refer Slide Time: 00:51)



So, you have an urn which has three different types of; and there are many such balls which you have put in it. And since you have put in it you know how many red balls are there, how many blue balls are there, and how many white balls are there. So, for you it is very easy to compute the probability of each of these things.

So, that is say our probability is 0.25, 0.35 and 0.4. Now, talking more formally what is happening here is that, you have a random variable X which can take on the values red blue or white, right? And this is the probability of each of those or the random variable X taking any of these values, ok. So, this is the setup. Now I am your friend and you tell me that you can peep into the zone, you cannot actually count take out all the balls and count them and estimate the probability. We can just take a look into this, and try to give me an estimate of what these actually; what are these probability values; that means, what is the value?

So, what is the probability that X is equal to red or X equal to white or X equal to blue. So, I just take a look at it turn it around a bit, and try to get some feel, ok. I see a lot of red balls, but a fewer blue balls or white balls and so on. And based on that I make my best estimate, right? So, I will just say that maybe these probabilities are 0.35 0.45 and 0.2, right? So, this is actually the true distribution, I will call it as p, right, because this is the correct one. And what I have estimated, I will call it as q.

And remember now p has you can think of p as a vector which has these 3 values, p 1, p 2, p 3 because there are 3 possible events here. And similarly q has 3 values, q 1, q 2 and q 3. So, in this case I clearly know that, I am wrong or when I give you these values you know that you are wrong. So, you tell me that whatever you have estimated is wrong. Then I obviously, ask you. Tell me how wrong was I? So, how would you give me that number? That is the thing that we are interested in.

So, the general problem that we are interested in is that, there is a true probability distribution, and there is an estimated probability distribution, and we want to find out how bad was the estimation. Now, can you tell me a simple way of computing this? It may not be correct, but still it makes sense.

Student: Squared error.

You can just take the squared error. So, what you are essentially telling me is that you could just treat these two as any other vector, right. And you could take the squared error difference between these quantities. So, what you are telling me is this, where i goes from 1 to 3, ok. So, this is one valid way of doing this, but then we are ignoring the fact that this is a distribution and hence, it has certain properties that the sum of the elements is 1 and so on all of them are positive and things like that. So, we are ignoring those kind

of things we are completely ignoring the fact, that we are not dealing with a normal vector, but a spatial vector which happens to be a distribution. So now, we want to find out a more principled way of computing the difference between two distributions. And in practice why are we interested in this? Because we will always have a true distribution and a predicted distribution. So, that is what we want to do, we have some way of computing it, but you want a better way of computing it.

Now, let me make a case for why do we care about such differences right. So, let me take a simple case of a classification problem. And to motivate that I will start from a different example and then I will come to the classification problem. Suppose there is a tournament going on, and there were 4 teams which leads the semifinals; let us call them A B C D, ok. Now, you were following the tournament up to the semifinals, and after that you didn't watch the tournament, and you do not know who eventually won; well, the tournament is over and someone has won it.

I actually watched the tournament and I know that B has won it. Now can I express this in terms of a probability distribution right? So, first let us look at what is the random variable here; what is a random variable here? The team which won, right? So, that is my random variable and it can take one of these 4 values.

Now, I know that team B won, because I saw the tournament, and I have seen that they won. So now, how can I write this as a distribution? What is the distribution comprised of? It comprised of these probabilities assigned to each of these events. And there are 4 such events here. So, how do I write this distribution? So, what you are telling me is I could write it as 0 1 0 0.

So, essentially they are telling me that all the probability mass is focused on one of these outcomes, because that is the certain outcome, that is already happened no one can change it so, that is the outcome for this tournament. So, I know that the probability of that even is event is 1 and everything else is 0. So, in other words the probability that the random variable X takes on the value B is 1 and everything else is 0.

So, what I am trying to tell you is that, even for a certain event, you could still write it in terms of a distribution, where all the mass is focused on that event. Now again I will bring the same setup that I did not watch the tournament after the semifinals. So now, you ask me give me your prediction, what which team would win. Or this is the

prediction which I made before just after the semifinals or, just before the semifinals that I think one of these teams is going to win the tournament, and the chance of each of them winning is something like this. So, I know the teams I follow this sport and I probably know that, ok. B has a very strong team and they have a very good record in the past few months and so on. So, maybe they have a higher probability of winning. So, these are the numbers which I assign.

Now, again I have made an estimate, was my estimate perfect? When would it have been perfect? If I had predicted with certainty won that B is going to win, but I was not willing to bet everything on B. So, I said there is a very high chance it will win, but there is still a chance that there could be some surprises. Now, how wrong was I? Now again tell me can you tell me what is p and what is q here? This is the true distribution and this is my predicted distribution. And what am I interested in again? The difference between them, how wrong did I go? And what again what is a simple way of doing this again, square errors. So, again this is what my formula would look like.

So, this is fine in this toy case, but why do we care about in real life? Examples, that we are going to deal with in machine learning. So, in watching learning will deal with a lot of problems which are classification problems. And in classification problems, you would again have this setup where you have a label the good thing of the label as a random variable, and it could take off one of many values. So, I will again assume that it could take suppose you are trying to take a picture of fruits. And you are trying to classify them.

And I could again think that I have 4 fruits say apple, banana, cherry and dragon fruit, ok. And this random variable can take one of these 4 values depending on the image that I am seeing, ok. Now I have been given some training data. So, for every training data I have been given an image, and I have been given the correct label. So, for that training data what is this distribution? Suppose I have been given the image of a banana, what does this distribution look like? 0 1 0 0, right again I have seen it so; I know it is certainly a banana.

So, I do not have any confusion all the probability mass is focused on that. Now the same image we are going to show to one of our models, ok. And it is going to make a prediction, and will again ask it to give us a distribution, the model will give us values

perhaps like this, ok. So, this is the models prediction, again the model has given us a distribution and we have a true distribution, and we are interested in knowing how wrong the model was. So, that.

Student: Correct the (Refer Time: 09:26).

We can correct the parameters of the model. So, this is our dash function loss function, right. So, a loss function is some notion of difference between p and q, right. And so, far we have been dealing with a very simplistic notion of this difference which is just the squared error loss, ok. And we want to do something better than this right? So, what I have told you is that you could always have a true distribution always have a predicted distribution. And you would be interested in finding the difference between them, that is the one first part; the second part is that even when you are given something with certainty, you could still write it as a distribution such that all the mass is focused on that event which was which has happened right, which was the label was banana in this case.

And then you could still predict this from your model, and now you are interested in knowing how wrong you are model wind because, that is the loss function that you will use, and then you will try to update your parameters with respect to this loss function, means that is the setup that we are interested in. So, that is so, I made a case for why we need to find differences between 2 distributions. How to do it? In a more principled manner we have not seen that yet we will get to that, ok. So, before I get that, I also need to tell you something about expectations. So, let us written to as sports example where there were 4 teams. And say based on pundits and that sport, they have said that these are the probabilities of winning.

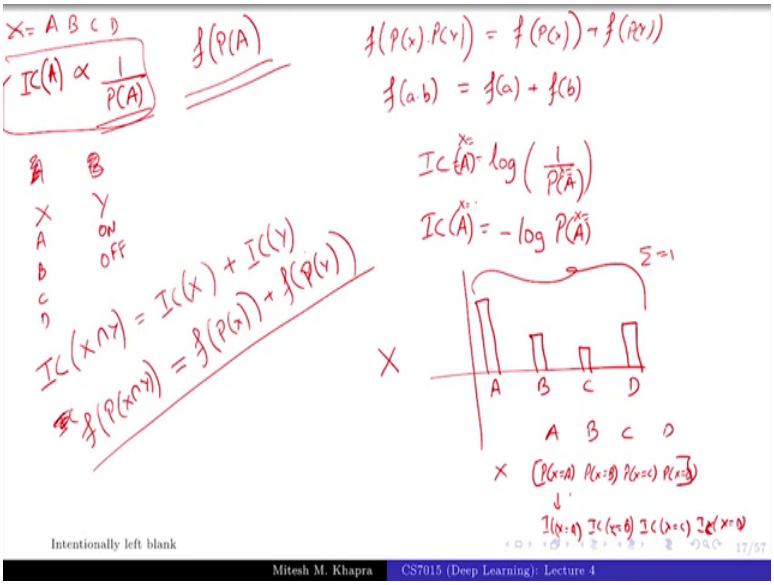
And now you are into betting, and you bet place your bets on these teams and you place our bets in a way that, suppose team events then you end up winning 10 k rupees, if team B wins then probably will end up winning 5 k rupees. And if team C wins probably 10 k, and if one of the other team wins maybe will end up losing money or something like that. Now you want to know, what is your expected reward? So now, let us see what is happening here? This was a random variable which could take on one of these 4 values. These are the probabilities of the random variable taking that value, taking on the value A or taking on the value B C and D, ok. This is your value or the gain or the profit associated with the random variable taking one of these values.

So, you have a random variable, you have a probability associated with every value of the random variable, and you have some gain or value associated with every value of the random variable. Now how do you calculate the expected gain or expected profit which is this? There is a 30 percent chance that you will earn 10 K, there is a 40 percent chance that you will earn 5 K, there is a 20 percent chance that you will earn 10 K, and 10 percent chance that you lose 30 K.

So, the way you will compute it is that and this is the simple expectation formula; which is the probability of now the event here belongs to ABCD, right. This is one of the 4 teams that will win, probability of that event happening in to the value associated with that event happen. This is a fair computation you get the intuition that this is how you will compute the net reward that you have. So, this is how you compute the expected value with respect to a particular distribution. So, this is the background that we need.

Now, I will just go on to the next slide, and now we will talk about entropy first, perhaps information content be first then entropy and then cross sector, ok. So now, what is information content?

(Refer Slide Time: 13:28)



So now, again let us take the same case that we have a random variable which can take on values A B C D. Now let us what we are trying to say is that, if I know a certain thing, what is the information that I have gained? So, you and I are talking you tell me

something. And I want to see whether my information was enhanced, whether my knowledge was enhanced, that is how we will quantify information content.

So, if you are talking to me and you tell me that my name is Mitesh, the 0 information content for me, right? Because I already know that there is no surprise in that ok, but if you talk to me and you tell me that today there is going to be a lunar eclipse, then there is a possibility there is some information content gain for me, right? Because that is not a event which happens every day if you just tell me you will see the moon today and you live in a region where it is not typically cloudy and there is no information gain there, right? So, what do you see here? When is the information gained high? When the event which happens is a very surprising event; and how do you say supplies in terms of probability.

Student: (Refer Time: 14:29).

It is a very low probability event, right? So, if there is again this tournament, and say D was the weakest team in the tournament, and A was the strongest team in the tournament if you come and tell me that D won then I would be really surprised that some information which I had gained, but if you tell me that A won then probably I already knew it at the back of my mind. Because A is clearly the strongest team in the tournament and there is no information gained for me.

So, one thing that we are trying to establish that the information content I see, ok; the information content of an event is inversely proportional to the probability of the event. There is a that is a fair intuition. Fine, now I want am still talking in terms of vague things I am saying it is inversely proportional, but I still need an exact function so that I can compute it. So, I want something I want a function where I plug in the probability of an event, and I get the information content of the event.

Right now I do not have that function I am just building some intuition towards that function, ok. But this is one requirement that I want the function to satisfy this is something that all of us agree with, ok. Now think of 2 events which are independent A and B, ok. So, A is the event whether the AC is on here or not, and B is a event which tells, maybe. So, let us consider 2 different random variables.

So, X is the random variable which can take on values 0 and 1. Sorry. So, X is again this random variable which can take on these 4 values ABCD whether who won in the tournament. And Y is this another random variable which can take on the value on and off depending on whether the A C's on in this room or not. What can you tell about these 2 random variables? They are independent random variables.

So, this is on or off and this is which team won the tournament. Now I come and tell you something about the random variable Y, and I come and tell you something about the random variable X, ok. So now, I want you to tell me this, what is the information content of X and Y? I tell you something about X, and I tell you something about Y, and these 2 events are in these 2 random variables are independent, then what can you tell me about the information content? What is the condition that you would want? You gain some information by knowing things about X, and you gain some information by knowing things about Y. So, what can you tell me? It should be the sum, right?

Because these two are independent events; so, whatever information I am getting from this random variable and this random variable which together enhancing my information, right. It is not cancelling out anything or is there is no common intersection there, right. If the 2 events were not independent then I would not expect this to hold because knowing something about the first event only tells me something about the second event, right, because they are dependent.

So, then that case the information gained would not be additive, ok. So now, let us see, I already made a case that this function which tells me the information gain is actually proportional to the probability, ok. So that means, this is what the input is going to be, right. And then what is the other condition that I want? This is a fair thing, right. I just replaced information content by a function, and I know that the function should depend on the probability, because that is what we have established here.

So, we know that the function depends on the probability, we still do not know what this f is exactly, but I am trying to impose some conditions on f. One condition of f, f is that this condition should hold, ok. Now let us look at this condition which I have underlined, this is f of is this fine because it 2 events are independent, you can write them as the joint probability as P of x into P of y. This is clear to everyone, right? You seem to be a bit lost Arvind clear, ok.

Now, what is happening here? I have a function f of a into b and that is actually equal to f of a plus f of b, what family of functions do you know which has this characteristic? Log, right, that is why log is a good choice for this. That is why information content is going to be the log of the probability, but I wanted to be inversely proportional, right so, it will be log of 1 by the probability, ok. So, that is why information content of this thing is; so, you see this how did we arrive at this log formula.

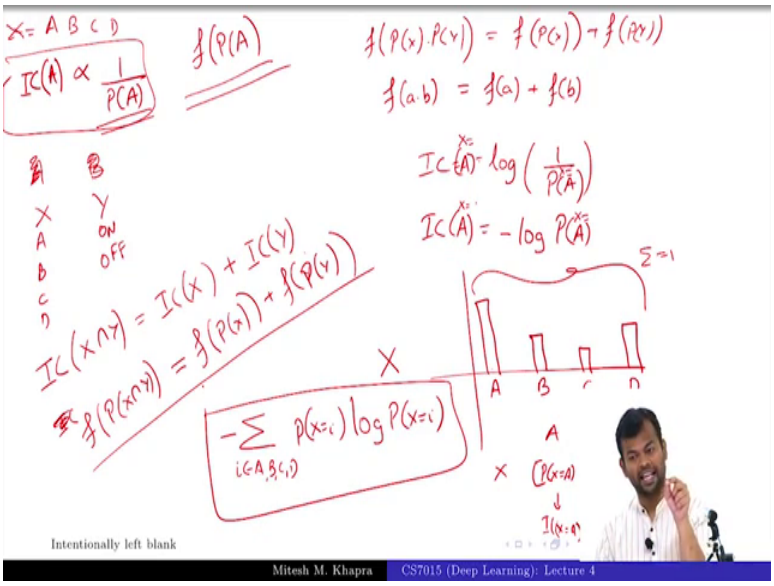
And this log can just be to any base, it does not matter. So, all of you get how we arrive at the formula for information contained. Now just give me a minute I need to think of what is the next thing that I have to say, ok. And so, we have found out the information content of one of these events happening. Which was the x taking on the value A.

Now, let us think of this random variable x. So, here actually I should have said x equal to a probability of x equal to A, ok. It makes sense because the random variable is x, and the event is x taking on the value A, how much information content is in that. So, if I know that x was A how much will I be surprised by it, ok. Now let us take this event, this random variable x which can take on values A B C and D as I said with each value there is a probability associated with it, such that this sums to 1. Now, I did not need to draw this diagram, ok. I should (Refer Time: 21:01).

So, X is a random variable which can take these 4 values, which each of these values, I have a probability associated, ok. So, these are the values these are the probability values. Now what do I also have? I have the information content associated with each of these, right. And the information content actually tells me the surprise of that evening. Now if I ask you what is the entropy of this random variable X? So, remember I had this case where I was betting I am with every poor outcome I had a value associated with it? I had the same situation here, with every outcome have a probability, and I also have a value associated with this, and the value is the information content.

Now, if I ask you what is the entropy or the expected information content of this random variable, then how will you compute that? I am asking you for an expectation.

(Refer Slide Time: 22:16)



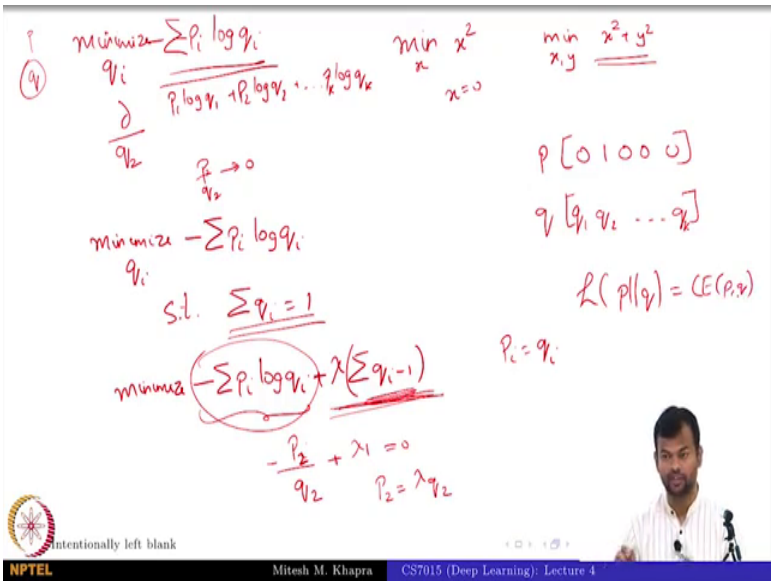
So, I will compute summation i belonging to A B C D, P of x equal to i information can take what is the formula for that.

Student: minus (Refer Time: 22:29).

Minus I will just take the minus outside, ok. So, this quantity is called the entropy of the random variable, right. It is the expected information content in the random variable. Now if you see what would be the expect entropy of a random variable if it is corresponding to a certain event? That means, say the sun rises always in the east, right? So, what is going to be the entropy of that 0? Why, you will have one of the sums in that summation as 1 log 1, right. And every other sum would be 0 into log of something.

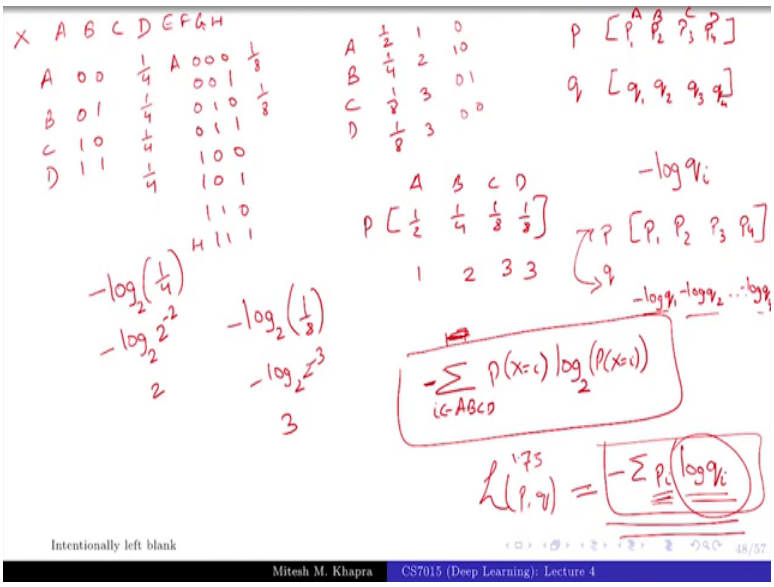
So, 0 into anything is going to be 0, even though that quantity is not defined 0 into anything is going to be 0. So, the total entropy is going to be 0, ok. So, this is entropy, now what is it that we are actually interested in? Cross entropy, so, we have not gone there yet, ok. So, we need to perhaps add one more slide, so far everything is clear, ok. So now, we are interested in something known as cross entropy.

(Refer Slide Time: 23:36)



So, there again the situation is that there is something which is the true distribution, and something which is the predicted distribution. Now, actually before going there so, let me just erase this off, how many of you have thought that entropy is related to the number of bits that you need to transmit something? Do you know why that connection exists? No, now again let us think of this that you are trying to transmit a message.

(Refer Slide Time: 23:59)



And that message is again a random variable, which can take on 4 values A B C D. So, think of these as 4 commands that we are trying to send to someone, right. And then

based on that command someone will take some action. Now in the digital case how will you transmit this? Encode it to bits, so, what is the encoding that you will use? 0 0 yeah we will come to that, 0 1 1 0 1 1. So, how many bits are you actually using for every message?

2 bits, ok, for every bit so, maybe this is A this is B this is C and this is D. So, for every message you are using 2 bits. Let us see actually what when you are doing this, what are you actually assuming. So, actually assuming that all of these are equally likely, if all of these are equally likely can you tell me the information content of any of them? It is going to be minus log of 1 by 4, ok. That is actually equal to minus log and this is to the base 2, ok 1 by 4 is 2 raise to minus 2, that is equal to 2.

So, the information content is actually equal to the number of bits that you are going to use to transmit that message. Now let us see if this is just in this special case or in a different case also. Suppose this could take 8 values, how many bits would you use? 3 bits right? So, you will have 0 0 0, 0 0 1, ok. And this would be A to H, now what are we actually assuming here? Each of these is equally likely, what is the probability? 1 by 8; what is the information content? 2 raise to minus 3, that is equal to 3.

So, the number of bits that you actually use to transmit something this, can you can talk of it in terms of the information content of that. Now suppose I want to transmit this over the long distance. So, I need to bit be a bit efficient in terms of number of bits that I use right. So now, in this one of these cases, suppose it is of the following form, right. That let us look at the case where x can take one of 4 values, and say let me just put the right values. So, I will say 1 by 2, 1 by 4, 1 by 8, 1 by 8, ok. Now what is the information content of each of these? 1 2 3 3, and this is the message that I am going to send.

So, what am I doing here? I am using a different number of bits depending on the probability of that event, why does this make sense? Why is this a smart thing to do? If you want to transmit something which you are going to transmit a lot of times, you better use less number of bits for that, and this is exactly what is happening here, A was having the highest probability, and you are using the lowest number of bits for that. Now what is the expected number of bits that I will use up? If it is a I will use one if it is B I will use 2 if it is C 3 and D 3.

So, what is the expected number of bits that I will use. Again I have the same situation, right, I want you to cast it into the same situation, I have the probability values, and with each of these guys, I have a cost or a value associated what is the cost? 1 bit, 2 bit, 3 bit, 3 bit. So, what is the expectation now? Can someone compute the expectation, 1.75 actually let me just write it down, it would be again I belonging to A B C D, P of x equal to i into the number of bits that you will use. So, that is just equal to log of log to the base 2 of P of x equal to i, minus 1, what does this quantity actually? This is the entropy we just saw this, A, this is the entropy of the random variable and what is it telling us actually that the entropy is 1.75.

So, what is the meaning of this actually? So, on average you will be needing 1.75 bits whereas, if you are assuming everything is equally likely on average you are using 2 bits, right? So, you see that on average you are making some savings here right? So, that is what the entropy tells you. If you know what the probability of these events is, then you better use that to decide the number of bits that you are going to use to send each of these.

So now let us complicate this a bit more, now we have the entropy. Now let us complicate it a bit more. So, there is some true distribution which exists. There is some true distribution from which these messages are coming, right? But you do not know what that true distributions, we never know the true distribution that is the entire problem that we have been dealing with in machine learning.

So, what you will do is we will somehow try to predict this distribution, and this then the and the recipe that you will use is the same as that I used for the example where I had an urn right? So, there are these thousands of 10,000's of messages which has going to keep coming on, you do not have access to the entire stream. But you have seen some thousand of those messages just, as I had peeped into the urn, and I had seen some balls and I had made an estimate; that I think based on these messages that I have seen so far I think these are the actual probabilities.

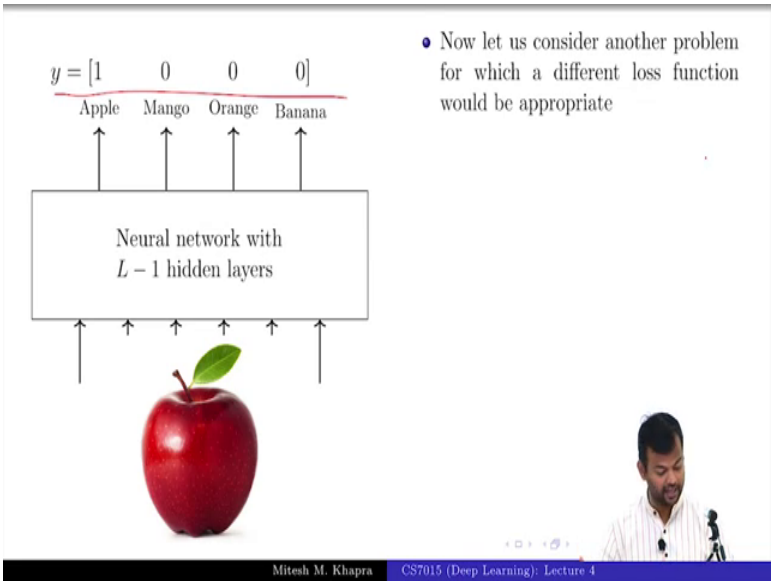
So, the true probabilities are say p 1, p 2, p 3 and p 4 corresponding to A, B, C and D, I do not know what this 2 properties are, but I can estimate them looking at some samples or basically using my domain knowledge, right. Maybe I would know that if one of these messages is stopped. And I am actually trying to talk to a computer or a computer

program, that maybe stop is something which are used very rarely only at the end of the program or something. So, you have some either some domain knowledge or based on some samples I can estimate the value of this probability.

And I just try to relate it to the exact example of urns, where you had these 10,000's of balls, but you could not see all of them, you sampled some and estimated a probability. Here again there is a continuous flow of message you cannot have access to all of these, because they are going to continue. But I have seen some of those and based on that you estimate these probabilities. Now based on this estimation how many bits?

So now this is the estimation that we have. Now based on this you will decide the number of bits that you will use for each of these messages, right. Because you have some estimate so, you want to be smart you do not want to keep 2 bits for all of them. So, you will just say that I will use log qi bits for the i'th message.

(Refer Slide Time: 31:23)



This is fair thing because I know that the information content is proportional to the probability. In fact, it is exactly given by this formula, minus log of qi. So, based on my estimated probability I am going to do this, ok. And this is the number of bits that I have reserved. Now do you see a problem with this? This is my estimation, but the data is actually going to come from the true distribution. It is not going to follow the distribution q; it is going to follow the distribution p.

So now, what is actually happened is this, right? This is the situation that we are dealing with. We have p which was a true distribution, that is the rate at which the data will come, but with each of these events the value that we have associated is now related to q, because q is what I have access to, I do not have access to p, I just have access to q so, I have associated a value based on q, does this make sense? I should have actually used log p 1 bit's log p 2 bits and log p 3 bits, but I do not know what p 1, p 2, p 3 are.

I just estimated them based on some samples so, that is q 1, q 2, q 3. And these are the number of bits that I am using. Now if I have to compute the expectation how will I do it? I have to use p because that is the true distribution from which the data is coming.

So, what would the expectation now look like? Everyone gets this? the actual probabilities are this, but because I am poor at estimating them I ended up associating these values, which could be wrong, right? Because I would have overestimated the probability of one of these messages and hence I have reserved lesser bits for that or underestimated the probability of one of these events, and hence reserved more bits for that or vice versa.

I could have assigned a wrong number of bits to them, right. p no. So, do we have access to p in the sense. So, someone knows that, right? I mean there is a again in the same case as in the label case, right? We have access to the true p there, and we are estimating a q, when we are given these images for the training data, we know that the distribution is 0 1 0 0 if the image is B for banana.

Student: (Refer Time: 33:49).

Then it is validated, right. So now, this is what is this quantity called? This is called the cross entropy, ok. You get why it is called the cross entropy because now you have two different distributions involved here, ok. You have the q distribution based on which you made your decisions, you assign values to these events based on the q distribution, but the true distribution is the p distribution.

So, the actual number of bits that you use up on average is going to be based on the true probability, they try to understand that. Now what will happen is for event A you have assigned a certain number of bits. Now how many bits will get used up it depends on.

The actual probability of p if that, message is repeated many times then that is how this summation would be computed.

So, this is called the cross entropy, but now why is this the difference between 2 distributions? That is what we wanted. Given 2 distributions we wanted to be able to find the difference between them. Now am telling you that cross entropy is a way of finding that difference. Why is it so? So what would you want this difference to what is the property that you would want this difference to have? If p is equal to q, then if p is equal to q then;

Student: (Refer Time: 35:08).

Not 0 maybe it should take the lowest possible value, right. So, this function, right. This is actually telling you loss of p comma q, right? This is what this is and we are calling it as the cross entropy, this function you take it is minimum value when p is equal to q, right. Because now at that point you are not really making any loss that is the best you could have done. Does this function take it is minimum value when p is equal to q? Yes, why? How is that obvious, but why there could be something else which is lower than the actual entropy, right? Why, how you have to we are trying to minimize something. So, you have to give me answer.

So, yeah so, let us do that, ok. So, how many of you it is obvious that q is the answer? I mean the answer is p is equal to q it is not, ok. Now this is the part which I am a bit worried about, but I will just do it anyways. So, let me see how do I put this ok. So, remember that we had a p and we had a q, and we want to find a q such that this quantity is minimized. That is what our objective is.

So, we want to minimize this with respect to qi, ok. Now how do you find the minimize suppose I have this problem? How do I find the minimum value? How do I find the value of X which minimizes this, take the derivative, and set it to 0, ok. And then in this case I will get x equal to 0 is that value. Can I do the same thing here. And suppose it was this so now, this is a function of 2 variables, again I could do the same thing, I could take the partial derivatives and set them to 0.

And I will get the minimum value. Now here this is actually a function of how many variables? K in general right, so, q 1, q 2, q 3 up to qk, ok. Now can you try doing the

same thing can you can you take the derivative and set it to 0. This is again a sum, right. It is very similar to this situation. It is actually let me just write it down, it is p 1 log q 1 plus p 2 log q 2 up to p k log qk.

Now I want to take the derivative with respect to one of these guys; say, q 2, what would it be? p 2 by q 2 is equal to what will I do? That is the derivative p 2 by q 2, I will set it to 0, do I get anything? What is it that I am doing wrong here? There is something that I am deliberately doing wrong. Is this an unconstrained optimization problem? There is a constraint on the variables, what is the constraint? So, why my true optimization problem is minimize with respect to q i's; such that summation q i's equal to 1.

Do you know an easy way of dealing with these problems? How many of you know the Lagrange multiplier. How will I use it here? What will my objective function become? Then summation of qi minus 1 lambda then minus ok, How many of you understand the intuition behind this, that is a good answer. Now let us let me try to explain why this makes sense, right. This is the constraint that we have to operate within this constraint.

What I have done is I have taken the so now if the constraint is not satisfied what will happen to this quantity if the constraint is not satisfied? That means, my summation is not equal to 1 that is what means whether the constraint is not satisfied, what will happen to this quantity, it will be a non-zero quantity, right. Fine then what will happen to my overall objective? And I think we have made a mistake this should be plus. I should add it, right. Should be plus no it does not oh the lambda can be, ok sorry, so, let me assume this is plus.

So, what I am trying to do is that, this is my objective function which I am trying to minimize. I have added another quantity to it, if this quantity is not equal to 0, then I will not be the absolute minimum, I will be at the minimum plus something right, but if this quantity if the constraint is satisfied, then this quantity will go to 0, then am actually at the minimum of the function, do you get this, right? So, this is the function that I want to minimize, I have added some quantity to it. Now, that quantity is actually related to the constraint that I do not want to violate. If I violate the constraint, this is going to be non-negative, right?

So, whatever minimum value I achieved, I will be slightly higher than that, because some non-negative value has got added to it, ok. Is that fine? But if the constraint is satisfied,

then I can achieve the minimum value. So, that is roughly the intuition behind using this Lagrangian multiplier, it is a very crude intuition, but there is of course, a lot of math behind that, but I am just giving you the intuition behind this which one.

Student: (Refer Time: 41:39).

Yeah that is what you could adjust the lambda, and ensure that it is not negatively, ok. So now, now can you do the same thing can you equate this to 0, can you take the derivative and equate to 0? What will you get now? This term will give you p I by q 2 as before, oh sorry p 2 by q 2 as before plus lambda times yeah plus lambda times one, ok. Fine so, equal to 0. So, then what will you end up getting? p 2 is equal to I think it is something wrong here, now this should be minus p 2 by k.

So, p 2 is equal to lambda times q 2, ok. And then further actually you can show that lambda is going to be equal to 1, how can you show that your constant is fine. So, do you see how we will get lambda equal to 1. So, what does it actually tell you, then p 2 is equal to q 2; that means, all. In fact, you can show that all p i's are equal to q i's; that means, the distribution p is equal to distribution q.

So, this cross entropy term will be minimized when your true distribution, or when your plated distribution is the same as the true distribution, and hence it captures the difference between the two, ok. And that is exactly what we were interested in, we were interested in a quantity which can allow us to capture the difference between the true difference between a true distribution and the predicted distribution.

So, we have arrived with that quantity and that quantity is cross entropy. So, therefore, for all our classification problems where we have this scenario that we are given the true distribution where all the masses focused on one of the labels, and you are estimating a distribution where you could give non zero quantities to many of those. And you want to find out how wrong your estimates were with respect to the true distribution, you can use cross entropy as a measure for that right. So now, your loss function which you wanted to depend on the difference between p and q, it can just be the cross entropy between p and q.